



UNIVERSIDAD DE TALCA
FACULTAD DE INGENIERÍA
ESCUELA DE INGENIERÍA CIVIL EN COMPUTACIÓN

**Sistema IoT de asistencia con autenticación
biométrica e integración empresarial**

AGUSTIN EDUARDO MEZA MELLA

Profesor Guía: RICARDO PÉREZ GUZMÁN

Memoria para optar al título de
Ingeniero Civil en Computación Curicó, Chile— Noviembre, 2026



UNIVERSIDAD DE TALCA
FACULTAD DE INGENIERÍA
ESCUELA DE INGENIERÍA CIVIL EN COMPUTACIÓN

**Sistema IoT de asistencia con autenticación
biométrica e integración empresarial**

AGUSTIN EDUARDO MEZA MELLA

Profesor Guía: RICARDO PÉREZ GUZMÁN

El presente documento fue calificado con nota: _____

Memoria para optar al título de
Ingeniero Civil en Computación Curicó, Chile— Noviembre, 2026

TABLA DE CONTENIDOS

Tabla de Contenidos	1
Índice de Figuras	4
Índice de Tablas	6
Resumen	7
1 Introducción	9
1.1 Contexto	9
1.2 Descripción del problema	10
1.3 Propuesta de solución	11
1.4 Objetivos	11
1.4.1 Objetivo general	12
1.4.2 Objetivos específicos	12
1.5 Alcances	13
1.6 Resumen capítulo	14
2 Marco teórico	15
2.1 Conceptos básicos	15
2.1.1 Conceptos globales	15
2.1.2 Conceptos de hardware y software	16
2.2 Tecnologías utilizadas	18
2.2.1 Microcontrolador ESP32-CAM	18
2.2.2 Lector de huella digital AS608	18
2.2.3 Protocolos de comunicación HTTP	19
2.2.4 Protocolo de comunicación MQTT	19
2.2.5 Contenedores Docker	19
2.2.6 Sistemas de bases de datos	19
2.2.7 Sensor PIR (HC-SR501)	20
2.2.8 Framework DeepFace	20
2.2.9 Broker Mosquitto en contenedor Docker	21
2.3 Estado del arte	21
2.3.1 Soluciones basadas en hardware	21
2.3.2 Soluciones basadas en software	22
2.4 Metodologías	23
2.4.1 Metodología de desarrollo	23
2.4.2 Metodología de evaluación	25

2.5	Resumen del capítulo	27
3	Metodologías	28
3.1	Metodología de desarrollo	28
3.1.1	Protipado Evolutivo Incremental	28
3.2	Plan de trabajo	30
3.3	Metodología de evaluación	31
3.3.1	Pruebas de integración	31
3.3.2	Pruebas de usabilidad	32
3.3.3	Pruebas de confiabilidad y disponibilidad	33
3.3.4	Pruebas de caja negra	34
3.3.5	Pruebas unitarias automatizadas	34
3.3.6	Resultados esperados de las pruebas	35
3.4	Resultados Esperados de los Objetivos Específicos	39
3.5	Resumen del capítulo	39
4	Desarrollo e implementación	41
4.1	Diseño de software	41
4.1.1	Arquitectura física	41
4.1.2	Arquitectura lógica	42
4.2	Implementación	44
4.2.1	Iteración 1: Integración de hardware y servidor embebido	44
4.2.2	Iteración 2: Almacenamiento local y modo offline	58
4.2.3	Iteración 3: Backend, base de datos y comunicación	64
4.2.4	Iteración 4: Reconocimiento facial, anti-spoofing y cifrado biométrico	75
4.2.5	Iteración 5: Autenticación, multi-tenant y enrolamiento de dispositivos	84
4.2.6	Iteración 6: Módulo antifraude y validación previa a la captura	91
4.2.7	Iteración 7: Panel web backend e integración con ERP	96
4.2.8	Iteración 8: Sincronización diferida, logs y cierre del proyecto	104
4.3	Resumen del capítulo	111
5	Análisis de resultados	113
5.1	Costo total del prototipo	113
5.2	Resultados de encuestas SUS	113
5.2.1	Usuarios no técnicos	114
5.2.2	Usuarios técnicos	115
5.2.3	Análisis comparativo	115
5.3	Métricas de reconocimiento facial	115
5.3.1	Tiempo de respuesta del ESP32-CAM	115

5.3.2	Precisión del reconocimiento	116
5.4	Resultados de pruebas de estrés offline	116
5.4.1	Escenario 1: Desconexión total	116
5.4.2	Escenario 2: Conectividad intermitente	116
5.4.3	Escenario 3: Reconexión prolongada	117
5.5	Resumen del capítulo	117
6	Bibliografía	118
	Anexo A: Preguntas complementarias de usabilidad	122

ÍNDICE DE FIGURAS

3.1	Metodología prototipado evolutivo con enfoque incremental	29
4.1	Arquitectura física	42
4.2	Arquitectura lógica	43
4.3	Componentes del ESP32-CAM — hardware y firmware.	44
4.4	Mockup menú principal en el ESP32-CAM	46
4.5	Mockup panel de configuración Wi-Fi	46
4.6	Mockup registro usuario en el ESP32-CAM	47
4.7	Mockup usuarios almacenados en el ESP32-CAM	47
4.8	Mockup visualización de turnos en el ESP32-CAM	48
4.9	Mockup creación de turnos en el ESP32-CAM	48
4.10	Mockup asignación de turnos en el ESP32-CAM	49
4.11	Flujo operacional para el registro de asistencia.	50
4.12	Conexiones entre el ESP32-CAM y el lector de huellas AS608.	51
4.13	Fragmento de código para inicializar el lector AS608 utilizando UART.	52
4.14	Asignación de rutas para el servidor web embebido.	53
4.15	Menú principal del sistema web embebido.	54
4.16	Interfaz de configuración Wi-Fi.	55
4.17	Vista para el registro de personas en el dispositivo.	55
4.18	Vista de asistencias locales almacenadas en el dispositivo.	56
4.19	Vista de personas registradas en el dispositivo.	56
4.20	Vista de gestión de turnos en el dispositivo.	57
4.21	Vista de gestión de asignaciones en el dispositivo.	57
4.22	Funciones para lectura y escritura de archivos JSON en LittleFS.	61
4.23	Máquina de estados del registro biométrico en el ESP32-CAM.	62
4.24	Diagrama entidad-relación de la base de datos (14 tablas).	68
4.25	Flujo MQTT pub/sub — tópicos, direcciones y QoS entre el ESP32-CAM, el broker Mosquitto y el backend.	69
4.26	Arquitectura de despliegue Docker: servicios, puertos, volúmenes y dependencias en la red <code>teleasist_network</code>	73
4.27	Diagrama de secuencia del enrolamiento biométrico completo.	77
4.28	Marcaje automático — centinela facial con identificación 1:N y push asíncrono a ERP.	79
4.29	Pipeline de cifrado biométrico	82
4.30	Diagrama de secuencia del login multi-empresa con JWT.	87

4.31 Diagrama de secuencia de la integración ERP — push asíncrono con field mapping.	99
4.32 Diagrama de actividad: sincronización bidireccional entre los modos online y offline del ESP32-CAM.	106

ÍNDICE DE TABLAS

3.1	Plan de trabajo del proyecto: 8 iteraciones de 3 semanas cada una (24 semanas total)	30
3.2	Cuestionario SUS estándar (Brooke, 1996) - Traducción al español . . .	33
3.3	Resultados esperados de las pruebas de integración	36
3.4	Resultados esperados de las pruebas de usabilidad (SUS)	36
3.5	Resultados esperados de las pruebas de confiabilidad y disponibilidad .	37
3.6	Resultados esperados de las pruebas de caja negra	38
3.7	Resultados esperados de las pruebas unitarias automatizadas	38
5.1	Desglose de costos del prototipo (pendiente de completar con precios de mercado)	113
5.2	Resultados de encuesta SUS - usuarios no técnicos (pendiente de aplicar encuestas)	114
5.3	Resultados de encuesta SUS - usuarios técnicos (pendiente de aplicar encuestas)	115
5.4	Tiempos de respuesta del ciclo de marcación facial (pendiente de medir)	116
5.5	Métricas de precisión del reconocimiento facial (pendiente de evaluar) .	116
5.6	Resultados de pruebas de estrés offline (pendiente de ejecutar)	117

RESUMEN

El presente proyecto aborda el desarrollo de un sistema IoT de control de asistencia biométrica, diseñado para responder a las limitaciones actuales de las soluciones comerciales y cumplir con los requisitos técnicos y legales establecidos por la Dirección del Trabajo de Chile, en particular la Resolución Exenta N.º38 (2024), que regula los sistemas electrónicos de registro de asistencia. El trabajo propone una alternativa modular, de bajo costo, código abierto y adaptable, orientada principalmente a pequeñas y medianas empresas que requieren un sistema flexible, con capacidad de funcionamiento autónomo y con posibilidades reales de integración con plataformas ERP.

La solución desarrollada utiliza como dispositivo central un ESP32-CAM, microcontrolador que incorpora cámara, conectividad Wi-Fi y capacidades de procesamiento suficientes para realizar captura de imágenes y ejecutar tareas de apoyo asociadas a la autenticación. A este módulo se integra un lector de huellas AS608, que permite una verificación biométrica complementaria frente a situaciones donde la captura facial no sea óptima. El sistema se estructura de manera híbrida, permitiendo registrar asistencia tanto mediante reconocimiento facial como mediante verificación dactilar, de acuerdo con las condiciones de operación.

Uno de los principales aportes del proyecto es la capacidad del dispositivo para operar en modo offline, utilizando SPIFFS como sistema de almacenamiento local para gestionar usuarios, turnos, asignaciones y asistencias en formato JSON. Cuando la conectividad se restablece, el sistema ejecuta un proceso de sincronización diferida que garantiza la integridad de los datos y la continuidad operativa del dispositivo, incluso en entornos con conectividad inestable o intermitente, situación común en empresas pequeñas o en faenas remotas. Asimismo, la arquitectura lógica y física diseñada permite incorporar módulos de seguridad adicionales, como un sensor PIR combinado con anti-spoofing por software, destinado a detectar intentos de suplantación mediante fotografías o pantallas.

El desarrollo del proyecto se llevó a cabo utilizando una metodología de prototipado evolutivo incremental, seleccionada por su capacidad para integrar hardware, firmware, backend y módulos biométricos de manera progresiva. A lo largo de ocho iteraciones se construyeron y validaron componentes funcionales independientes que luego fueron integrados: capturas biométricas, almacenamiento local, comunicación HTTP/MQTT, panel web embebido, mecanismos antifraude, backend con base de datos PostgreSQL y módulo de comunicación estándar para ERP. Cada iteración incluyó análisis, diseño, implementación y pruebas, permitiendo detectar fallas tempranas y optimizar el prototipo de manera continua.

La etapa de evaluación incorporó pruebas de integración, usabilidad, confiabilidad y

disponibilidad, fundamentales para determinar el desempeño del sistema en escenarios reales. Las pruebas de integración verificaron la correcta comunicación entre los distintos módulos del prototipo, incluyendo la interoperabilidad entre el ESP32-CAM, el lector AS608, el almacenamiento SPIFFS, el backend y la base de datos. Las pruebas de usabilidad, aplicadas tanto a usuarios técnicos como no técnicos mediante el método System Usability Scale (SUS), permitieron evaluar la claridad de las interfaces y la facilidad de uso del sistema. Complementariamente, se desarrollaron pruebas de confiabilidad y disponibilidad destinadas a medir la robustez del sistema frente a desconexiones, reconexiones y funcionamiento prolongado sin supervisión.

El sistema resultante es un prototipo funcional que permite registrar asistencia, almacenar datos localmente, sincronizar registros con un backend cuando existe conectividad, operar de manera continua bajo distintos escenarios y comunicar información hacia sistemas ERP mediante una API estándar. A través del análisis de los resultados obtenidos, se concluye que la solución propuesta es técnicamente viable, cumple con los requisitos de funcionamiento planteados en los objetivos específicos y representa una alternativa accesible y adaptable frente a las restricciones de los dispositivos biométricos comerciales actuales.

1. Introducción

En el siguiente capítulo se presenta el contexto, la problemática actual y la propuesta de solución que fundamentan el desarrollo de este sistema IoT de control de asistencia biométrico.

1.1 Contexto

El control de asistencia laboral es un requisito obligatorio para todas las empresas en Chile, conforme a lo dispuesto por la Dirección del Trabajo. Este registro puede realizarse de manera manual o mediante sistemas digitales que cumplan con la Resolución Exenta N°38 (2024) [1], la cual establece los requisitos técnicos y legales que deben cumplir los sistemas electrónicos para ser considerados válidos ante la autoridad.

El cumplimiento de estas normas no solo busca garantizar la trazabilidad y transparencia de las jornadas laborales, sino que también impacta directamente en el cálculo de remuneraciones y en la gestión administrativa de la empresa[2], si no se lleva un registro que cumpla las características presentes en la norma, tales como la inclusión de métodos que garanticen el registro de asistencia al trabajador. Por ello, muchas organizaciones optan por integrar sus sistemas de asistencia con plataformas ERP (EntERPrise Resource Planning), las cuales centralizan distintos procesos como recursos humanos, inventario, finanzas y contabilidad dentro de un mismo entorno[3].

Esta integración permite que los datos de asistencia alimenten automáticamente los módulos de remuneraciones o de gestión de personal, evitando errores y reduciendo los tiempos administrativos[4]. En este contexto, los sistemas de asistencia pueden ser parte del propio ERP o soluciones externas especializadas, las cuales se comunican mediante APIs o servicios web.

Las máquinas de control de asistencia digital, por su parte, permiten registrar las marcaciones de entrada y salida de los trabajadores de forma autónoma[5]. Sin embargo, muchas de las soluciones disponibles en el mercado presentan limitaciones: operan únicamente en red local, exportan datos en formatos cerrados (como CSV o Excel) y

no siempre permiten integrarse de manera flexible con otros sistemas, como los ERP corporativos.

Este proyecto propone el desarrollo de un sistema IoT de control de asistencia basado en ESP32-CAM, capaz de realizar reconocimiento facial y enviar los registros al backend, permitiendo su posterior integración con un ERP o funcionamiento independiente en modo local.

1.2 Descripción del problema

Aunque la normativa chilena exige que los sistemas de control de asistencia garanticen trazabilidad, integridad y disponibilidad de los registros de acuerdo con la Resolución Exenta N.º 38 (2024), una parte importante de los dispositivos utilizados actualmente especialmente los relojes de control biométricos de bajo costo presentan limitaciones que dificultan su adopción en empresas pequeñas y medianas. Muchos de estos equipos requieren conexión permanente a internet, operan con software propietario o solo permiten exportar datos en formatos estáticos como CSV, lo que genera dependencia de procesos manuales y restringe la interoperabilidad con otros sistemas.

Al mismo tiempo, la mayoría de los ERP utilizados por las empresas no incorpora módulos nativos para capturar asistencia desde hardware, por lo que su integración con dispositivos externos resulta costosa o directamente incompatible. La ausencia de APIs abiertas, la falta de mecanismos de sincronización automática y la poca flexibilidad para adaptar el hardware al contexto operativo dificultan la incorporación de tecnología biométrica en entornos reales.

Estas limitaciones se vuelven más evidentes en escenarios donde la conectividad es inestable. Muchos relojes de control no pueden almacenar marcaciones localmente de manera segura ni realizar una sincronización diferida una vez restablecida la red, lo que expone a las empresas a la pérdida de datos y al incumplimiento normativo. Asimismo, la mayoría de las soluciones comerciales no permite personalizar la lógica de validación biométrica, integrar módulos anti fraude ni extender el sistema a necesidades particulares, como gestión de turnos o funcionamiento híbrido.

Actualmente no existe en el mercado una alternativa accesible, modular y adaptable que permita capturar asistencia mediante biometría (huella o reconocimiento facial), operar de forma autónoma sin internet, almacenar registros en el dispositivo, detectar intentos de suplantación y sincronizar datos con sistemas ERP mediante APIs estándar. Esta brecha tecnológica afecta especialmente a organizaciones que requieren una solución flexible, de bajo costo y consistente con las exigencias de la autoridad laboral.

En síntesis, el problema consiste en la falta de un reloj control biométrico capaz de operar offline, almacenar y sincronizar marcaciones de manera segura, y conectarse a

sistemas ERP mediante interfaces estándar cumpliendo la normativa vigente.

1.3 Propuesta de solución

La propuesta consiste en diseñar e implementar un prototipo de control de asistencia utilizando un módulo ESP32-CAM junto con un lector de huellas dactilares. El ESP32-CAM es una placa de bajo costo que incorpora una cámara, conectividad Wi-Fi y Bluetooth, lo que permite capturar imágenes, procesarlas y comunicarse con otros sistemas sin necesidad de equipos adicionales.

El objetivo es que el dispositivo pueda registrar la asistencia de los trabajadores mediante reconocimiento facial y huella digital, funcionando tanto con conexión a internet como sin ella. En modo offline, el equipo almacenará temporalmente los registros en una tarjeta microSD o en su memoria interna, lo que le permitirá operar de manera autónoma durante varios días o incluso semanas, dependiendo de la capacidad de almacenamiento configurada. Cuando se recupere la conexión, los datos se sincronizarán automáticamente con un servidor web o con un sistema ERP, según la configuración establecida por el usuario.

El lector de huellas permitirá complementar la verificación facial, ofreciendo una alternativa en lugares donde las condiciones de luz o el ángulo de la cámara dificulten el reconocimiento.

El prototipo contará además con una interfaz web integrada, accesible desde la misma red local, que permitirá configurar parámetros de red, administrar usuarios y definir el destino de los datos. Junto con ello, se desarrollará una documentación técnica de apoyo, donde se describirá el funcionamiento general del sistema, el formato de los registros generados y las opciones de integración con diferentes plataformas ERP. Esta integración se realizará mediante un endpoint o API estándar, lo que permitirá la conexión con cualquier sistema ERP que acepte intercambio de datos a través de solicitudes HTTP o JSON, brindando flexibilidad sin depender de un proveedor específico.

Finalmente, para mejorar la seguridad en el proceso de marcación, se incorporarán métodos que eviten la suplantación de identidad, impidiendo que se utilicen fotografías o imágenes impresas para registrar asistencias falsas.

1.4 Objetivos

En el siguiente apartado se procederá a presentar el objetivo general del proyecto, junto con los objetivos específicos que guiarán el desarrollo y evaluación del prototipo.

1.4.1 Objetivo general

Desarrollar y validar un prototipo funcional de sistema IoT de control de asistencia de código abierto, basado en un microcontrolador ESP32-CAM, que permita evaluar la viabilidad del registro laboral autónomo (offline) y la sincronización automatizada con plataformas empresariales en pequeñas y medianas empresas.

1.4.2 Objetivos específicos

Objetivo específico 1

Diseñar la arquitectura física y lógica del prototipo IoT, seleccionando componentes que garanticen el funcionamiento autónomo y demostrando su viabilidad económica mediante un presupuesto de hardware total que no supere los **50 USD**.

Objetivo específico 2

Implementar el módulo de autenticación biométrica híbrida, logrando un tiempo de procesamiento inferior a **5 segundos** por marcación facial (incluyendo transmisión y respuesta del servidor) y garantizando una precisión superior al **90%** en la identificación correcta durante las pruebas de laboratorio.

Objetivo específico 3

Integrar el sistema de archivos local (LittleFS) para la operación offline, asegurando una capacidad de almacenamiento segura de al menos **1.000 registros** de asistencia en formato JSON, sin pérdida de datos ante simulaciones de corte de energía o desconexión de red.

Objetivo específico 4

Construir una interfaz web embebida y un panel de administración para la configuración del sistema, validando su facilidad de uso mediante la obtención de un puntaje igual o superior a **68 puntos** en la evaluación de usabilidad SUS (System Usability Scale) aplicada a un grupo de usuarios de prueba.

Objetivo específico 5

Desarrollar una API RESTful que permita la integración automatizada con sistemas ERP externos, verificando que el sistema logre sincronizar y transmitir el **100%** de los paquetes de datos generados en modo offline una vez restablecida la conectividad.

Objetivo específico 6

Evaluar la robustez y disponibilidad del prototipo integrado en escenarios de conectividad intermitente, comprobando la correcta ejecución de los mecanismos de reintento automático y analizando técnicamente las limitaciones del dispositivo para su futura escalabilidad comercial.

1.5 Alcances

A continuación se presentarán los alcances propuestos para este proyecto.

- Se diseñará la arquitectura de hardware y software del sistema, incluyendo los módulos de captura de imagen, registro de huella, almacenamiento local y comunicación con el servidor.
- Se implementará un prototipo funcional capaz de registrar asistencias mediante reconocimiento facial y huella digital, operando en modo offline con almacenamiento local y sincronización posterior.
- Se desarrollará una interfaz web embebida para la configuración del dispositivo, administración de usuarios y definición del destino de los datos.
- Se integrará una API RESTful para permitir la comunicación entre el dispositivo y un servidor o sistema ERP, utilizando formatos de intercambio JSON sobre HTTP.
- Se incorporarán medidas básicas de seguridad biométrica orientadas a evitar la suplantación de identidad mediante imágenes o fotografías.
- Se realizarán pruebas de funcionamiento offline, midiendo la autonomía del sistema (tiempo máximo de operación sin conexión y capacidad de almacenamiento).
- Se ejecutarán pruebas de integración entre los módulos de hardware, software y el ERP simulado, verificando el flujo completo de datos desde el registro hasta la sincronización.
- Se aplicarán pruebas de usabilidad para evaluar la facilidad de configuración y el nivel de satisfacción del usuario final con la interfaz embebida.
- Se efectuarán pruebas de confiabilidad e integridad, comprobando que los datos registrados sin conexión se transmitan correctamente una vez restablecida la conectividad.

- Los resultados se analizarán y documentarán para determinar la viabilidad técnica y operativa del sistema propuesto.
- El prototipo hará uso de la conexión a internet mediante Wi-Fi para realizar las sincronizaciones con los datos existentes en el servidor como así las funcionalidades de reconocimiento facial avanzado.

1.6 Resumen capítulo

El capítulo 1 presenta el contexto normativo del control de asistencia en Chile y expone las limitaciones de los sistemas biométricos actuales, especialmente su dependencia de internet y su baja interoperabilidad con ERP. Ante esta problemática, se propone un sistema IoT basado en ESP32-CAM con autenticación por rostro y huella, capaz de operar offline y sincronizar datos posteriormente. Además, se establecen los objetivos, alcances y lineamientos generales del proyecto.

2. Marco teórico

A continuación se presentan los fundamentos teóricos y tecnológico que facilitan la comprensión del proyecto

2.1 Conceptos básicos

En esta sección se definen los términos fundamentales que sustentan el desarrollo del sistema, abarcando tanto el contexto normativo y general del problema, como las herramientas específicas utilizadas para su resolución.

2.1.1 Conceptos globales

A continuación se detallan los conceptos generales vinculados a la gestión corporativa, los estándares biométricos y los paradigmas tecnológicos que enmarcan la propuesta.

- Control de asistencia: Proceso por el cual una empresa u organización registra las horas de entrada y salida de sus trabajadores.
- Sistema ERP: EntERPrise Resource Planning, sistema de gestión empresarial que permite procesos de una empresa u organización, tales como recursos humanos, contabilidad, finanzas, inventario, asistencia.
- Internet de las cosas (IoT): Paradigma tecnologico que permite la conexión entre dispositivos físico a través de redes de comunicación.
- Resolucion exenta n°38: Norma promulgada por la dirección del trabajo (DT) en 2024 (actualizada en 2025), que establece los requisitos obligatorios para los sistemas electrónicos de registro y control de asistencia.
- Biometria: La biometría corresponde al conjunto de técnicas utilizadas para identificar personas mediante características físicas o conductuales únicas, tales como el rostro o la huella digital. Este tipo de autenticación es ampliamente

utilizada en sistemas de control de asistencia por su alta precisión y dificultad de suplantación [36].

- Sincronización offline-online: Proceso mediante el cual un dispositivo opera sin conexión utilizando almacenamiento local y, una vez restablecida la conectividad, transmite los datos acumulados hacia un servidor o ERP. Es necesario en entornos con conectividad inestable.

2.1.2 Conceptos de hardware y software

En este apartado se describen los componentes físicos, protocolos de red, arquitecturas y herramientas de programación que conforman la infraestructura técnica del proyecto.

- ESP32-CAM: Microcontrolador basado en el esp32, con la inclusión de una cámara OV2640, conectividad WiFi y Bluetooth, además de contar con la capacidad de procesamiento local. [6]
- API (Aplicacion Programming Interface): Conjunto de métodos o puntos de conexión que permiten la conexión entre diferentes aplicaciones mediante protocolos de redes. [7]
- Backend: Parte del sistema encargada de la lógica del negocio, manejo de datos y comunicación con bases de datos. [8]
- Frontend: Conjunto de interfaces y funcionalidades con las que interactúa el usuario final. [9]
- Mqtt: Protocolo de comunicación ligero basado en el modelo publicador/suscriptor, diseñado para intercambiar mensajes de manera eficiente, utilizado entre dispositivos que cuenten con recursos limitados.[10]
- Javascript Object Notation (JSON): Formato de datos que se suele utilizar en entornos de desarrollo web para el intercambio de datos de manera legible tanto por las personas como las maquinas. [11]
- SPIFFS (SPI Flash File System): Sistema de archivos diseñado para microcontroladores que permite almacenar información en memoria flash mediante archivos. En este proyecto se utiliza para guardar usuarios, turnos, asignaciones y asistencias en formato JSON, permitiendo el funcionamiento offline [37].
- API RESTful: Estilo de arquitectura de servicios web basado en el protocolo HTTP, que permite la comunicación entre aplicaciones usando operaciones estándar como GET, POST, PUT o DELETE. Es utilizado para integrar el dispositivo IoT con sistemas ERP externos mediante intercambio de datos en formato JSON [38].

- Microservicios: Arquitectura de software basada en dividir un sistema en servicios independientes que se comunican entre sí. Esta estructura permite escalabilidad y facilita la separación entre el procesamiento de imágenes, backend y comunicación con ERP[39].
- Broker MQTT: Servidor encargado de recibir, gestionar y distribuir mensajes enviados por dispositivos IoT utilizando el protocolo MQTT. Actúa como intermediario entre el publicador (el ESP32-CAM) y los suscriptores (backend u otros servicios)[40].
- Sensor PIR (Passive Infrared Sensor): Sensor que detecta cambios en la radiación infrarroja del entorno para identificar la presencia de personas por su calor corporal. En este proyecto se utiliza como “portero” del sistema: activa los mecanismos de captura biométrica solo cuando detecta movimiento frente al dispositivo, reduciendo el consumo energético y evitando falsas activaciones. A diferencia de los sensores IR activos, el PIR no emite luz infrarroja sino que detecta pasivamente la que irradian los cuerpos.
- Anti-spoofing biométrico: Técnica de seguridad que permite detectar intentos de suplantación de identidad mediante fotografías, pantallas o máscaras frente a un sistema de reconocimiento biométrico. DeepFace, la librería utilizada en este proyecto, incorpora un detector de suplantación basado en redes neuronales convolucionales que analiza micro-texturas, reflectancia de la piel y profundidad en la imagen para distinguir entre un rostro real y una superficie plana.
- JWT (JSON Web Token): Estándar abierto (RFC 7519) que define un formato compacto para transmitir información de autenticación y autorización entre partes como un objeto JSON firmado digitalmente. En este proyecto se utiliza para proteger los endpoints del backend, transportando el identificador del usuario, su empresa y su rol, con firma HMAC-SHA256 y expiración de 24 horas.
- Arquitectura multi-tenant: Modelo de software donde una única instancia de la aplicación atiende a múltiples organizaciones (empresas), manteniendo los datos de cada una lógicamente aislados. En este proyecto se implementa mediante una columna `empresa_id` en las tablas principales del sistema, garantizando que los usuarios de una empresa no puedan acceder a los datos de otra. Adicionalmente, incluye un endpoint público de auto-registro que permite a nuevas empresas incorporarse al sistema sin intervención del administrador central, creando la organización, el usuario administrador y la asociación entre ambos en una única transacción atómica.
- Cifrado Fernet: Esquema de cifrado simétrico que utiliza AES-128 en modo CBC

con autenticación HMAC-SHA256, proporcionando confidencialidad e integridad. En este proyecto se emplea para cifrar los embeddings faciales (vectores de características biométricas) antes de almacenarlos en la base de datos, protegiendo los datos sensibles incluso ante accesos no autorizados a la base de datos.

- DeepFace: Framework de reconocimiento facial de código abierto que integra múltiples modelos de deep learning (Facenet, VGG-Face, ArcFace, entre otros) y detectores faciales (RetinaFace, MTCNN, OpenCV). En este proyecto se utiliza con el modelo Facenet para generar embeddings faciales de 128 dimensiones y con un detector configurable mediante variable de entorno, utilizando MTCNN por defecto por su velocidad (tres veces más rápido que RetinaFace con precisión comparable) para localizar rostros en las imágenes capturadas por el ESP32-CAM.

2.2 Tecnologías utilizadas

A continuación se presentarán todas las tecnologías que harán presente durante el desarrollo del proyecto. Dentro de estas

2.2.1 Microcontrolador ESP32-CAM

El ESP32-CAM es un dispositivo de desarrollo que integra un microcontrolador ESP32-CAM con conectividad Wi-Fi y Bluetooth, además de una cámara OV2640. La elección de este dispositivo se debe a su bajo costo, capacidad de procesamiento local, compatibilidad con el entorno de Arduino y su amplia comunidad de soporte. De las características que se pueden destacar encontramos la captura de imágenes para el reconocimiento facial, gestionar sensores biométricos y mantener conectividad directa con el servidor o la red local. Otras alternativas como la Raspberry Pi Zero W o Arduino MKR1000 fueron descartadas por su mayor consumo y costo implementación. [12]

2.2.2 Lector de huella digital AS608

El lector de huellas digital usa el sensor AS608, un lector de huella con la capacidad de registrar y verificar la identidad de los usuarios mediante patrones dactilares. La elección de este dispositivo fue por la compatibilidad con el ESP32-CAM, su bajo consumo y su resistencia a condiciones de luz ambiental en comparación a otros sensores ópticos tradicionales como el R307. El AS608 se comunica con el microcontrolador a través de una interfaz UART, y la integración de este se realiza mediante la biblioteca

Adafruit FingERPrint Sensor Library, lo que facilita el registro, búsqueda y verificación de huellas directamente desde el dispositivo IoT. [13]

2.2.3 Protocolos de comunicación HTTP

El protocolo de comunicación HTTP (Hypertext Transfer Protocol) es un protocolo cliente-servidor utilizado para transmitir información mediante solicitudes y respuestas que se transmite a través de los métodos GET, POST, PUT Y DELETE, generalmente utilizando formatos estructurados como JSON (JavaScript Object Notation). Permite la comunicación directa entre dispositivos y servidores, por lo que para este proyecto se le puede dar un buen uso para realizar la petición de información.[14]

2.2.4 Protocolo de comunicación MQTT

El protocolo de comunicación MQTT o Message Queuing Telemetry Transport es un protocolo ligero de mensajería diseñado específicamente para entornos IoT. Se base en la arquitectura de publicador-suscriptor, donde los dispositivos envían mensajes a un broker central que los distribuye entre los clientes suscritos a los distintos tópicos que pueden existir. Este modelo reduce el uso de ancho de banda, lo que lo hace ideal para redes con conectividad limitada o intermitente. [15]

2.2.5 Contenedores Docker

Docker es una plataforma de virtualización ligera que permite empaquetar aplicaciones junto las dependencias dentro de servidores virtuales aislados denominados contenedores. Cada contenedor ejecuta un entorno idéntico al que se utilizaría en producción, lo que facilita el despliegue, la portabilidad de los sistemas. Para el área de IoT y web, Docker es útil para ejecutar servicios como bases de datos, servidores o brokers MQTT de forma independiente y controlada, lo que lo hace útil para el trabajo al momento de levantar los servicios en distintos servidores. [16]

2.2.6 Sistemas de bases de datos

Un sistema de gestor de bases de datos es un conjunto de programas que permiten administrar bases de datos de manera estructurada. La principal función es proporcionar una interfaz entre los usuarios y los datos almacenados, garantizando integridad consistencia y seguridad de la información. Los SGBD relacionales, basado en el modelo propuesto por Edgar F. Codd (1970), organizan los datos en tablas vinculadas mediante relaciones, lo que facilita su consulta a través del lenguaje SQL. Lo SGBD más utilizados que se destacan son MySql, MariaDb, Oracle Database y PostgreSQL. Estos

sistemas ofrecen las características fundamentales, tales como el control de concurrencia, mecanismos de respaldo y recuperación, gestión de usuarios y soporte para transacciones atómicas (ACID). PostgreSQL es un sistema de base de datos relacional y orientado a objetos de código abierto, desarrollado inicialmente en la Universidad de California en Berkeley, caracterizada principalmente por ofrece un motor avanzado con soporte para consultas complejas, procedimientos almacenados, tipos de datos personalizados, índices avanzados y compatibilidad con JSON y XML, lo que permite la fácil integración con aplicaciones web e IoT. [17] Para el contexto del proyecto PostgreSQL se utiliza como base de datos, ya que tiene compatibilidad con entornos de desarrollo como python.

2.2.7 Sensor PIR (HC-SR501)

El sensor infrarrojo pasivo HC-SR501 es un módulo de bajo costo diseñado para detectar movimiento mediante la medición de cambios en la radiación infrarroja del entorno. Cuando una persona se sitúa dentro de su campo de visión (aproximadamente 6 metros de alcance y 120° de apertura), el sensor emite una señal digital HIGH. En este proyecto se utiliza como detector de presencia para activar los mecanismos de captura biométrica solo cuando hay una persona frente al dispositivo, optimizando así el consumo energético del lector de huellas AS608 y la cámara OV2640 durante los períodos de inactividad.

La elección del sensor PIR, en lugar del sensor IR activo originalmente planificado, respondió a dos factores: (1) la extrema limitación de pines GPIO disponibles en el ESP32-CAM tras la asignación del bus paralelo de la cámara, que hubiera requerido sacrificar otra funcionalidad para añadir el sensor IR; y (2) la disponibilidad del detector anti-spoofing integrado en DeepFace, que proporciona una capa de seguridad contra suplantación mediante software sin necesidad de hardware adicional [34][35].

2.2.8 Framework DeepFace

DeepFace es un framework de reconocimiento facial para Python desarrollado por Sefik Ilkin Serengil, que envuelve múltiples modelos de deep learning en una API unificada. Soporta los modelos Facenet, VGG-Face, ArcFace, DeepFace, DeepID y Dlib, así como los detectores faciales RetinaFace, MTCNN, OpenCV, SSD y Dlib. Para este proyecto se seleccionó el modelo Facenet por su equilibrio entre precisión y tamaño de embedding (128 dimensiones). El detector facial es configurable mediante variable de entorno, utilizando MTCNN por defecto por su velocidad superior (tres veces más rápido que RetinaFace), lo que resulta crítico para un dispositivo embebido como el ESP32-CAM.

DeepFace incluye además un módulo de anti-spoofing capaz de distinguir entre rostros reales y fotografías o pantallas, analizando micro-texturas y reflectancia. Como capa adicional de seguridad antes de invocar DeepFace, el sistema aplica un filtro

de calidad de imagen basado en la varianza del Laplaciano, que rechaza imágenes excesivamente borrosas o planas (posibles fotos de pantalla o papel) antes de llegar al modelo. Para optimizar el rendimiento, el modelo Facenet se precarga al iniciar el servidor y los embeddings de todos los trabajadores se mantienen en una caché en memoria con tiempo de vida configurable, eliminando la latencia de consultas repetitivas a la base de datos. El sistema soporta además múltiples fotos de referencia por persona (multi-encoding), almacenadas en una tabla dedicada `encodings_faciales`, lo que permite comparar cada captura en vivo contra todos los embeddings de referencia y seleccionar la menor distancia, mejorando la precisión ante variaciones de iluminación y ángulo.

2.2.9 Broker Mosquitto en contenedor Docker

Eclipse Mosquitto es un broker MQTT de código abierto, ligero y adecuado para dispositivos de bajo consumo. En este proyecto se despliega mediante Docker Compose, utilizando la imagen oficial `eclipse-mosquitto:latest`, exponiendo el puerto **1884** (externo, mapeado al 1883 interno del contenedor) para conexiones MQTT nativas y el puerto 9001 para WebSockets. El ESP32-CAM se conecta exclusivamente al broker mediante WebSockets seguros (`wss://`) sobre el puerto 9001, ya que el cliente MQTT de ESP-IDF (`mqtt_client`) solo soporta este modo de transporte en su versión utilizada en el firmware. La elección de WebSockets responde a la facilidad de atravesar firewalls y proxies corporativos en comparación con conexiones MQTT sobre TCP puro [16][41].

2.3 Estado del arte

A continuación se revisaran los sistemas y tecnológicas existentes relacionados dispositivos de control de asistencia biométricos y su integración con plataformas empresariales, con el propósito de identificar el estado que se encuentra tanto el mercado actual como los avances más relevantes e indicar las limitaciones que motivan el desarrollo de la propuesta de este proyecto.

Si bien los sistemas de control de asistencia es un campo del cual se tienen variados recursos y sistemas, la gran mayoría se encuentran ligados a un software propio, restringiendo a los sistemas ERP tener que pagar suscripciones a estos sistemas.

2.3.1 Soluciones basadas en hardware

En esta sección hará énfasis en aquellos dispositivos de reloj control biométricos que permiten registrar la asistencia de los trabajadores mediante la verificación de características únicas, como huellas digitales, reconocimiento facial, entre otras. Estos

equipos incluyen funcionalidad tales como almacenamiento interno, conectividad por red (LAN/Wi-Fi), a continuación se describirán los principales dispositivos utilizados en Chile.

1. GeoVictoria, relojes de control: GeoVictoria dentro de sus planes y productos se encuentran hardware biométrica con una plataforma en la nube para realizar el control de asistencia. Estos dispositivos pueden operar mediante una conexión LAN o 3G para zonas remotas. Sin embargo, dentro de sus limitantes encontramos que su API esta sujeto a restricciones comerciales y acceso mediante suscripción, lo que limita la personalización o integración libre. Además se tiene un ecosistema cerrado que dificulta modificaciones independientes por parte del cliente [18].
2. ZKTeco, terminales biométricas de asistencia: ZKTeco es una marca internacional con presencia en Chile y Latinoamérica en general, esta compañía ofrece terminales de control de asistencia "stand-alone" que combinan distintos métodos de registro tales como huella, rostro, PIN y tarjeta. Su limitante es que se encuentra cerrado totalmente a su sistema o quedan sin integración directa con un ERP, además ofrece exportación de datos mediante archivos csv siendo poco amigable para su uso en un ERP [19].
3. Dispositivos genéricos: Consideramos como dispositivos genéricos cualquier tipo de reloj control que se pueda adquirir en algún portal de venta web, que incluyan características tales como marcaje mediante huella o rostro. En base a lo anterior la gran mayoría de estos dispositivos tienen la capacidad de aplicar estas funciones pero al costo que su información solo queda en el dispositivo, teniendo métodos como exportación de datos por usb en un formato que solo se pueden visualizar y difícil de integrar a ERP.

2.3.2 Soluciones basadas en software

En el siguiente apartado se presentarán soluciones relacionadas al control de asistencia relacionados con plataformas web o servicios en la nube (Saas) que permiten gestionar la asistencia laboral sin requerir un reloj biométrico físico.

1. Talana/Buk: Talana y Buk son plataformas chilenas que ofrecen módulos de gestión de asistencia, turnos, entre otras características mediante aplicaciones web, en ambas aplicaciones se encuentran en la misma situación, con dependencia de conectividad a internet, ausencia de hardware biométrico físico, y costos asociados a suscripciones web [20][21].

2. Defontana/Softland: Defontana y Softland son empresas dedicadas a implementar sistemas ERP para las empresas, dentro de sus funcionalidades integran control de asistencia, permitiendo registrar las jornadas de trabajo, calculo de horas y conexion con modulo de remuneraciones. La gran limitante de estas aplicaciones web que estan ligadas 100% a su uso web, es decir, se debe contratar el ERP completo [22][23].

De los productos y servicios presentados anteriormente podemos concluir que el mercado chileno cuenta con una oferta amplia en cuanto a soluciones para el control de asistencia, tanto en dispositivos biométricos, como en plataformas en la nube mediante software. Sin embargo, cada uno de los elementos presentados en este estudio del arte, presentan limitaciones en cuanto a su estructura.

Por un lado tenemos los relojes de control biométricos ofrecidos por GeoVictoria o ZKTeco, que tienen soluciones bastante adaptadas a las necesidades del usuario pero con un enfoque en empresas más grandes, ligados a software propietario de una conectividad constante con los servidores del fabricante, esto reduciendo la flexibilidad, elevando costos para los servicios que los ERP finalmente ofrecieran a las medias y pequeñas empresas.

Por otra parte las soluciones basadas en software o servicios SaaS, tales como Talana, Buk, Defontana y Softland se centran en la gestion de la asistencia de manera remota y mediante plataformas web, limitandose en su ambiente operacional dedicado solo para trabajar sobre los mismos servicios de la misma empresa, además de haber ausencia de hardware, lo que limita la aplicabilidad en entornos con condiciones poco ideales para el funcionamiento con internet constante.

En consecuencia de lo comentado anteriormente, se evidencia una falta de soluciones híbridas que puedan unir elementos como la autenticación biométrica, con la integración configurable hacia sistemas ERP, sin depender totalmente de una infraestructura externa.

2.4 Metodologías

En este apartado se presentan las metodologías de desarrollo y evaluación que serán utilizadas para este proyecto.

2.4.1 Metodología de desarrollo

Considerando que el proyecto está relacionado con el desarrollo de un prototipo IoT que integra hardware, firmware y software, y pensando en su futura implementación en un entorno real, se identificaron y evaluaron distintos modelos del ciclo de vida de

desarrollo. Para el contexto específico de este trabajo, se contrastó el enfoque tradicional del Modelo en Cascada (Lineal Secuencial) frente al modelo de Prototipado Evolutivo Incremental.

El Modelo en Cascada exige definir la totalidad de los requerimientos y diseñar el sistema completo de forma estricta antes de comenzar cualquier implementación. En proyectos IoT, este enfoque presenta un alto riesgo: las restricciones técnicas del hardware (como las limitaciones de memoria del ESP32-CAM o la disponibilidad de pines GPIO para los sensores) a menudo se descubren durante la etapa de construcción. Esperar hasta las fases finales del proyecto para integrar y probar la comunicación entre los componentes físicos, el backend y la base de datos podría resultar en fallas críticas y la necesidad de rediseñar el sistema completo de forma tardía [27].

Por esta razón, se descartó el enfoque secuencial y se optó por emplear una metodología de prototipado evolutivo incremental, adaptándola a la dinámica de un proyecto en solitario. En este esquema, el profesor guía asume el rol de contraparte o cliente, entregando retroalimentación técnica al final de cada ciclo. Esta metodología es la más indicada para el proyecto, ya que permite mitigar riesgos técnicos construyendo, probando y validando módulos independientes de manera progresiva [25][26].

La metodología de prototipado evolutivo con un enfoque incremental propone un desarrollo basado en versiones sucesivas del prototipo, en la que cada iteración incorpora nuevas funcionalidades y mejoras sobre el anterior. De esta forma se puede probar, evaluar y ajustar continuamente el sistema a medida que va evolucionando. La estructura de la metodología, representada más adelante en la Figura 3.1 del capítulo 3, se compone de cuatro fases principales:

- **Diseño y planificación:** Se definen los objetivos específicos de cada iteración, se seleccionan componentes y se diseña la arquitectura.
- **Implementación y prueba:** Se desarrolla la funcionalidad planificada, se integra con los módulos que ya existen y se verifica su correcto funcionamiento
- **Evaluación y mejora:** Una vez desarrollado los procesos anteriores se analizan los resultados obtenidos identificación errores y planificando ajustes para la siguiente iteración.

[27]

Cada una de estas fases termina con un incremento funcional validado, lo que permite mantener un progreso controlado y documentado del prototipo.

2.4.2 Metodología de evaluación

Para la evaluación del proyecto se deben considerar 5 aspectos claves para determinar la viabilidad del proyecto, de los cuales nos permiten evaluar y verificar nuestros objetivos. Las pruebas evaluarán el prototipo, enfocándose en como funciona con todas las piezas y/o software en conjunto, saber que tan útil puede llegar a ser, validar la lógica del backend desde sus endpoints sin conocer la implementación interna, verificar cada módulo de forma aislada y automatizada, y finalmente al tener como objetivo su uso en un ERP, se necesita saber el nivel de satisfacción que se tiene por el prototipo final.

Pruebas de integración

Las pruebas de integración permiten evaluar diferentes componentes de un sistema ya sea hardware software o base de datos, asegurando que todos estos componentes funcionen correctamente en conjunto asegurando la comunicación e interoperabilidad entre ellos. Para este caso se realizarán pruebas de integración ascendentes de manera que se vayan realizando pruebas de los módulos más pequeños hasta llegar al módulo más grande [28]. La selección de esta prueba va relacionada con la metodología de desarrollo que consiste en el prototipado evolutivo, el cual se integran progresivamente los módulos validos en etapas previas.

Pruebas de usabilidad

Las pruebas de usabilidad tienen como objetivo evaluar la facilidad de uso, comprensión y satisfacción del usuario con el sistema, conforme a los estándares de calidad de software [29] [30]. Para llevar a cabo las pruebas de usabilidad se utilizará el System Usability Scale (SUS) [31]. El SUS es un método reconocido por su simplicidad de aplicación y por permitir una evaluación rápida y fiable de la usabilidad de un sistema [32]. La metodología del SUS consiste en obtener una puntuación global de usabilidad mediante la aplicación de un cuestionario basado en una escala de Likert de 5 puntos, compuesto por 10 preguntas que abordan diferentes aspectos de la experiencia de usuario, como la facilidad de uso, la claridad de la interfaz y la eficacia del sistema [31].

La razón del uso de esta prueba se debe a que el prototipo propuesto "prototipo de Control de asistencia Biometrico", debe ser fácil de usar para los trabajadores puedan realizar marcaciones sin complicaciones, como así para quienes integran este sistema a su ERP.

Pruebas de caja negra

Las pruebas de caja negra son una técnica de validación de software en la cual se evalúa el comportamiento del sistema exclusivamente a partir de sus entradas y salidas, sin

conocer ni inspeccionar su implementación interna [?]. Este enfoque se seleccionó para el presente proyecto debido a que el backend implementa una API REST con múltiples endpoints que deben responder correctamente ante combinaciones válidas e inválidas de parámetros, encabezados de autenticación y distintos roles de usuario.

Dado que el sistema incorpora lógica de negocio compleja (modelo multi-tenant por empresa, control de acceso basado en roles jerárquicos, enrolamiento biométrico mediante PIN de un solo uso y sincronización diferida de datos offline), las pruebas de caja negra permiten verificar que cada endpoint cumpla su contrato funcional sin necesidad de examinar el código fuente, cubriendo así la mayor parte de la aplicación desde la perspectiva del consumidor de la API. Esta técnica se complementa con las pruebas unitarias automatizadas para lograr una cobertura integral del backend.

Pruebas unitarias automatizadas

Las pruebas unitarias automatizadas permiten verificar de forma aislada y repetible el comportamiento de funciones, métodos y módulos individuales del sistema. Para este proyecto se implementó una suite de pruebas utilizando el framework `pytest` sobre Python 3.12, complementada con herramientas de mocking (`unittest.mock`, `pytest-mock`) para simular dependencias externas críticas como los modelos de reconocimiento facial DeepFace, las operaciones de visión por computadora con OpenCV, la comunicación MQTT con dispositivos ESP32-CAM y el envío de correos electrónicos vía SMTP [?].

Esta estrategia de simulación permite ejecutar las pruebas de forma determinista en cualquier entorno (incluyendo pipelines de integración continua) sin depender de hardware físico ni de servicios externos reales. Las pruebas unitarias se enfocan específicamente en validar la corrección de la lógica de negocio, la integridad del cifrado de datos biométricos, la robustez ante condiciones de error como fallos de conexión a la base de datos o webhooks ERP inaccesibles, y la precisión de los mecanismos de control de acceso por roles.

Pruebas de confiabilidad y disponibilidad

Las pruebas de confiabilidad y disponibilidad son cruciales en sistemas que operan en entornos con conectividad intermitente o hardware limitado, como los dispositivos IoT[33]. La finalidad de la prueba de confiabilidad es medir la capacidad del sistema para funcionar correctamente bajo condiciones normales durante un periodo prolongado sin errores ni pérdidas de datos, mientras que las pruebas de disponibilidad hace referencia a la capacidad del sistema para mantenerse operativo incluso cuando se presentan condiciones extremas, tales como pérdidas conexión, desconexión temporal de energía[34], etc. P Según Perry & Petry, estas pruebas son esenciales para garantizar que

el sistema mantenga la integridad de los datos y su disponibilidad en todo momento[35].

2.5 Resumen del capítulo

El capítulo 2 presenta el marco teórico que sustenta el proyecto, describiendo los conceptos esenciales relacionados con control de asistencia, IoT, ERP, y los requisitos normativos definidos por la Resolución Exenta N.º38. Luego se detallan las tecnologías utilizadas, incluyendo el microcontrolador ESP32-CAM, el lector de huellas AS608, los protocolos HTTP y MQTT, el uso de contenedores Docker y la base de datos PostgreSQL.

También se revisa el estado del arte, analizando las limitaciones de dispositivos biométricos comerciales y plataformas SaaS como Talana, Buk, Defontana y Softland, evidenciando la falta de soluciones híbridas, abiertas y con funcionamiento offline. Finalmente, el capítulo expone las metodologías de desarrollo y evaluación seleccionadas, donde se justifica el uso del prototipado evolutivo incremental y se describen las pruebas de integración, usabilidad, caja negra, unitarias automatizadas, y confiabilidad y disponibilidad que se aplicarán al sistema.

3. Metodologías

En el presente capítulo esta destinado a describir la manera en la que se aplicarán las metodologías propuestas en el capítulo anterior, además de presentar los resultados esperados.

3.1 Metodologia de desarrollo

Como se menciona el capítulo anterior en la sección 2.4.1 el proyecto propuesto se lleva a cabo mediante una metodología de tipo prototipado evolutivo con enfoque incremental. Esta metodología permite crear versiones funcionales del prototipo de manera progresiva, mejorando en cada iteración en base a pruebas funcionales del prototipo.

A continuación se procederá a explicar las distintas fases del proceso y como se aplicará la metodología.

3.1.1 Protipado Evolutivo Incremental

La metodología de prototipado evolutivo con un enfoque incremental propone un desarrollo basado en versiones sucesivas del prototipo, en la que cada iteración incorpora nuevas funcionalidades y mejoras sobre el anterior. De esta forma se puede probar, evaluar y ajustar continuamente el sistema a medida que va evolucionando. Como se ilustra en la Figura 3.1, la estructura de la metodología se compone de cuatro fases principales:

- **Requerimientos Iniciales:** Se identifica la problemática a resolver, se definen los objetivos específicos del prototipo para la iteración actual y se establecen las restricciones operativas y el alcance.
- **Diseño del Prototipo:** Se define la arquitectura preliminar y la funcionalidad mínima esperada, seleccionando las tecnologías y componentes de hardware o software necesarios.

- **Implementación Incremental:** Se construyen las versiones funcionales, programando e integrando progresivamente los componentes hasta obtener un prototipo ejecutable.
- **Pruebas y Validación:** El prototipo ejecutable se somete a pruebas funcionales. Se realizan los ajustes y correcciones pertinentes, y la retroalimentación obtenida retroalimenta directamente a la etapa de diseño para iniciar la siguiente iteración.

Cada uno de estos ciclos termina con un incremento funcional validado, lo que permite mantener un progreso controlado y documentado del prototipo a lo largo del tiempo. A continuación, se presenta el diagrama que representa este flujo de trabajo.

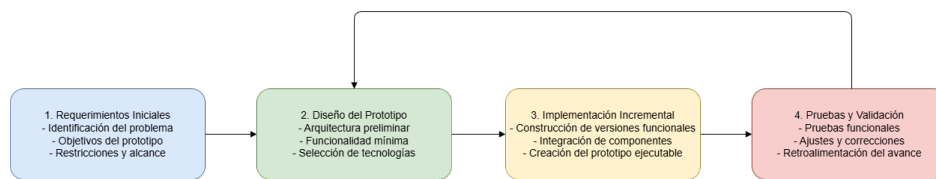


Figura 3.1: Metodología prototipado evolutivo con enfoque incremental

1. Requerimientos Iniciales

En esta fase de arranque se definen los objetivos específicos de la iteración correspondiente. Para cada ciclo se identifican las necesidades puntuales, como las limitaciones de memoria del ESP32-CAM o la necesidad de aislar datos por empresa. Esta etapa garantiza que cada iteración tenga un alcance claro y responda a una problemática técnica o funcional bien delimitada, estableciendo las bases para el trabajo posterior.

2. Diseño del Prototipo

A partir de los requerimientos, se definen los módulos que se incorporarán al prototipo. Se desarrollan diagramas preliminares, modelos de datos, mockups de interfaz y definiciones de comunicación entre módulos (HTTP o MQTT). Asimismo, se determinan las dependencias entre componentes, lo que permite asegurar que el incremento planificado se pueda integrar de manera fluida y coherente con la arquitectura de las iteraciones anteriores.

3. Implementación Incremental

En esta fase se procede a construir la funcionalidad definida en el diseño. La implementación incluye tareas prácticas como el desarrollo del firmware del ESP32-CAM, la programación del lector de huellas AS608, la gestión del almacenamiento en LittleFS, el desarrollo del backend en Flask y la integración de modelos biométricos. El objetivo es materializar el diseño en un bloque de código y hardware real, logrando un componente ejecutable.

4. Pruebas y Validación

Finalizada la implementación, el incremento es sometido a pruebas integradas para verificar su correcto funcionamiento. Se evalúa tanto el comportamiento individual del módulo como su interacción con los componentes ya existentes. Si se detectan errores, se documentan y se corrigen antes de cerrar la iteración. Esta fase es fundamental para asegurar la calidad del prototipo y validar que cumple con los requerimientos definidos al inicio del ciclo antes de pasar al siguiente incremento.

3.2 Plan de trabajo

La planificación del proyecto se estructura mediante la metodología de prototipado evolutivo con enfoque incremental descrita en la sección 3.1. Cada iteración incorpora nuevas funcionalidades sobre el prototipo anterior, permitiendo evaluar, corregir y fortalecer el sistema progresivamente. A continuación se presenta la carta Gantt con las 8 iteraciones planificadas, sus objetivos generales, duración estimada y avance acumulado.

Iter.	Objetivo	Duración	Avance
1	Integración HW + servidor embebido	3 semanas	12,5%
2	Almacenamiento local + modo offline	3 semanas	25%
3	Backend + BD + comunicación HTTP/MQTT	3 semanas	37,5%
4	Reconocimiento facial + anti-spoofing + cifrado	3 semanas	50%
5	Autenticación JWT + multi-tenant + enrolamiento	3 semanas	62,5%
6	Módulo antifraude (PIR + flash + firma de movimiento)	3 semanas	75%
7	Panel web gestión dispositivo + integración ERP	3 semanas	87,5%
8	Sincronización diferida + logs + cierre	3 semanas	100%

Cuadro 3.1: Plan de trabajo del proyecto: 8 iteraciones de 3 semanas cada una (24 semanas total)

El avance de cada iteración se evalúa mediante la superación de las pruebas funcionales definidas en la fase de implementación de cada ciclo. La metodología de evaluación detallada (pruebas de integración, usabilidad SUS, confiabilidad y disponibilidad) se describe en la sección 3.3. El desarrollo detallado de cada iteración, incluyendo análisis, diseño, implementación y resultados de pruebas, se presenta en el capítulo 4.

3.3 Metodología de evaluación

En esta sección se explica de qué manera se aplicaron, dentro del desarrollo del prototipo, las metodologías de evaluación descritas en el apartado 2.4.2 del marco teórico. Cada técnica de evaluación fue seleccionada en función de los objetivos del proyecto y del enfoque metodológico utilizado prototipado evolutivo con incrementos funcionales, lo que permitió validar el funcionamiento conjunto del hardware, software, backend y mecanismos de autenticación biométrica en entornos reales o simulados. Las evaluaciones se aplicaron de manera iterativa al finalizar cada incremento del prototipo, garantizando que las funcionalidades incorporadas funcionaran correctamente antes de avanzar al siguiente nivel de complejidad.

3.3.1 Pruebas de integración

Las pruebas de integración, como por su nombre dice son pruebas que permiten integrar componentes de un sistema verificando que su interacción sea correcta cuando se implementa un nuevo módulo o dispositivo como es en el contexto de este proyecto. Dado que el proyecto está compuesto por múltiples módulos independientes, lector de huellas, cámara del ESP32-CAM, el almacenamiento local, backend, reconocimiento facial, sistema antifraude (PIR + anti-spoofing), panel web y módulo ERP, es necesario verificar su interoperabilidad a medida que se ensamblan.

Con el objetivo de asegurar que todos los componentes mencionados anteriormente interactúen correctamente entre si se realizaron pruebas de integración de manera ascendente. El procedimiento que irá realizando el siguiente procedimiento.

1. Integración del hardware: Se evaluara la comunicación entre el ESP32-CAM y el modulo de huella digital. Además de comprobar la lectura y envío de datos de sensores IR.
2. Integración Almacenamiento: Se realizaron pruebas entorno al almacenamiento de los identificadores de huella, imágenes capturadas además de los registro de asistencia, donde se verificara la consistencia del almacenamiento.
3. Integración con el backend: Se enviaron distintos tipos de datos al backend, imágenes en base64, registros de logs, entre otros para verificar la comunicación entre el ESP32 y el backend.
4. Integración Backend con la base de datos: Se realizaron inserciones y búsqueda de elementos para verificar la conectividad constante con la base de datos.
5. Integración backend con interfaz web: Se comprobara si se rescatan los registros enviados por el prototipo y actualización en tiempo real con los datos.

Estas pruebas permiten evaluar el sistema propuesto de manera sistemática, de manera que cada elemento más pequeño se va integrando a medida que se avanza en el proyecto.

3.3.2 Pruebas de usabilidad

Las pruebas de usabilidad, como se mencionó en el punto 2.4.2, permiten evaluar la facilidad de uso, la eficacia del sistema y la claridad de la interfaz. Dado que los usuarios esperados del prototipo incluyen tanto trabajadores (usuarios no técnicos) como desarrolladores e integradores ERP (usuarios técnicos), la evaluación debe abarcar ambos perfiles.

Para llevar a cabo esta evaluación se utilizará el cuestionario **System Usability Scale (SUS)** desarrollado por Brooke [31], un instrumento estandarizado y ampliamente validado compuesto por exactamente 10 preguntas que se responden mediante una escala Likert de 5 puntos, donde 1 equivale a “Totalmente en desacuerdo” y 5 a “Totalmente de acuerdo”. Las preguntas impares son afirmaciones positivas, mientras que las pares son afirmaciones negativas (invertidas). La misma encuesta se aplica a ambos grupos de usuarios, garantizando que la puntuación obtenida sea comparable y metodológicamente válida.

Cuestionario SUS estándar

A continuación se presentan las 10 preguntas del instrumento SUS en su traducción al español, utilizadas para ambos perfiles de usuario:

N.º	Pregunta	Tipo
1	Creo que me gustaría usar este sistema con frecuencia.	Positiva
2	Encontré el sistema innecesariamente complejo.	Invertida
3	Pensé que el sistema era fácil de usar.	Positiva
4	Creo que necesitaría apoyo de una persona técnica para poder usar este sistema.	Invertida
5	Consideré que las funciones del sistema estaban bien integradas.	Positiva
6	Pensé que había demasiada inconsistencia en el sistema.	Invertida
7	Imagino que la mayoría de las personas aprenderían a usar este sistema rápidamente.	Positiva
8	Encontré el sistema muy engorroso de usar.	Invertida
9	Me sentí seguro al usar el sistema.	Positiva
10	Necesité aprender muchas cosas antes de poder empezar a usar este sistema.	Invertida

Cuadro 3.2: Cuestionario SUS estándar (Brooke, 1996) - Traducción al español

La puntuación SUS se calcula según el método descrito por Brooke [31]: para cada pregunta impar se resta 1 a la respuesta, y para cada pregunta par se resta la respuesta de 5. La suma de los 10 valores se multiplica por 2.5, obteniendo una puntuación entre 0 y 100. Según Bangor et al. [32], una puntuación ≥ 70 se considera “aceptable”, entre 50 y 70 “marginal” y < 50 “no aceptable”.

Adicionalmente, se aplicarán preguntas complementarias de contexto específicas para cada grupo (ver Anexo A), las cuales no forman parte del cuestionario SUS estándar y se analizan de forma descriptiva separada.

3.3.3 Pruebas de confiabilidad y disponibilidad

Para un prototipo IoT con funcionamiento offline, la confiabilidad del almacenamiento local y la disponibilidad del sistema durante fallas de red son esenciales. Estas pruebas evaluarán que tan robusto puede llegar a ser el prototipo frente a escenarios reales de operaciones y fallos. Para el procedimiento se realizara lo siguiente.

- Escenario de conectividad: Para realizar esta prueba se consideran distintos escenarios que permita evaluar el funcionamiento del prototipo en estos escenarios de los cuales consideramos escenarios de desconexión total, conectividad intermitente, reconexión luego de una desconexión prolongada.
- Métricas a evaluar: Dentro de las métricas que se evaluarán corresponden a los porcentajes de registros almacenados sin errores, cantidad de re intentos

automáticos, consistencia del log de eventos, tiempo de recuperación tras re conectar (sincronizar datos).

- Verificación con base de datos: Con la finalidad de comprobar la confiabilidad del sistema se debe compara el numero de registros capturados offline en comparación con los almacenados en la base de datos o que fueron sincronizados anteriormente.

3.3.4 Pruebas de caja negra

Las pruebas de caja negra evalúan el comportamiento del sistema a partir de sus entradas y salidas, sin examinar su estructura interna [?]. Se diseñan casos de prueba basados en la especificación de requisitos, explorando valores válidos, condiciones de borde y entradas erróneas [?]. Como no requieren conocer el código fuente, son útiles para validar que el comportamiento externo del sistema coincida con lo documentado.

En este proyecto, las pruebas de caja negra verifican el **contrato funcional de la API REST** desde la perspectiva de sus consumidores (dispositivos ESP32-CAM, panel web y sistemas ERP). Para cada endpoint se revisa que el código de estado HTTP sea el correcto, que la estructura de los cuerpos JSON coincida con lo esperado, que los mensajes de error sean claros sin exponer detalles internos, y que los mecanismos de autenticación y autorización (JWT con roles jerárquicos) restrinjan el acceso según el perfil del usuario.

La pertinencia de esta técnica para el proyecto se justifica por tres razones. Primero, la API REST es la interfaz externa única del backend, por lo que cualquier defecto en un endpoint afecta a todos los consumidores del sistema. Segundo, la lógica de negocio (modelo multi-tenant, roles jerárquicos, enrolamiento biométrico con PIN, sincronización offline) genera una cantidad de escenarios funcionales que no puede verificarse solo inspeccionando el código. Tercero, en un proceso de prototipado evolutivo incremental, estas pruebas permiten revalidar el comportamiento de todos los endpoints al final de cada iteración, detectando regresiones sin modificar los casos de prueba existentes.

La cobertura abarcó todos los endpoints del backend, agrupados por área funcional: autenticación y control de acceso, registro de usuarios y empresas, operaciones CRUD sobre entidades del modelo de datos, marcaciones y sincronización offline, y reconocimiento facial. Para cada uno se diseñaron casos de flujo normal y condiciones de error (parámetros faltantes, valores fuera de rango, autenticación insuficiente).

3.3.5 Pruebas unitarias automatizadas

Las pruebas unitarias verifican funciones o modulos de forma aislada, reemplazando dependencias externas con mocks para que el resultado dependa solo de la logica bajo

prueba [?]. A diferencia de las pruebas de integracion, se enfocan en la correccion algoritmica de cada pieza de codigo, permitiendo localizar defectos con precision y ejecutar la suite rapidamente.

En este proyecto, las pruebas unitarias cubren cuatro aspectos del backend:

- **Logica de negocio:** validacion de RUT, herencia de roles jerarquicos, asignacion de turnos, aislamiento multi-tenant, deteccion de marcaciones duplicadas y generacion de PIN de enrolamiento.
- **Datos sensibles:** cifrado y descifrado de embeddings faciales con Fernet, persistencia de huellas dactilares y gestion de consentimiento biometrico.
- **Robustez ante errores:** manejo de fallos de conexion a la base de datos, timeouts en webhooks ERP, interrupcion del servidor SMTP y mensajes MQTT malformados.
- **Control de acceso:** verificacion de que los roles jerarquicos (administrador, empleador, trabajador) denieguen el acceso a recursos no autorizados en cada endpoint.

El mocking es necesario porque el sistema depende de componentes externos (DeepFace, OpenCV, broker MQTT, servidor SMTP, webhooks ERP) que requerian hardware fisico o servicios activos para ejecutar las pruebas. Al simular estas dependencias, la suite se ejecuta de forma determinista en cualquier entorno (incluyendo CI/CD) sin necesidad del hardware real, lo que permite revalidar todo el codigo en cada iteracion del prototipado evolutivo.

La suite se organizo por modulos del backend: base de datos, cifrado, correo electronico, MQTT, autenticacion, reconocimiento facial, CRUD de entidades, sincronizacion offline, integraciones ERP y un emulador del ESP32-CAM. Esta organizacion permite ejecutar solo las pruebas del modulo modificado, acelerando la retroalimentacion durante el desarrollo.

3.3.6 Resultados esperados de las pruebas

Este apartado define los valores objetivos y criterios de aceptación que se esperan alcanzar al ejecutar el conjunto de pruebas descritas en la sección 3.3 (Metodología de evaluación). Estos valores sirven como referencia para comparar los resultados reales que se presentarán en el capítulo 5.

Pruebas de integración

Las pruebas de integración ascendente definidas en la sección 3.3.1 apuntan a verificar la correcta comunicación entre los módulos del sistema. Según Pressman [27] e IBM [28], una prueba de integración ascendente debe garantizar el 100% de interoperabilidad entre los módulos verificados; un valor inferior indicaría una falla crítica de acoplamiento que invalidaría el incremento. Se espera alcanzar los siguientes resultados:

Criterio	Valor esperado
Comunicación ESP32-CAM ↔ AS608 (UART, 57600 baud)	100%
Captura de imagen facial (OV2640) → envío al backend	100%
Persistencia de marcaciones en LittleFS tras 50 marcaciones	100%
Sincronización diferida con backend (reintento tras reconexión)	100%
Inserción/consulta en PostgreSQL sin errores	100%
Renderizado de vistas web con datos actualizados	100%

Cuadro 3.3: Resultados esperados de las pruebas de integración

Pruebas de usabilidad (SUS)

La encuesta System Usability Scale (SUS) descrita en la sección 3.3.2 se aplicará a dos grupos de usuarios. Según Bangor et al. [32], un puntaje $SUS \geq 70$ se considera “aceptable”, entre 50 y 70 “marginal” y < 50 “no aceptable”. Sistemas con flujos de embedding facial y configuración de red suelen situarse en el rango 65 a 75, por lo que se fija como objetivo mínimo un 70 global. Se esperan los siguientes puntajes:

Métrica	Valor esperado
Puntuación SUS - usuarios no técnicos	≥ 68
Puntuación SUS - usuarios técnicos	≥ 70
Puntuación SUS global del sistema	≥ 70
Tasa de completitud de tareas (no técnicos)	$\geq 85\%$
Tasa de completitud de tareas (técnicos)	$\geq 90\%$
Tiempo medio de configuración inicial (no técnicos)	≤ 10 min
Tiempo medio de configuración inicial (técnicos)	≤ 5 min

Cuadro 3.4: Resultados esperados de las pruebas de usabilidad (SUS)

Pruebas de confiabilidad y disponibilidad

Las pruebas de confiabilidad y disponibilidad descritas en la sección 3.3.3 someten al sistema a tres escenarios de conectividad. Según Perry [35] y Atlassian [34], un sistema IoT con operación offline debe garantizar cero pérdida de datos y una disponibilidad acumulada $\geq 90\%$ en escenarios reales de uso para ser considerado operacionalmente viable. Se esperan los siguientes resultados:

Escenario	Métrica	Valor esperado
Desconexión total	Registros almacenados sin error	100%
Desconexión total	Reintentos automáticos de reconexión Wi-Fi	5 antes de AP
Conectividad intermitente	Reintentos exitosos de sincronización	$\geq 95\%$
Conectividad intermitente	Consistencia del log de eventos	0 inconsistencias
Reconexión prolongada	Tiempo de sincronización de 50 marcaciones	< 60 s
Operación continua	Disponibilidad acumulada (uptime)	$\geq 90\%$
Persistencia	Datos conservados tras corte de energía	100%

Cuadro 3.5: Resultados esperados de las pruebas de confiabilidad y disponibilidad

Pruebas de caja negra

Las pruebas de caja negra descritas en la sección 3.3.4 evaluaron los 28 endpoints de la API REST del backend. Dado que estos endpoints son el punto de entrada principal al backend desde los dispositivos ESP32, el panel web y los sistemas ERP externos, deben verificar exhaustivamente tanto entradas válidas como inválidas. Un endpoint que no responda correctamente ante un error compromete la experiencia de usuario y la integridad de los datos. Se esperan los siguientes resultados:

Criterio	Valor esperado
Endpoints de autenticación (login, me, register, register-company)	100% funcionales
Endpoints de CRUD (personas, turnos, asignaciones, dispositivos, ERP)	100% funcionales
Endpoints de asistencias y sincronización	100% funcionales
Endpoints de reconocimiento facial (identificar, verificar, registrar)	100% funcionales
Endpoints de enrolamiento de dispositivos (PIN generar/enrolar)	100% funcionales
Códigos de error HTTP (400, 401, 403, 404, 409, 500)	100% correctos
Aislamiento multi-tenant por empresa	100%

Cuadro 3.6: Resultados esperados de las pruebas de caja negra

Pruebas unitarias automatizadas

Las pruebas unitarias automatizadas descritas en la sección 3.3.5 se implementaron con pytest sobre una base de datos PostgreSQL en Docker. En proyectos con componentes biométricos y comunicación IoT, una cobertura de código $\geq 90\%$ es considerada un estándar de calidad robusto para el backend, ya que garantiza que la lógica de negocio, la gestión de errores y los mecanismos de seguridad han sido verificados de forma exhaustiva [?]. Los módulos críticos de seguridad como el cifrado de datos biométricos deben mantener 100% de cobertura para asegurar que no existan caminos de código no probados. Se esperan los siguientes resultados:

Métrica	Valor esperado
Pruebas unitarias totales	334
Archivos de prueba	19
Pruebas ejecutadas exitosamente (0 fallos)	100%
Cobertura de código total del backend	$\geq 90\%$
Cobertura de <code>encryption.py</code>	100%
Cobertura de <code>email_service.py</code>	100%
Cobertura de <code>database.py</code>	$\geq 99\%$
Cobertura de <code>routes/turnos.py</code>	100%
Cobertura de <code>routes/asignaciones.py</code>	$\geq 96\%$
Cobertura de <code>routes/auth.py</code>	$\geq 84\%$
Tiempo total de ejecución de la suite	< 3 minutos

Cuadro 3.7: Resultados esperados de las pruebas unitarias automatizadas

3.4 Resultados Esperados de los Objetivos Específicos

A continuación en esta sección se presentarán los resultados esperados para los objetivos específicos.

- Objetivo específico 2: Implementar el módulo biométrico de registro de asistencia mediante reconocimiento facial y huella digital.
 - Resultado esperado: Para este objetivo se espera tener un prototipo que permita registrar asistencia de un trabajador, mediante reconocimiento facial o huella digital.
- Objetivo específico 3: Incorporar capacidad de funcionamiento offline con almacenamiento local y sincronización posterior.
 - Resultado esperado: Para este objetivo se espera que el prototipo pueda tener la capacidad de funcionamiento offline almacenando los registros de manera local para su posterior sincronización con el servidor.
- Objetivo específico 6: Integrar y validar el funcionamiento conjunto del hardware y software desarrollado.
 - Resultado esperado: Para este objetivo se espera que el hardware construido funcione correctamente en conjunto como así su operabilidad con el backend y software.
- Objetivo específico 7: Evaluar el desempeño, confiabilidad y usabilidad del sistema en escenarios de prueba reales.
 - Resultado esperado: Para este objetivo se espera obtener alta disponibilidad con al menos un 90% de su funcionamiento en los escenarios en donde el dispositivo se encuentre offline, o con conexión intermitente.

3.5 Resumen del capítulo

El capítulo 3 presenta la metodología de prototipado evolutivo incremental adoptada para el desarrollo del sistema, describiendo sus tres fases (diseño y planificación, implementación y prueba, evaluación y mejora) aplicadas al contexto del proyecto. Se incluye el plan de trabajo con la carta Gantt de las 8 iteraciones, detallando los objetivos, duración estimada y avance acumulado de cada una.

Asimismo, se explica la metodología de evaluación, que incluye pruebas de integración, usabilidad mediante el método SUS, y pruebas de confiabilidad y disponibilidad para

medir el desempeño del sistema en escenarios reales. El capítulo concluye con los resultados esperados, tanto de los objetivos específicos del proyecto como de las pruebas a realizar, los cuales serán contrastados con los resultados reales en el capítulo 5.

4. Desarrollo e implementación

En el siguiente capítulo se presentan en detalle las 8 iteraciones del proyecto, desarrolladas según la metodología de prototipado evolutivo incremental descrita en el capítulo 3. Para cada iteración se describen las fases de análisis, diseño, implementación y pruebas, reflejando la construcción progresiva del prototipo desde la integración inicial de hardware hasta la sincronización diferida y el cierre del proyecto.

4.1 Diseño de software

Esta sección está destinada a presentar las arquitecturas desarrolladas para este proyecto como así diagramas y mockups que permitan dar entender la lógica del proyecto como así futuras vistas.

4.1.1 Arquitectura física

La arquitectura física propuesta se basa en una arquitectura basada en microservicios. En la figura 4.1 se puede presenciar como sería la comunicación entre el dispositivo IoT que emite el evento para realizar el registro, para luego que el servidor con el broker MQTT reciba las imágenes y utilice el servicio de reconocimiento facial, para luego enviarlas al servidor backend en donde se distribuyen para el sistema ERP y para la registrarlo en la base de datos, además de disponer del panel web para la visualización de los datos de manera remota.

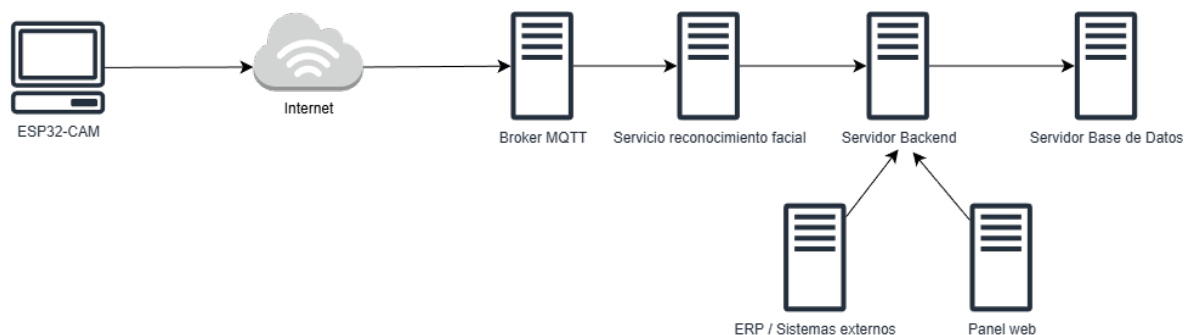


Figura 4.1: Arquitectura física

4.1.2 Arquitectura lógica

La arquitectura lógica del sistema organiza los módulos de software que permiten el funcionamiento del dispositivo IoT y su comunicación con el backend. Tal como se muestra en la Figura 4.2, el ESP32-CAM integra los componentes encargados de la captura de imágenes, la lectura de la huella digital, el almacenamiento local mediante SPIFFS y el servidor web embebido, los cuales interactúan entre sí para registrar usuarios, capturar datos biométricos y almacenar o enviar asistencias.

El backend recibe la información enviada por el dispositivo, procesa las imágenes, administra la base de datos y expone una API que permite la integración con sistemas externos como ERP. Finalmente, la arquitectura lógica contempla la comunicación entre cada módulo mediante protocolos HTTP o MQTT, lo que posibilita el funcionamiento offline, la sincronización posterior y la consulta remota de los datos desde el panel web.

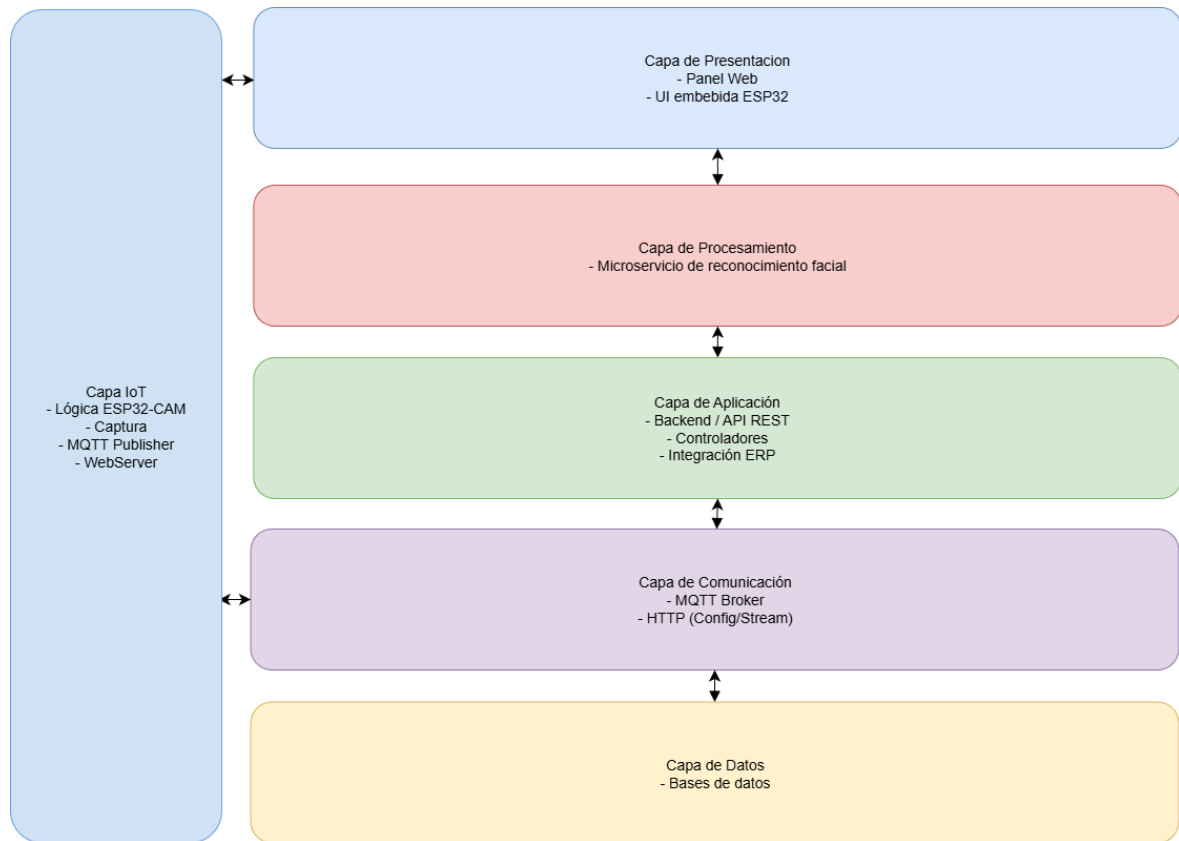


Figura 4.2: Arquitectura lógica

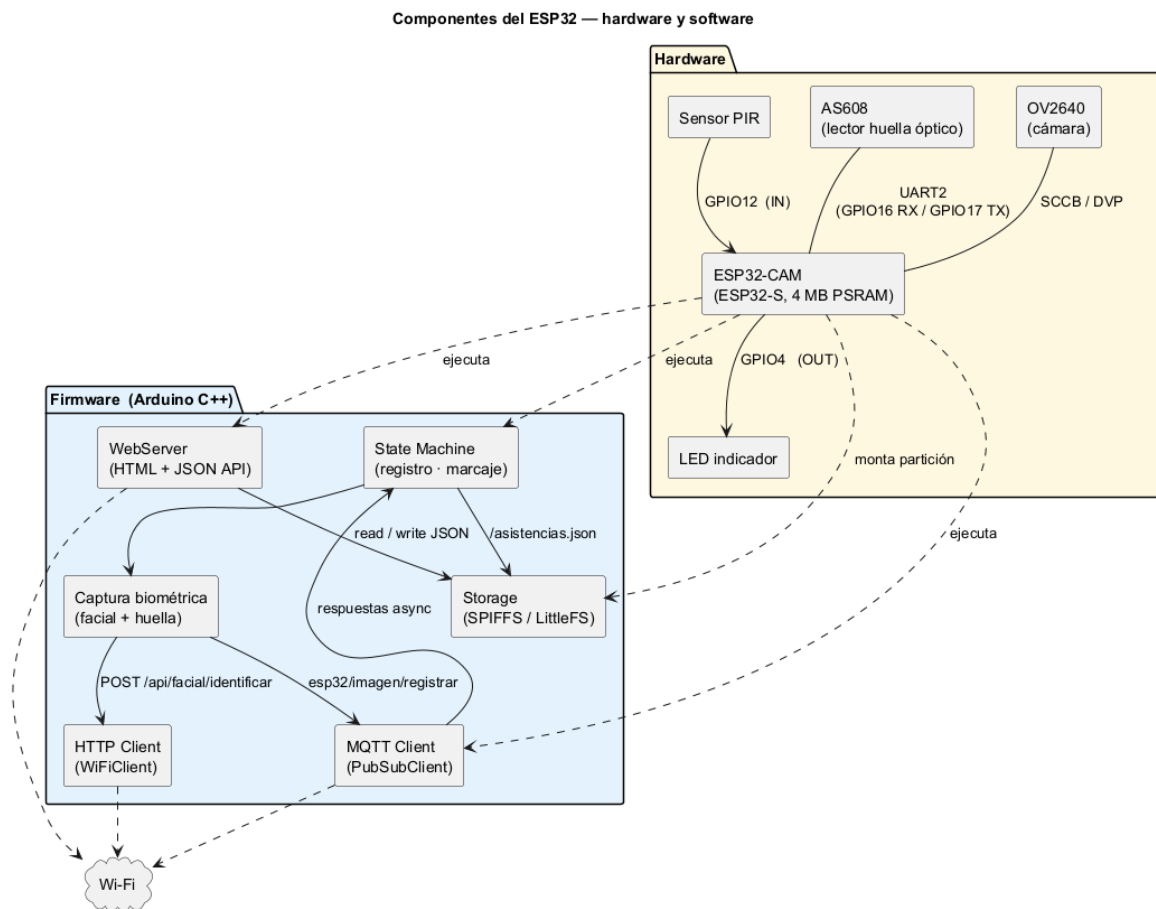


Figura 4.3: Componentes del ESP32-CAM — hardware y firmware.

4.2 Implementación

En este apartado se procederá a explicar como se llevaron a cabo las iteraciones planteadas en el capítulo 3.1.

4.2.1 Iteración 1: Integración de hardware y servidor embebido

La primera iteración corresponde al diseño conceptual del sistema y a la preparación del entorno de desarrollo, incluyendo la integración inicial del hardware biométrico seleccionado. Esta etapa establece la base funcional sobre la cual se construirán las iteraciones posteriores.

Análisis

En esta iteración se definieron los requisitos mínimos para validar la viabilidad técnica del sistema, analizando los componentes del hardware y las funcionalidades necesarias para las iteraciones siguientes.

El análisis se centró en tres ejes principales:

- **Definición de funcionalidades mínimas (MVP):** Se identificaron las capacidades básicas que el dispositivo debía cumplir desde el inicio: capturar imágenes desde la cámara integrada, leer huellas dactilares mediante el sensor AS608, montar un servidor HTTP local con páginas HTML embebidas y operar de forma autónoma mediante Wi-Fi. Estas funciones representan la base operativa sobre la cual se desarrollarán características más avanzadas como turnos, sincronización y gestión de usuarios.
- **Identificación de limitaciones técnicas:** Se revisaron las restricciones tecnológicas del ESP32-CAM, como la limitada memoria RAM disponible (520 KB), el uso compartido del bus de cámara con los pines GPIO, la ausencia de pines GPIO libres -lo que forzó el uso de los pines GPIO14 (RX) y GPIO15 (TX) para la comunicación UART con el lector de huellas- y la necesidad de un manejo eficiente de cadenas HTML grandes en memoria. También se identificó que el flash LED integrado consume corriente significativa (250 mA), requiriendo protección mediante PWM para evitar caídas de tensión durante la captura.
- **Estrategia de operación autónoma:** Dado que uno de los requisitos fundamentales del sistema es la autonomía, se analizó la necesidad de montar un servidor web local. En caso de no contar con Wi-Fi configurada, el dispositivo debe activar automáticamente un modo Access Point (AP) con SSID ESP32-ASISTENCIA, permitiendo al administrador conectarse directamente para configurar la red y registrar usuarios sin dependencia de internet. Esto reduce riesgos y asegura la continuidad operacional del dispositivo.

Como resultado del análisis, se establecieron los criterios de diseño que guiarían el desarrollo de esta iteración: simplicidad, bajo consumo de recursos, modularidad de componentes y autonomía operativa.

Diseño

En esta iteración se elaboraron los primeros elementos arquitectónicos del sistema:

- **Arquitectura física,** representando los componentes utilizados y su interconexión, la cual se puede revisar en la Figura 4.1 del 4.1.1 Arquitectura física.
- **Arquitectura lógica,** definiendo los módulos internos del firmware tales como captura de imágenes, lectura de huella, almacenamiento y conectividad, la cual se puede revisar en la Figura 4.2 del 4.1.2 Arquitectura lógica.

- **Mockups iniciales** de la interfaz embebida del ESP32-CAM, incluyendo:
 - Menú principal.

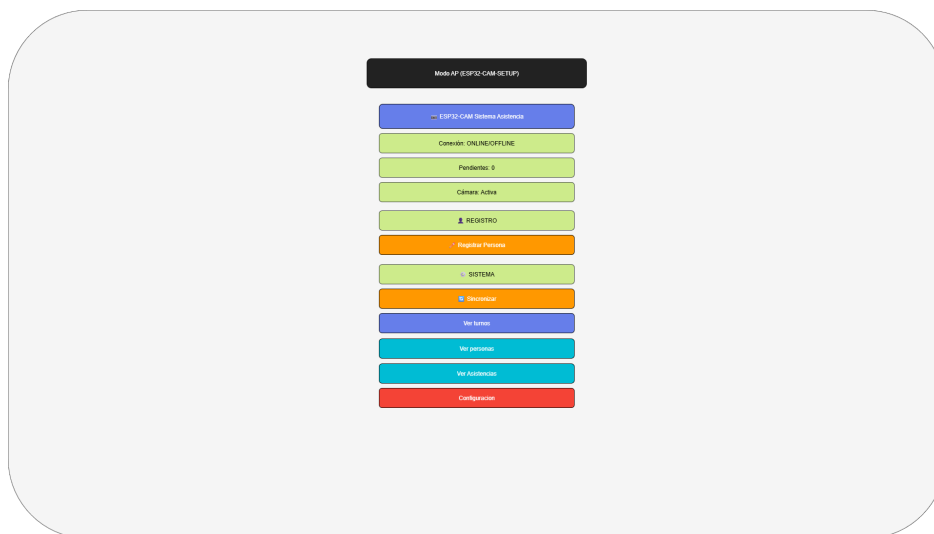


Figura 4.4: Mockup menú principal en el ESP32-CAM

- Panel de configuración de Wi-Fi.

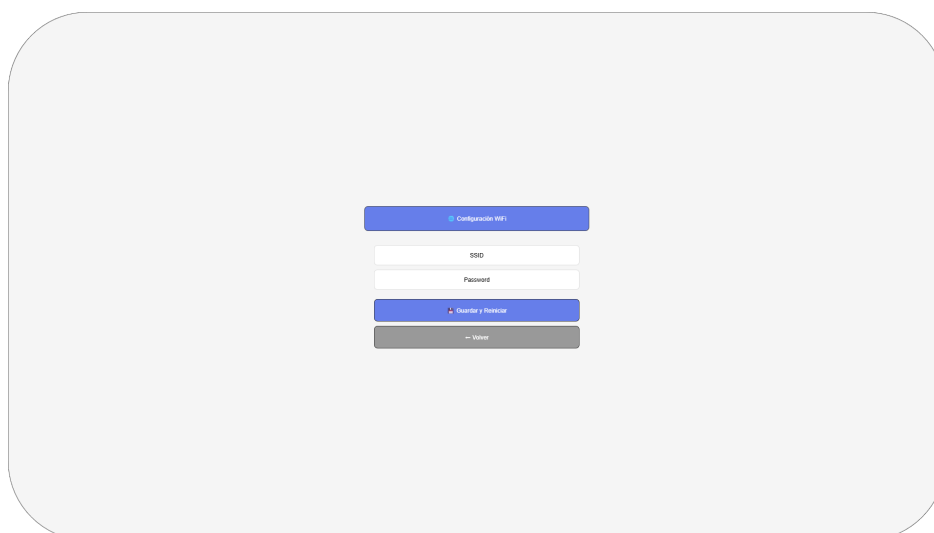


Figura 4.5: Mockup panel de configuración Wi-Fi

- Vista de registro de usuario.

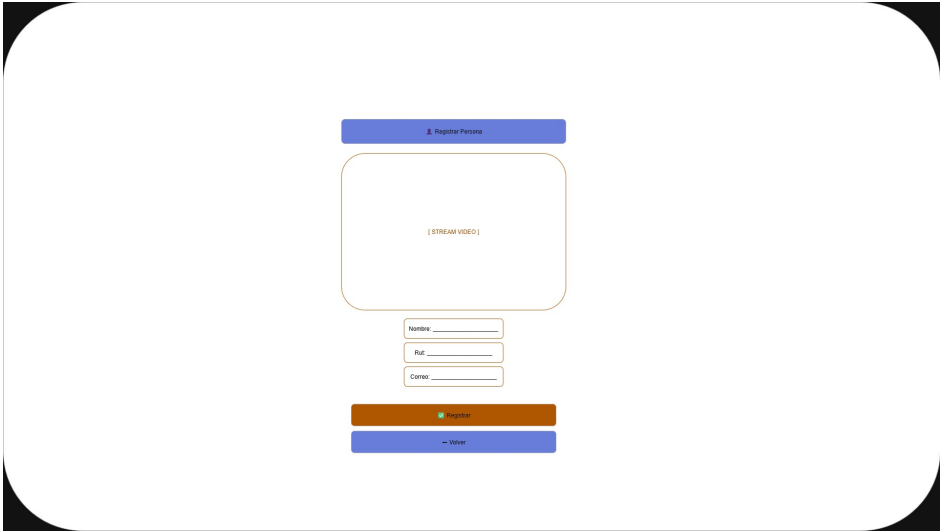


Figura 4.6: Mockup registro usuario en el ESP32-CAM

– Vista de usuarios almacenados localmente.

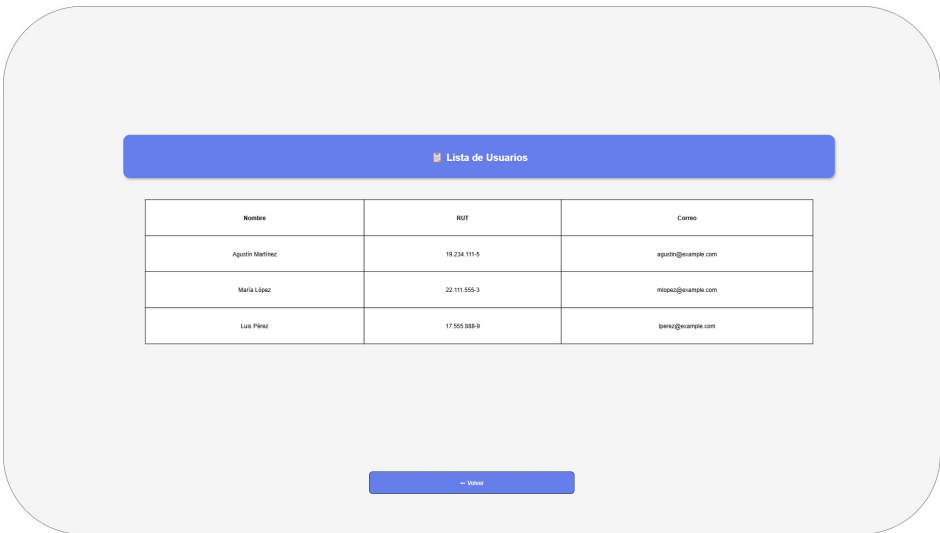


Figura 4.7: Mockup usuarios almacenados en el ESP32-CAM

– Vista de turnos almacenados localmente.

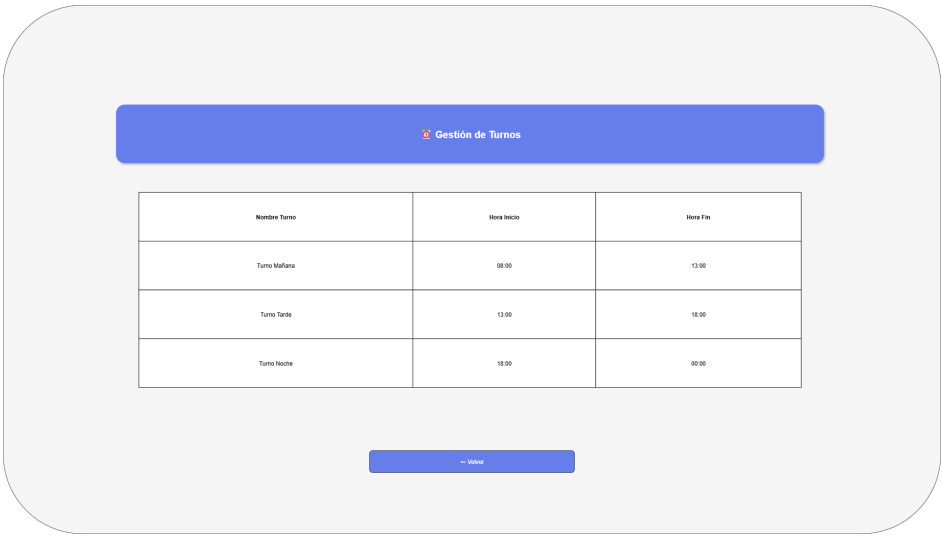


Figura 4.8: Mockup visualización de turnos en el ESP32-CAM

– Vista de crear turnos.

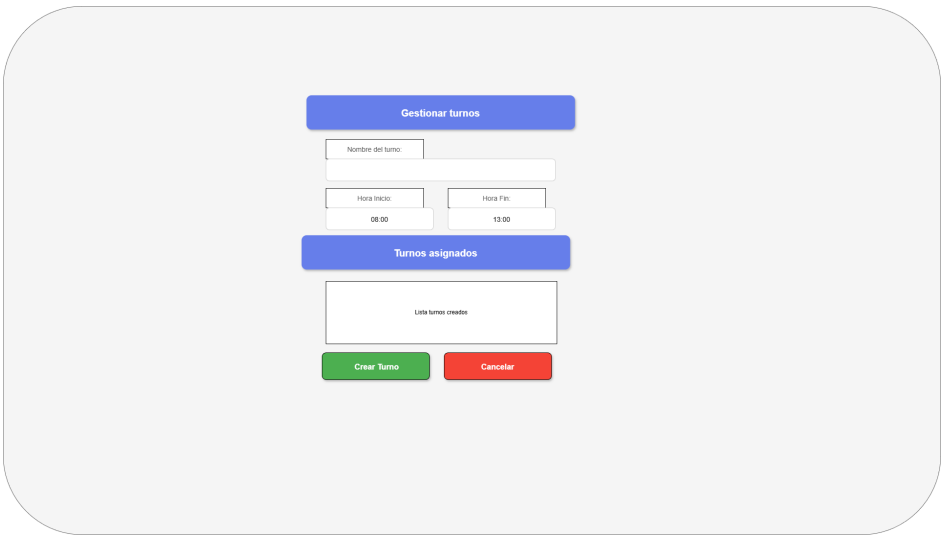


Figura 4.9: Mockup creación de turnos en el ESP32-CAM

– Vista de Asignación de turnos.

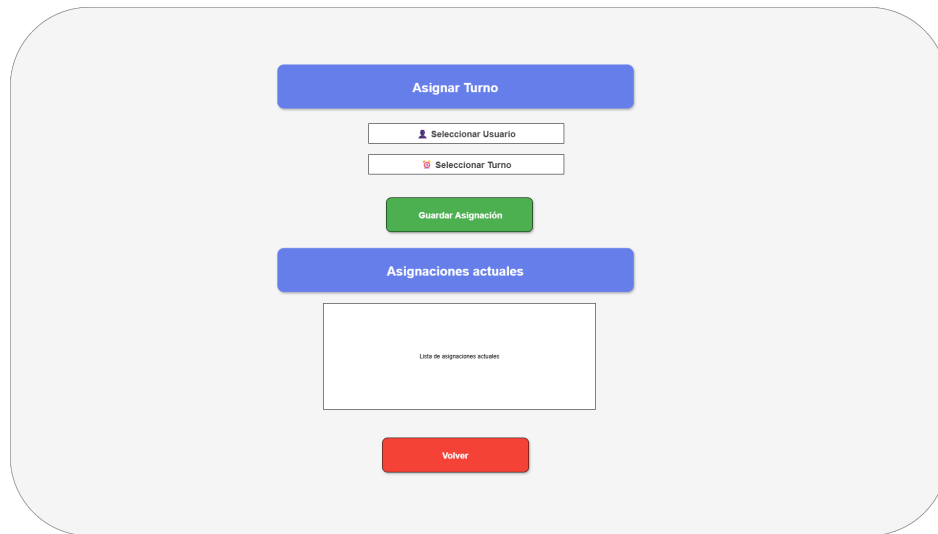


Figura 4.10: Mockup asignación de turnos en el ESP32-CAM

- **Diseño del flujo operacional:** Representa el flujo del proceso de marcación para los casos en se cuenta con conexión a internet y sin internet.

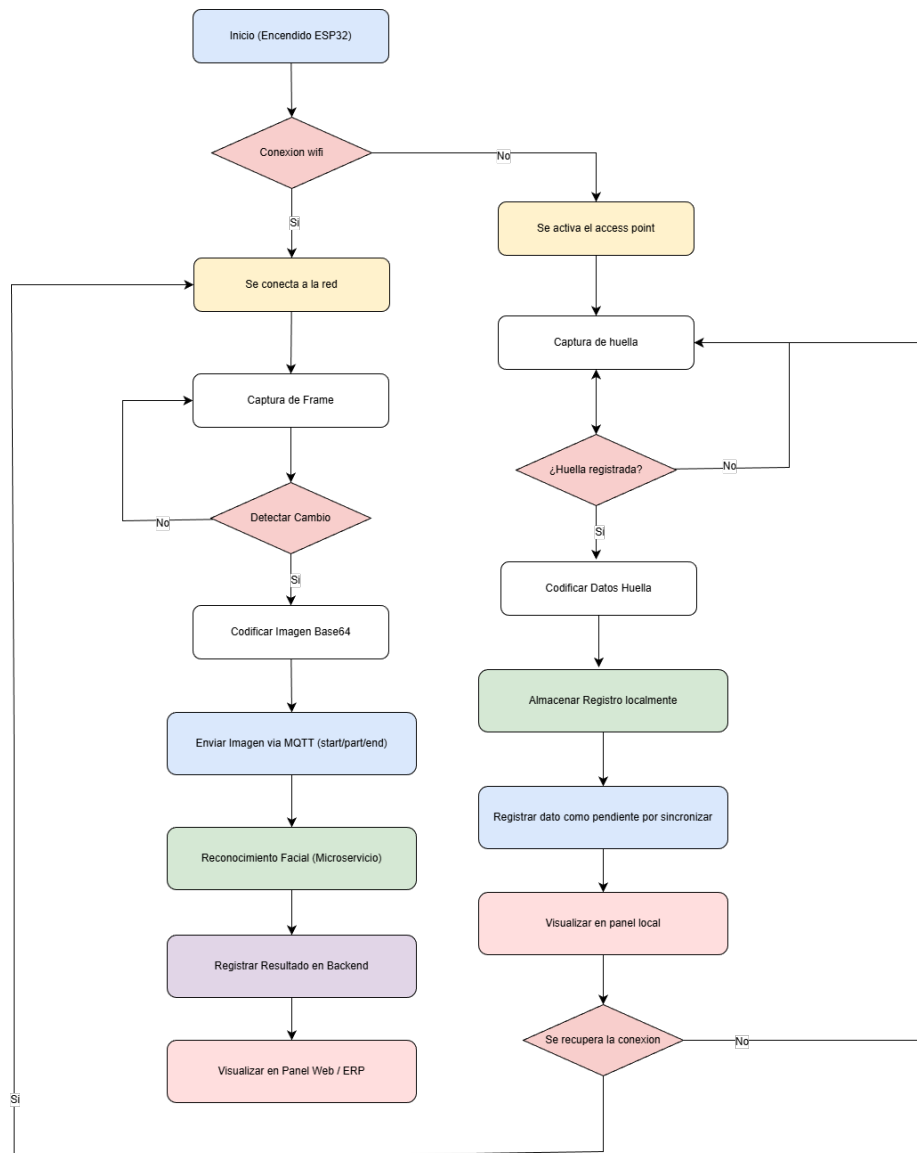


Figura 4.11: Flujo operacional para el registro de asistencia.

Estos elementos permiten visualizar el flujo básico del dispositivo antes de incorporar funcionalidades adicionales como turnos, sincronización o panel web backend, los cuales se desarrollarán en iteraciones posteriores.

Implementación

En esta etapa se integró tanto el hardware principal del prototipo como el desarrollo de las vistas web embebidas que permiten la interacción del usuario con el dispositivo. Para ello se utilizaron las siguientes librerías:

- `esp_camera`, utilizada para inicializar y configurar la cámara OV2640 integrada en el ESP32-CAM.

- `HardwareSerial` y `Adafruit_Fingerprint`, empleadas para la comunicación UART con el lector biométrico AS608.
- `WebServer`, utilizada para montar un servidor web HTTP en el puerto 80 y atender rutas asociadas a las vistas.
- `LittleFS` y `ArduinoJson`, empleadas para almacenamiento persistente en memoria flash y manipulación estructurada de datos JSON.

Integración del lector de huellas AS608 El lector biométrico AS608 fue conectado directamente al ESP32-CAM mediante la interfaz UART secundaria (`HardwareSerial 2`), utilizando los pines GPIO14 (RX) y GPIO15 (TX). Esta conexión, que emplea pines distintos a los documentados inicialmente, fue forzada por la escasez de GPIO libres tras la asignación del bus de cámara. La velocidad de comunicación se estableció en 57600 baudios, y la librería `Adafruit_Fingerprint` se vincula a dicho puerto serial. La Figura 4.12 presenta la conexión física entre ambos dispositivos.

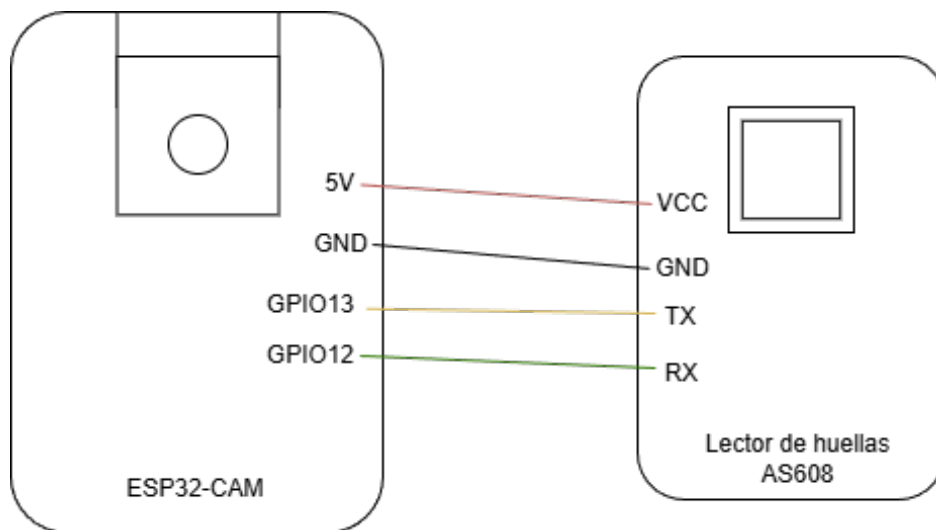


Figura 4.12: Conexiones entre el ESP32-CAM y el lector de huellas AS608.

Una vez establecida la comunicación, el firmware puede ejecutar operaciones como:

- Captura de imagen dactilar.
- Conversión de la imagen en una plantilla biométrica.
- Almacenamiento del template en un slot interno del sensor (1-127).
- Identificación mediante búsqueda de coincidencias en memoria.

El fragmento de código que se ve en la Figura 4.12 es un ejemplo que se construyó para ser utilizado en iteraciones posteriores para el uso del lector de huellas junto al ESP32-CAM.

```
HardwareSerial FingerSerial(2);
Adafruit_Fingerprint finger = Adafruit_Fingerprint(&FingerSerial);

void initFinger() {
    FingerSerial.begin(57600, SERIAL_8N1, 12, 13);
    finger.begin(57600);

    if (finger.verifyPassword()) {
        Serial.println("Sensor AS608 listo.");
    } else {
        Serial.println("Error: no se detecta el AS608");
    }
}
```

Figura 4.13: Fragmento de código para inicializar el lector AS608 utilizando UART.

Configuración de la cámara y el flash PWM La cámara OV2640 se configuró con resolución VGA (640×480) y compresión JPEG con calidad 8 (escala 0-63, donde valores bajos indican mayor calidad). La frecuencia del reloj externo (XCLK) se fijó en 20 MHz. Para evitar el parpadeo visible y las caídas de tensión provocadas por el flash LED durante la captura, se implementó un control PWM a 5 kHz con resolución de 8 bits, aplicando un ciclo de trabajo del 50% (duty 128 de 255) durante la iluminación. Esto permite encender el flash con brillo suficiente sin sobrecargar la fuente de alimentación del ESP32-CAM.

Calibración del sensor PIR Se integró un sensor PIR (Passive Infrared) conectado al GPIO12 configurado con resistencia de pull-down interna. El sensor PIR actúa como detector de presencia: cuando una persona se sitúa frente al dispositivo, el PIR detecta el cambio de radiación infrarroja y activa el modo de captura. El setup incluye una fase de calibración de 3 segundos para estabilizar la lectura del sensor antes de iniciar el bucle principal. La lógica de uso del PIR como control de activación se detalla en la iteración 6.

Montaje del servidor web embebido Para habilitar la interacción local con el dispositivo, se implementó un servidor web mediante la librería `WebServer` en el puerto 80. Este servidor permite entregar páginas HTML, datos JSON y recursos estáticos. Cuando no hay conectividad a una red externa, el ESP32-CAM activa automáticamente

un Access Point con SSID ESP32-ASISTENCIA y contraseña Asistencia2026, sirviendo las mismas páginas sin depender de internet.

Las rutas principales del sistema incluyen:

- `/`: Menú principal con indicadores de estado.
- `/wifi-setup`: Configuración de red Wi-Fi y parámetros del backend.
- `/register`: Registro de personas con captura biométrica.
- `/asistencias`: Lista de asistencias almacenadas localmente.
- `/gestion`: Gestión de turnos (creación y administración).
- `/personas`: Visualización de personas registradas.
- `/asignaciones`: Vista para realizar asignaciones de turnos.
- `/logs`: Visualización de logs del sistema.
- `/turnos`: Visualización de turnos creados.

Para la implementación del servidor HTTP embebido, se utilizó el módulo `WebServer` del ESP32. Cada operación del sistema (registro de usuarios, captura de huellas, creación de turnos, asignación de turnos, lectura de asistencias, configuración Wi-Fi, entre otras) fue mapeada a un endpoint específico mediante funciones manejadoras (*handler functions*). Este mecanismo permite encapsular la lógica de cada componente del sistema y mantener una estructura modular del firmware.

```
server.on("/", []() { server.send(200, "text/html", htmlPage); });
server.on("/register", []() { server.send(200, "text/html", registerPage); });
server.on("/gestion", []() { server.send(200, "text/html", gestionPage); });
server.on("/personas", []() { server.send(200, "text/html", personasPage); });
server.on("/asistencias", []() { server.send(200, "text/html", asistenciasPage); });
server.on("/turnos", []() { server.send(200, "text/html", turnosPage); });
server.on("/asignaciones-view", []() { server.send(200, "text/html", asignacionesPage); });
server.on("/wifi-setup", []() { server.send(200, "text/html", wifiConfigPage); });
server.on("/wifi-config", handleWiFiConfig);
server.on("/registrar", handleRegisterUser);
server.on("/crear_turno", handleCreateTurn);
server.on("/asignar", handleAssignTurn);
server.on("/marcar", handleMarcarAsistencia);
server.on("/api/personas", handleGetPersonas);
server.on("/api/turnos", handleGetTurnos);
server.on("/api/asignaciones", handleGetAsignaciones);
server.on("/api/asistencias", handleGetAsistencias);
server.on("/limpiar", handleLimpiarDatos);

server.begin();
```

Figura 4.14: Asignación de rutas para el servidor web embebido.

Creación de vistas HTML embebidas Las vistas fueron desarrolladas como archivos `.html` independientes almacenados en el sistema de archivos LittleFS del ESP32-CAM, conteniendo HTML, CSS y JavaScript autónomo, sin dependencias de CDN externas. El servidor web embebido las sirve mediante la función `servirArchivo()`, que lee el archivo solicitado desde LittleFS y lo envía al cliente. Esto garantiza que el ESP32-CAM sirva cada página incluso en modo Access Point sin conexión a internet.

Menú principal La Figura 4.15 presenta la vista principal accesible en la ruta `/`. Desde esta interfaz el usuario puede visualizar el estado del dispositivo (online/offline, sensor activo, enrolado), registrar personas, ver asistencias, gestionar turnos, ver asignaciones, acceder a la configuración de red, sincronizar con el backend y limpiar datos del dispositivo.

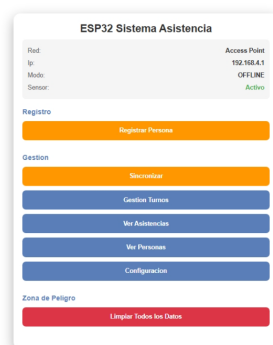


Figura 4.15: Menú principal del sistema web embebido.

Vista de configuración Wi-Fi Mediante la ruta `/wifi-setup`, el dispositivo ofrece un formulario para actualizar las credenciales de la red local (SSID, contraseña), la URL del backend, la dirección del broker MQTT y el PIN de enrolamiento. Estos valores se persisten en SPIFFS mediante el archivo `wifi.json`. La Figura 4.16 muestra esta interfaz.



Figura 4.16: Interfaz de configuración Wi-Fi.

Vista de registro de personas La ruta `/register` despliega una vista que combina un formulario para ingresar el nombre del usuario, RUT y correo electrónico, y muestra indicadores visuales del estado del proceso de enrolamiento (huella, rostro). La Figura 4.17 lo muestra.



Figura 4.17: Vista para el registro de personas en el dispositivo.

Vista de asistencias Para visualizar registros de asistencia tanto locales como sincronizados, se implementó la vista en la ruta `/asistencias`. Esta consulta archivos JSON almacenados en LittleFS. La Figura 4.18 lo ejemplifica.



Figura 4.18: Vista de asistencias locales almacenadas en el dispositivo.

Vista de Personas registradas Para visualizar las personas registradas, se implementó la vista en la ruta `/personas`. Esta consulta archivos JSON almacenados en LittleFS y permite editar nombre/correo, actualizar huella o rostro, y eliminar personas. La Figura 4.19 lo ejemplifica.



Figura 4.19: Vista de personas registradas en el dispositivo.

Vista de Crear Turnos Para la creación de turnos y visualización de los existentes, se implementó la vista en la ruta `/gestion`, complementada con `/turnos`. Estas consultan archivos JSON almacenados en LittleFS. La Figura 4.20 lo ejemplifica.

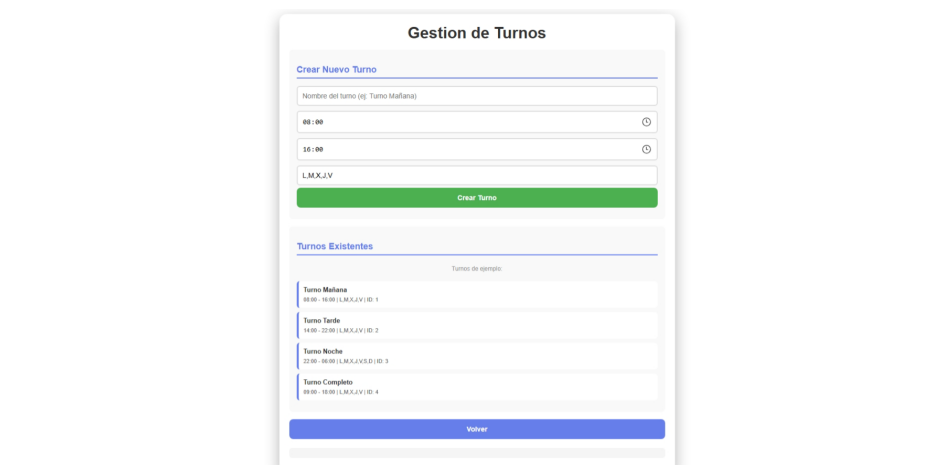


Figura 4.20: Vista de gestión de turnos en el dispositivo.

Vista de asignación de turnos Para visualizar las asignaciones de turnos y realizar la acción de asignar un turno a una persona, se implementó la vista en la ruta /asignaciones. La Figura 4.21 lo ejemplifica.



Figura 4.21: Vista de gestión de asignaciones en el dispositivo.

Pruebas

Para validar el funcionamiento del prototipo y comprobar la correcta integración entre los distintos módulos del sistema, se diseñaron pruebas de integración enfocadas exclusivamente en los componentes implementados durante esta iteración:

- **Prueba de captura de imágenes:** Verificar que la cámara OV2640 capture imágenes JPEG de forma continua y estable, sin corrupción de frames ni errores de inicialización del sensor. Se espera que la función `esp_camera_fb_get()` retorne buffers consistentes tras cada llamada.

- **Prueba de comunicación UART con AS608:** Verificar que el lector biométrico responda correctamente al comando de verificación de contraseña (`finger.verifyPassword()`) y que la comunicación serie a 57600 baudios se establezca sin pérdida de paquetes. Se espera confirmación de conexión en menos de 1 segundo tras el inicio.
- **Prueba de lectura biométrica:** Verificar que el AS608 pueda capturar una imagen dactilar (`finger.getImage()`), convertirla a plantilla (`finger.image2Tz()`) y almacenarla en un slot del sensor (`finger.storeModel()`). Se espera que el proceso completo tome menos de 2 segundos.
- **Prueba de acceso vía AP:** Verificar que al iniciar sin credenciales Wi-Fi configuradas, el ESP32-CAM active el Access Point **ESP32-ASISTENCIA** y que las páginas HTML sean accesibles desde un navegador conectado a la IP 192.168.4.1. Se espera que todas las rutas definidas respondan con código HTTP 200.
- **Prueba de carga de vistas vía Wi-Fi externa:** Verificar que al conectar el dispositivo a una red Wi-Fi configurada, las mismas vistas se carguen correctamente a través de la IP asignada por el router, manteniendo estilos, scripts y funcionalidad.

4.2.2 Iteración 2: Almacenamiento local y modo offline

La segunda iteración se centra en habilitar el funcionamiento autónomo del dispositivo sin conexión a internet, integrando un sistema de almacenamiento persistente basado en LittleFS (sucesor moderno de SPIFFS para ESP32) y los mecanismos de captura biométrica offline. Esta etapa materializa lo definido en el apartado 3.1.2 del capítulo anterior.

Análisis

Para el correcto funcionamiento offline del prototipo se identificaron los siguientes requerimientos:

- Almacenar localmente los datos de usuarios, turnos, asignaciones y asistencias en formato JSON.
- Permitir el registro de marcaciones aun cuando no exista conectividad externa, tanto por huella (offline total) como con validación de turnos locales.
- Garantizar persistencia de la información tras reinicios del ESP32-CAM o cortes de energía.
- Habilitar la gestión completa del sistema mediante una interfaz web embebida accesible vía Access Point.

- Mantener compatibilidad con la futura sincronización automática en modo online, marcando cada registro con un campo **sincronizado** (booleano) para su posterior envío al backend.

Para cumplir estos requisitos se seleccionó el sistema de archivos **LittleFS** y el formato **JSON** debido a su bajo peso, facilidad de manipulación mediante la librería **ArduinoJson** y compatibilidad con las capacidades del ESP32-CAM (4 MB de flash, de los cuales aproximadamente 1.5 MB están disponibles para LittleFS). Adicionalmente, se definió un archivo de configuración, **wifi.json**, destinado a almacenar credenciales Wi-Fi, URL del backend, dirección del broker MQTT y PIN de enrolamiento.

Diseño

Considerando la arquitectura establecida, se diseñaron las siguientes estructuras de almacenamiento en formato JSON:

- **personas.json**: Arreglo de objetos con campos **id**, **nombre**, **rut**, **email**, **huella_id**, **fecha_registro**, **sincronizado**.
- **turnos.json**: Arreglo de objetos con campos **id**, **backend_id**, **nombre**, **inicio**, **fin**, **dias**, **sincronizado**.
- **asignaciones.json**: Arreglo de objetos con campos **persona_id**, **turno_id**, **backend_id**, **fecha_asignacion**, **sincronizado**.
- **asistencias.json**: Arreglo de objetos con campos **persona_id**, **nombre**, **tipo** (entrada/salida), **metodo** (huella_offline, facial, etc.), **timestamp**, **sincronizado**.
- **wifi.json**: Objeto con campos **ssid**, **pass**, **backend**, **mqtt**, **pin** (para enrolamiento del dispositivo).
- **erp-config.json**: Arreglo con configuraciones de integraciones ERP, **sincronizado** desde el backend cuando hay conectividad.

El campo **sincronizado** (booleano) en cada entidad actúa como bandera para el proceso de sincronización diferida: cuando el dispositivo recupera conectividad, itera sobre los registros con **sincronizado = false** y los envía al backend. Una vez confirmada la recepción, se marca **sincronizado = true**.

Implementación

En esta iteración se integraron las librerías `LittleFS`, `ArduinoJson`, `HardwareSerial` y `Adafruit_Fingerprint`. Las dos primeras permiten gestionar archivos JSON alojados en la memoria flash del ESP32-CAM, mientras que las dos últimas habilitan la comunicación UART con el lector biométrico AS608 para el registro y verificación de huellas digitales.

Inicialización del sistema de archivos y persistencia El primer componente de la implementación consiste en montar `LittleFS` mediante `LittleFS.begin(true)` y verificar la existencia de los archivos necesarios. En caso de no existir, el sistema los crea con una estructura inicial vacía (arreglo `[]` para las colecciones, objeto con valores por defecto para `wifi.json`), asegurando que el dispositivo pueda operar incluso tras un primer arranque sin configuración previa.

Se implementaron dos funciones genéricas para la manipulación de archivos JSON:

- `loadArray(const char* path, DynamicJsonDocument& doc)`: Abre un archivo, deserializa su contenido como `JsonArray` y lo retorna. Si el archivo no existe o está corrupto, retorna un arreglo vacío.
- `saveArray(const char* path, DynamicJsonDocument& doc)`: Serializa el documento JSON y lo escribe en el archivo correspondiente, sobrescribiendo el contenido anterior.

Estas funciones encapsulan toda la lógica de persistencia del sistema y son reutilizadas por todos los módulos que necesiten leer o escribir datos.

El tamaño de cada `DynamicJsonDocument` se dimensionó según el volumen esperado de datos de cada archivo: `personas.json` y `turnos.json` utilizan 2048 bytes, `asignaciones.json` utiliza 1024 bytes y `asistencias.json` utiliza 8192 bytes, suficiente para aproximadamente 55 registros de asistencia. Adicionalmente, en la función `procesarAsistencia()` se incorporó una validación sobre el retorno de `createNestedObject()`: si el objeto retornado es nulo (indicando desbordamiento del documento), se registra una advertencia en el log del sistema, permitiendo detectar tempranamente cuellos de botella de memoria sin comprometer la integridad del archivo JSON.

```

JSONArray loadArray(const char* path, DynamicJsonDocument &doc) {
    File file = SPIFFS.open(path, "r");
    String content = file.readString();
    file.close();
    deserializeJson(doc, content);
    return doc.as<JSONArray>();
}

void saveArray(const char* path, DynamicJsonDocument &doc) {
    File file = SPIFFS.open(path, "w");
    serializeJson(doc, file);
    file.close();
}

```

Figura 4.22: Funciones para lectura y escritura de archivos JSON en LittleFS.

Registro de usuarios con huella digital El registro de usuarios se implementó mediante un flujo que combina la captura biométrica gestionada por el lector AS608 con el almacenamiento en `personas.json`. El lector administra internamente modelos de huellas mediante *slots* numerados del 1 al 127. Por ello, antes del enrolamiento es necesario identificar un slot disponible dentro del sensor. La función `encontrarSlotLibre()` recorre el arreglo de personas, determina el ID de huella más alto registrado y retorna el siguiente disponible.

Una vez encontrado un slot, se ejecuta el proceso de enrolamiento en dos fases:

1. **Primera captura** (`ESTADO_ESPERANDO_HUELLA_REGISTRO`): El sistema espera que el usuario coloque el dedo. Al detectar una imagen válida, la convierte a plantilla en el buffer 1 y transiciona a `ESTADO_ESPERANDO_SOLTAR_DEDO`.
2. **Espera y segunda captura** (`ESTADO_REGISTRO_SEGUNDA_HUELLA`): Una vez que el sensor detecta que el dedo fue retirado (`FINGERPRINT_NOFINGER`), el sistema solicita una segunda captura del mismo dedo. Al obtenerla, convierte a plantilla en el buffer 2, crea el modelo combinado (`finger.createModel()`) y lo almacena en el slot correspondiente (`finger.storeModel()`).

Luego del enrolamiento exitoso, se almacena un nuevo objeto dentro de `personas.json` con un identificador único (generado localmente con prefijo `local-` o provisto por el backend si hay conectividad), el nombre, RUT, correo electrónico y el slot asignado. La Figura 4.23 ilustra la máquina de estados que gobierna este proceso de enrolamiento.

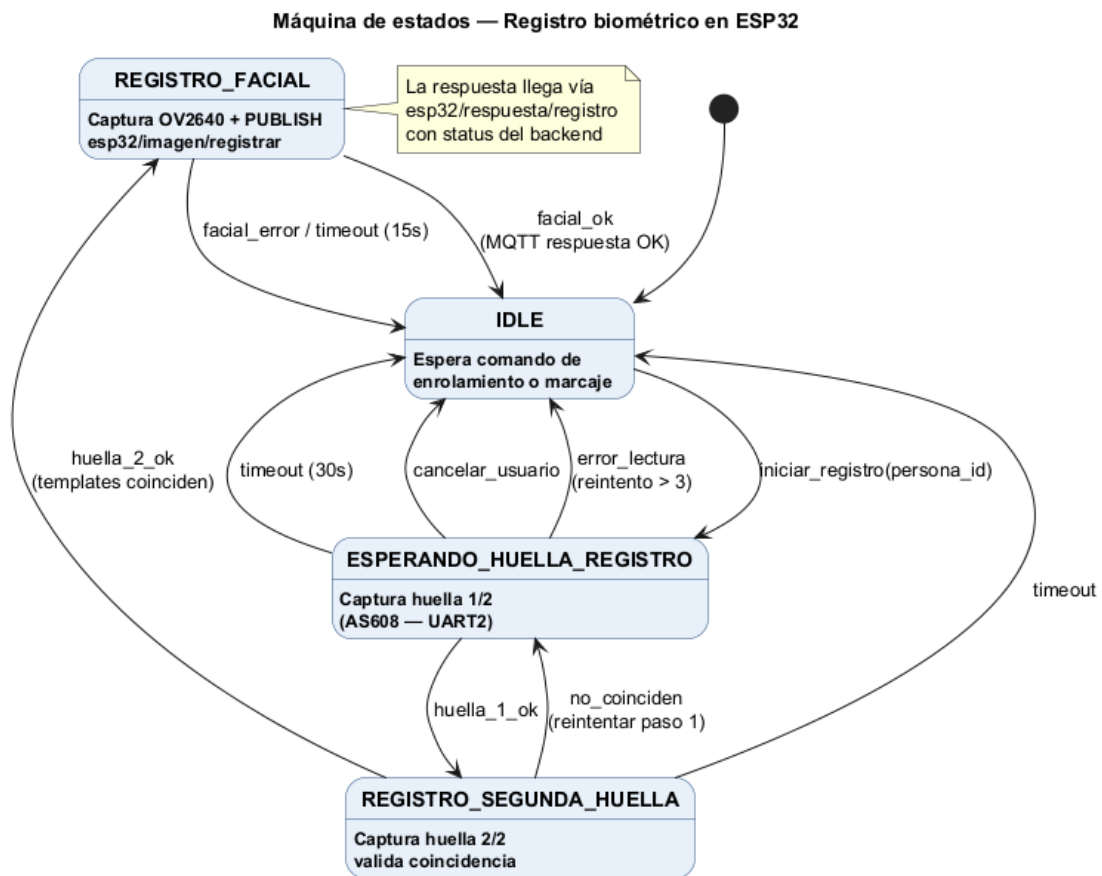


Figura 4.23: Máquina de estados del registro biométrico en el ESP32-CAM.

Registro de asistencia en modo offline El registro de asistencia se realiza completamente en el dispositivo, sin necesidad de conexión externa. El flujo implementado consiste en:

1. Capturar una huella mediante el lector AS608 (`finger.getImage()`).
2. Convertir la imagen a plantilla (`finger.image2Tz()`) y ejecutar una búsqueda biométrica (`finger.fingerSearch()`) que retorna el `fingerID` del slot coincidente.
3. Identificar al usuario asociado mediante el campo `huella_id` en `personas.json` con la función `buscarPersonaPorHuella()`.
4. Verificar que el usuario cuente con un turno asignado consultando `asignaciones.json` mediante `turnoActivo()`. Si no tiene turno, el sistema rechaza la marcación.
5. Determinar el tipo de evento (entrada/salida) alternando automáticamente respecto al último registro de la persona en `asistencias.json`.
6. Generar un nuevo registro con `sincronizado = false` (salvo que se logre enviar al backend en ese momento).

El manejador `handleMarcarAsistencia()` expone esta funcionalidad a través de la interfaz web, permitiendo también marcaciones manuales. El escaneo automático continuo se controla mediante intervalos configurables: 2 segundos entre verificaciones de huella y 6 segundos entre verificaciones faciales.

Gestión de turnos y asignaciones Además del registro de asistencias, el sistema permite crear turnos (`handleCreateTurn()`) y asignarlos a personas (`handleAssignTurn()`) desde la interfaz web. Cada operación persiste los datos en sus archivos JSON correspondientes y, si hay conectividad, intenta replicar la operación en el backend mediante HTTP POST.

Operación offline del dispositivo Con el objetivo de asegurar autonomía total, el ESP32-CAM fue configurado para operar en dos modos:

- **Modo Wi-Fi STA:** Si hay credenciales configuradas en `wifi.json`, el dispositivo intenta conectarse a la red externa. Si la conexión falla reiteradamente (5 intentos), activa el modo AP como fallback sin deshabilitar el intento de reconexión.
- **Modo Access Point:** El punto de acceso utiliza el SSID `ESP32-ASISTENCIA` y contraseña `Asistencia2026`, permitiendo que un administrador se conecte directamente al dispositivo en la IP `192.168.4.1` para realizar todas las operaciones mediante la interfaz web embebida.

Pruebas

Se diseñaron las siguientes pruebas para validar el correcto funcionamiento del almacenamiento local y el modo offline:

- **Prueba de persistencia tras reinicio:** Registrar usuarios, turnos, asignaciones y asistencias, luego reiniciar el ESP32-CAM (por corte de energía o comando software). Verificar que todos los archivos JSON conserven su contenido íntegro y que el sistema pueda leerlos correctamente tras el reinicio.
- **Prueba de lectura y deserialización:** Abrir cada archivo JSON generado y verificar que `ArduinoJson` pueda deserializar su contenido sin errores ni pérdida de campos. El tamaño de cada `DynamicJsonDocument` se configuró según la capacidad esperada del archivo: 2048 bytes para `personas.json` y `turnos.json`, 1024 bytes para `asignaciones.json` y 8192 bytes para `asistencias.json`. Se incluyó además una validación post-inserción (mediante `isNull()`) en la función `procesarAsistencia()` para detectar y registrar en logs cualquier desbordamiento del documento al crear nuevos registros de asistencia.

- **Prueba de escritura concurrente:** Realizar múltiples operaciones de escritura (registro de persona + asignación de turno + marcación de asistencia) en rápida sucesión sin reiniciar el sistema. Verificar que no se produzcan corrupciones de archivos ni pérdida de datos.
- **Prueba de marcación offline por huella:** Desconectar el dispositivo de toda red externa, registrar un usuario con huella y turno, y efectuar una marcación. Verificar que el registro aparezca en `asistencias.json` con `sincronizado = false`.
- **Prueba de alternancia entrada/salida:** Realizar dos marcaciones consecutivas para un mismo usuario. Verificar que el sistema registre correctamente “entrada” seguido de “salida”.
- **Prueba de validación de turno offline:** Intentar marcar asistencia para un usuario sin turno asignado. Verificar que el sistema rechace la marcación con el mensaje “Sin turno asignado”.
- **Prueba de funcionamiento del AP:** Desconectar el ESP32-CAM de cualquier red, verificar que el AP ESP32-ASISTENCIA se active automáticamente y que todas las operaciones de gestión (registro, turnos, asignaciones, asistencias) funcionen correctamente desde un navegador conectado a la IP 192.168.4.1.

4.2.3 Iteración 3: Backend, base de datos y comunicación

El propósito de esta iteración consiste en implementar un backend centralizado que permita procesar la información enviada por el ESP32-CAM, administrar la base de datos, gestionar registros de trabajadores y turnos, y habilitar la comunicación bidireccional mediante MQTT y HTTP. Esta etapa corresponde a la construcción del servidor del sistema, incorporando funcionalidades que serán extendidas en iteraciones posteriores.

Análisis

En esta iteración se realiza el estudio de los mecanismos de comunicación necesarios para establecer el flujo de datos entre el dispositivo ESP32-CAM y el backend del sistema, considerando tanto la operación en línea como escenarios donde la conectividad es limitada o intermitente.

En primer lugar, se evaluaron los protocolos **HTTP** y **MQTT** como alternativas principales para el envío de datos. La decisión final fue asignar a cada protocolo el rol donde ofrece mayores ventajas:

- **MQTT** (con transporte WebSockets seguros `wss://`) se utiliza exclusivamente para el **registro facial durante el enrolamiento**. En este flujo, la imagen Base64 se publica una sola vez en el tópico `esp32/imagen/registrar` con QoS 1, lo que entrega confirmación de recepción por parte del broker y permite desacoplar la captura de la espera del procesamiento (el backend responde luego por el tópico `esp32/respuesta/facial`).
- **HTTP** se utiliza para la **identificación facial durante la marcación**, enviando el JPEG crudo como `application/octet-stream` (33% más eficiente que Base64) al endpoint `POST /api/facial/identificar`, así como para todas las operaciones REST (CRUD de personas, turnos, asignaciones, asistencias), sincronización diferida, consulta de configuración ERP y enrolamiento del dispositivo.

Esta combinación permite equilibrar la naturaleza asíncrona de MQTT (idónea para el registro, que puede tardar segundos en procesarse) con la simplicidad síncrona de HTTP (idónea para la marcación, que requiere respuesta inmediata).

Asimismo, se analizó la **conectividad Wi-Fi** como medio de enlace principal del dispositivo. En escenarios con conexión estable, el ESP32-CAM transmite imágenes al broker MQTT y consultas HTTP al backend; mientras que, en situaciones de baja o nula conectividad, el dispositivo almacena registros localmente (Iteración 2) y los reenvía cuando la red se restablece. Se definieron mecanismos de reconexión progresiva con backoff exponencial (3, 6, 9, 12 y 15 segundos entre intentos), manejo de desconexiones mediante el evento `ARDUINO_EVENT_WIFI_STA_DISCONNECTED` y un watchdog que intenta restaurar la conectividad periódicamente.

Finalmente, se evaluó la compatibilidad del flujo de datos con la arquitectura general definida en el apartado 4.1.1, determinando que esta iteración corresponde a la habilitación de la capa de comunicación que enlaza el dispositivo con el backend.

Diseño

El diseño de esta iteración se centra en la definición detallada de los mecanismos de comunicación y conectividad, así como en la arquitectura de la base de datos que soportará todas las funcionalidades del sistema.

El backend se organiza de forma modular mediante **blueprints de Flask**, cada uno responsable de un dominio específico del sistema: personas, turnos, asignaciones, asistencias, reconocimiento facial, dispositivos, logs, integraciones ERP y autenticación. Esta separación facilita el mantenimiento y la escalabilidad del código.

La API REST se diseña para soportar las funcionalidades de gestión operativa del sistema. Los endpoints definidos para esta iteración incluyen:

- **GET /api/personas:** Obtiene el listado de personas registradas (con filtro por roles y empresa en iteraciones futuras).
- **POST /api/personas:** Registra una nueva persona (nombre, RUT, email, huella_id).
- **GET /api/turnos:** Devuelve todos los turnos almacenados.
- **POST /api/turnos:** Permite crear un nuevo turno indicando nombre, hora de inicio, hora de término y días.
- **DELETE /api/turnos/<id>:** Elimina un turno específico.
- **GET /api/asignaciones:** Lista todas las asignaciones activas entre personas y turnos.
- **POST /api/asignaciones:** Asigna un turno a una persona.
- **DELETE /api/asignaciones/<id>:** Elimina una asignación.
- **POST /api/asistencias:** Registra una asistencia enviada desde el dispositivo.
- **POST /api/asistencias/sync:** Recibe un lote de asistencias acumuladas en modo offline y las inserta con deduplicación.

Todos estos endpoints operan sobre una base de datos relacional PostgreSQL. El diseño del esquema incluye las siguientes tablas:

- **empresas:** Entidad multi-tenant con nombre, RUT empresa, email de contacto, teléfono y dirección.
- **dispositivos:** Registro de cada ESP32-CAM enrolado, con MAC address, IP local, estado (activo/inactivo), último heartbeat y código de enrolamiento (PIN de 8 caracteres).
- **usuarios_web:** Usuarios del panel web backend con autenticación por email y contraseña (hash bcrypt).
- **usuario_empresa:** Tabla de relación muchos-a-muchos entre usuarios y empresas, con rol (admin, empleador, trabajador).
- **personas:** Trabajadores registrados con nombre, RUT (único), email, huella_id, encoding facial (cifrado) y empresa asociada.
- **turnos:** Jornadas laborales con nombre, hora de inicio, hora de fin, días y empresa asociada.

- **asignaciones**: Relación entre personas y turnos, con vigencia.
- **asistencias**: Registros de marcación con persona, dispositivo, tipo (entrada/salida), método, timestamp, origen y estado de sincronización.
- **sincronizacion_log**: Bitácora de procesos de sincronización con conteo de registros enviados y procesados correctamente.
- **integraciones_erp**: Configuración de webhooks para integración con sistemas ERP externos (nombre, tipo, URL, headers, field mapping, envío automático).
- **consentimientos**: Registro de aceptación de política de privacidad biométrica por persona.
- **logs_biometricos**: Auditoría de operaciones biométricas (registro, verificación, identificación) con resultado, timestamp e IP de origen.
- **eliminaciones_biometricas**: Registro histórico de datos biométricos eliminados (embeddings anteriores, fotos) para cumplimiento normativo y trazabilidad.
- **encodings_faciales**: Almacena múltiples embeddings faciales cifrados por persona (FOREIGN KEY con eliminación en cascada), permitiendo que un trabajador tenga varias fotos de referencia con distintos ángulos e iluminación para mejorar la precisión del reconocimiento.

La Figura 4.24 presenta el diagrama entidad-relación correspondiente. En él se visualizan las 14 tablas que componen el esquema, organizadas en torno a tres dominios principales: **gestión de personas y turnos** (personas, turnos, asignaciones), **registro biométrico y de asistencia** (asistencias, encodings_faciales, logs_biometricos, consentimientos, eliminaciones_biometricas), y **administración del sistema** (empresas, dispositivos, usuarios_web, usuario_empresa, integraciones_erp, sincronizacion_log). Las flechas indican las relaciones de clave foránea entre tablas, destacando cómo la entidad **empresas** actúa como raíz del modelo multi-tenant, la tabla **personas** como eje central que conecta la información laboral con los datos biométricos, y las tablas **asistencias** y **sincronizacion_log** como destino final del flujo de datos operacional.

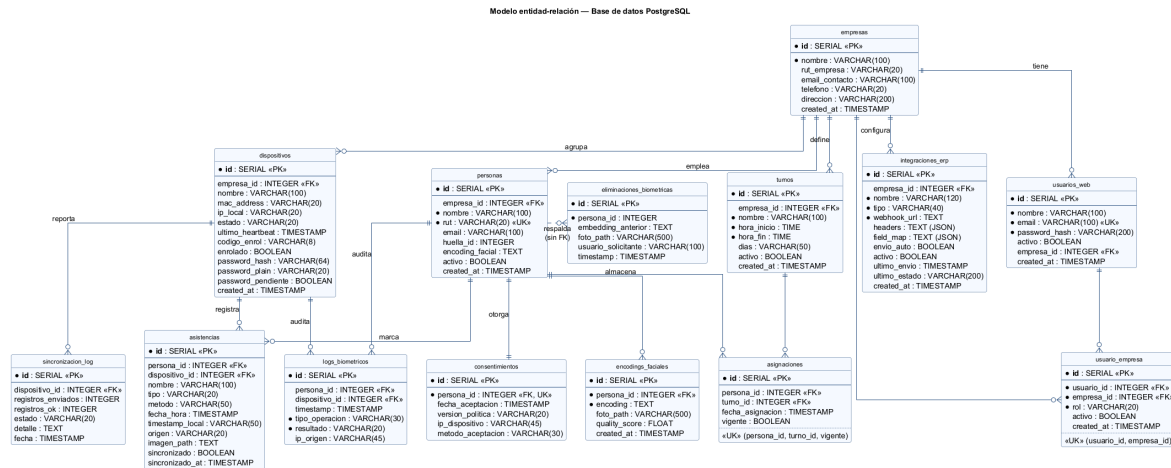


Figura 4.24: Diagrama entidad-relación de la base de datos (14 tablas).

La comunicación MQTT se diseña para transmitir imágenes faciales de **registro** desde el ESP32-CAM hacia el backend en un único mensaje JSON por el tópico `esp32/imagen/registrar`, conteniendo el rut y la imagen codificada en Base64. El backend, mediante un cliente MQTT (librería `paho-mqtt` de Python), se suscribe a este tópico, decodifica la imagen y procesa el enrolamiento facial. Es importante destacar que las imágenes de **identificación** (consulta 1:N durante la marcación) **no viajan por MQTT**, sino por HTTP al endpoint `POST /api/facial/identificar` como JPEG crudo en formato `application/octet-stream`.

Adicionalmente, se definen los tópicos `esp32/heartbeat/<MAC>` para monitorizar la actividad de los dispositivos y `esp32/lwt/<MAC>` para detectar desconexiones mediante el mecanismo de Last Will Testament (LWT). El backend recibe las respuestas del procesamiento facial por el tópico `esp32/respuesta/facial`, que el ESP32-CAM escucha para confirmar el éxito o fallo del registro. Para la detección activa de desconexión, se implementó el tópico `esp32/ping/<MAC>`: el backend publica un ping cada 30 segundos; si el ESP32-CAM no responde con un heartbeat dentro de 60 segundos, se marca como inactivo. La Figura 4.25 resume la topología de publicación y suscripción.

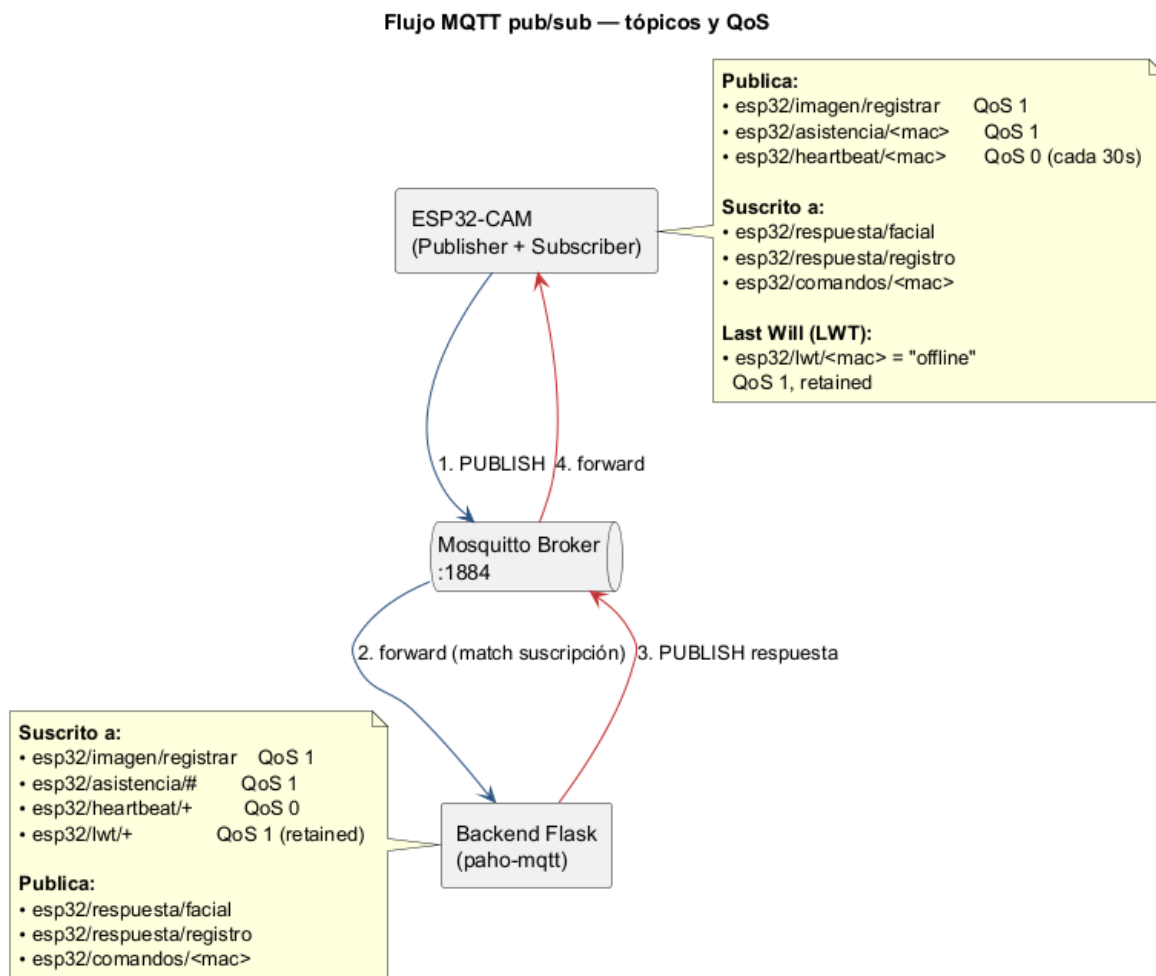


Figura 4.25: Flujo MQTT pub/sub — tópicos, direcciones y QoS entre el ESP32-CAM, el broker Mosquitto y el backend.

La infraestructura se complementa con **Docker Compose**, que levanta el broker Mosquitto en un contenedor independiente, exponiendo el puerto **1884** (externo, mapeado al 1883 interno del contenedor) para MQTT nativo y el puerto 9001 para WebSockets.

Implementación

La implementación de la Iteración 3 se centró en habilitar la comunicación entre el dispositivo ESP32-CAM y el backend desarrollado en Python con Flask, así como en construir la base de datos PostgreSQL que soporta todo el sistema.

Inicialización de la base de datos Se implementó el módulo `database.py` con una función `init_db()` que crea todas las tablas del sistema utilizando sentencias `CREATE TABLE IF NOT EXISTS`, asegurando idempotencia en los despliegues. Adicionalmente, se

aplican sentencias `ALTER TABLE ADD COLUMN IF NOT EXISTS` para garantizar migraciones no destructivas. La función incluye la creación de índices sobre columnas frecuentemente consultadas (`persona_id`, `dispositivo_id`, `fecha_hora`, `empresa_id`) y la inserción de datos semilla (empresa por defecto, usuario administrador con contraseña hasheada).

Conexión Wi-Fi del ESP32-CAM y gestión de conectividad Para permitir la operación en red, el dispositivo implementa un mecanismo de conexión automática a Wi-Fi mediante la función `tryConnectWiFi()`. Esta función lee las credenciales desde `wifi.json`, configura el modo STA y deshabilita el ahorro de energía Wi-Fi (`WIFI_PS_NONE`) para mantener una conexión estable. La verificación continua de conectividad se realiza en `verificarConexionWiFi()`, que implementa un mecanismo de reintentos con backoff progresivo: tras detectar una desconexión, espera un tiempo creciente antes de reintentar (3s, 6s, 9s, 12s, 15s). Si tras 5 intentos fallidos no se logra reconectar, el dispositivo activa el Access Point como fallback.

El evento `ARDUINO_EVENT_WIFI_STA_DISCONNECTED` queda registrado con su código de razón y un contador de desconexiones, información expuesta a través del endpoint `/wifi-diag` para diagnóstico remoto.

Envío de imágenes faciales: MQTT para registro, HTTP para identificación Siguiendo el diseño establecido, durante el **registro facial** la imagen es capturada, codificada en Base64 mediante una implementación optimizada que evita el uso de buffers intermedios, y enviada como un único mensaje JSON al tópico `esp32/imagen/regarstrar` con QoS 1 (entrega garantizada). El payload contiene dos campos: `rut` y `imagen` (string Base64). Este enfoque evita la complejidad de la fragmentación en múltiples mensajes y es factible porque el ESP32-CAM puede manejar cadenas de hasta 80 KB en memoria con calidad JPEG reducida (`q=10` para registro).

La función `registrarRostroEnBackend()` encapsula este flujo y retorna `true` si la publicación MQTT fue exitosa. La respuesta del backend llega luego al tópico `esp32/respuesta/facial`, donde el firmware confirma visualmente el éxito (`flashExito()`) o el error (`flashError()`).

En contraste, durante la **identificación facial** (consulta 1:N al momento de marcar asistencia), el flujo es diferente: la imagen se envía por **HTTP** y no por MQTT. La función `identificarPorRostro()` reduce la calidad JPEG a 10, captura el frame directamente del buffer de la cámara (`camera_fb_t`) y lo transmite como `application/octet-stream` mediante un `HTTPClient.POST(fb->buf, fb->len)` al endpoint `POST /api/facial/identificar`, evitando la sobrecarga de codificación Base64. El backend responde sincrónicamente con el `rut` identificado (o HTTP 404 si el rostro no coincide). Esta decisión se tomó porque la marcación requiere respuesta

inmediata para validar la identidad y proceder con el registro de asistencia en el mismo flujo.

Recepción de imágenes en el backend vía MQTT El backend implementa un cliente MQTT mediante la librería `paho-mqtt` en el módulo `mqtt_handler.py`. Al recibir un mensaje en `esp32/imagen/registrar`, la función `on_message` decodifica el JSON, extrae el `rut` y la imagen Base64, verifica que exista consentimiento biométrico registrado en la base de datos, resuelve el `rut` al `id` interno, decodifica la imagen, la guarda en `static/previews/<id>.jpg` y extrae el `embedding` facial. Si el procesamiento es exitoso, publica en `esp32/respuesta/facial` un JSON con `status: "ok"` y el nombre del archivo; en caso de error, publica `status: "error"` con el detalle.

Adicionalmente, el cliente MQTT procesa los heartbeats (`esp32/heartbeat/<MAC>`) actualizando el estado y la IP del dispositivo en la base de datos, y los mensajes LWT (`esp32/lwt/<MAC>`) marcando el dispositivo como inactivo. También se implementó un hilo `device_pinger()` que publica pings en el tópico `esp32/ping/<MAC>` cada 30 segundos como mecanismo de verificación activa de conectividad. Cada cambio de estado (heartbeat, LWT, ping, watchdog) se difunde a los clientes web mediante el streaming SSE (`/sse/devices`) para actualizar el panel en tiempo real.

Mantenimiento de la conexión MQTT en el ESP32-CAM La función `mantenerConexionMQTT` se encarga de iniciar el cliente MQTT en el ESP32-CAM si el broker está configurado y hay conectividad. Soporta URLs con esquema `mqtt://`, `ws://` o `wss://` (WebSockets seguros con TLS mediante `esp_crt_bundle_attach`). Configura un mensaje LWT para que el backend sea notificado si el dispositivo se desconecta abruptamente, y se suscribe al tópico `esp32/respuesta/facial` para recibir confirmaciones del backend, así como al tópico `esp32/ping/<MAC>` para responder a los pings de conectividad activa del backend. Al recibir un ping, el ESP32-CAM responde publicando inmediatamente un heartbeat con el campo `"pong": true`, lo que permite al backend verificar que el dispositivo sigue en línea.

Envío de datos mediante HTTP desde el ESP32-CAM Se implementó la capacidad del ESP32-CAM para comunicarse con la API REST del backend mediante solicitudes HTTP usando la librería `HTTPClient`. La función auxiliar `beginHttp()` configura cada petición con timeout de 10 segundos y el header `X-Device-MAC` para que el backend pueda identificar al dispositivo emisor. Esta función es utilizada por todos los handlers que requieren comunicación con el backend:

- `sincronizarPersonasDesdeBackend()`: Descarga la lista completa de personas desde `GET /api/personas` y la persiste en `personas.json`.

- `sincronizarTurnosDesdeBackend()`: Descarga turnos desde GET `/api/turnos`.
- `sincronizarAsignacionesDesdeBackend()`: Descarga asignaciones desde GET `/api/asignaciones`.
- `crearTurnoEnBackend()`: Crea un turno en el backend vía POST `/api/turnos`.
- `postAsistenciaEnBackend()`: Envía una asistencia individual vía POST `/api/asistencias`.
- `sincronizarAsistencias()`: Envía un lote de asistencias pendientes vía POST `/api/asistencias/sync`.

Implementación de la API REST del backend El backend se implementó con Flask, organizando las rutas en blueprints independientes por dominio:

- `personas_bp` (`routes/personas.py`): CRUD de personas con validación de email, unicidad de RUT y manejo de empresa.
- `turnos_bp` (`routes/turnos.py`): CRUD de turnos con filtro por empresa.
- `asignaciones_bp` (`routes/asignaciones.py`): CRUD de asignaciones con verificación de pertenencia a empresa.
- `asistencias_bp` (`routes/asistencias.py`): Registro de asistencias individuales y sincronización por lotes.
- `facial_bp` (`routes/facial.py`): Endpoints de reconocimiento facial (registro, verificación, identificación 1:N).
- `dispositivos_bp` (`routes/dispositivos.py`): Gestión y verificación de dispositivos.
- `logs_bp` (`routes/logs.py`): Consulta de logs de sincronización.
- `erp_bp` (`routes/erp.py`): CRUD de integraciones ERP y envío de datos.
- `auth_bp` (`routes/auth.py`): Autenticación de usuarios del panel web.

Cada blueprint se registra en la aplicación Flask principal (`app.py`), junto con la configuración de CORS para permitir peticiones desde cualquier origen. El backend se ejecuta en el puerto 5000 con modo debug activado y recarga automática deshabilitada para evitar conflictos con el cliente MQTT.

Infraestructura con Docker Compose Para garantizar un entorno reproducible, se definió un archivo `docker-compose.yml` que levanta el broker Mosquitto (imagen `eclipse-mosquitto:latest`) con el puerto **1884** (externo, mapeado al 1883 interno) para MQTT nativo y el puerto 9001 para WebSockets expuestos, volúmenes para configuración, datos y logs persistentes, y una red bridge dedicada llamada `teleasist_network`. El backend Flask y la base de datos PostgreSQL pueden ejecutarse en el host o en contenedores adicionales según la configuración de despliegue. La Figura 4.26 presenta el diagrama de despliegue completo.

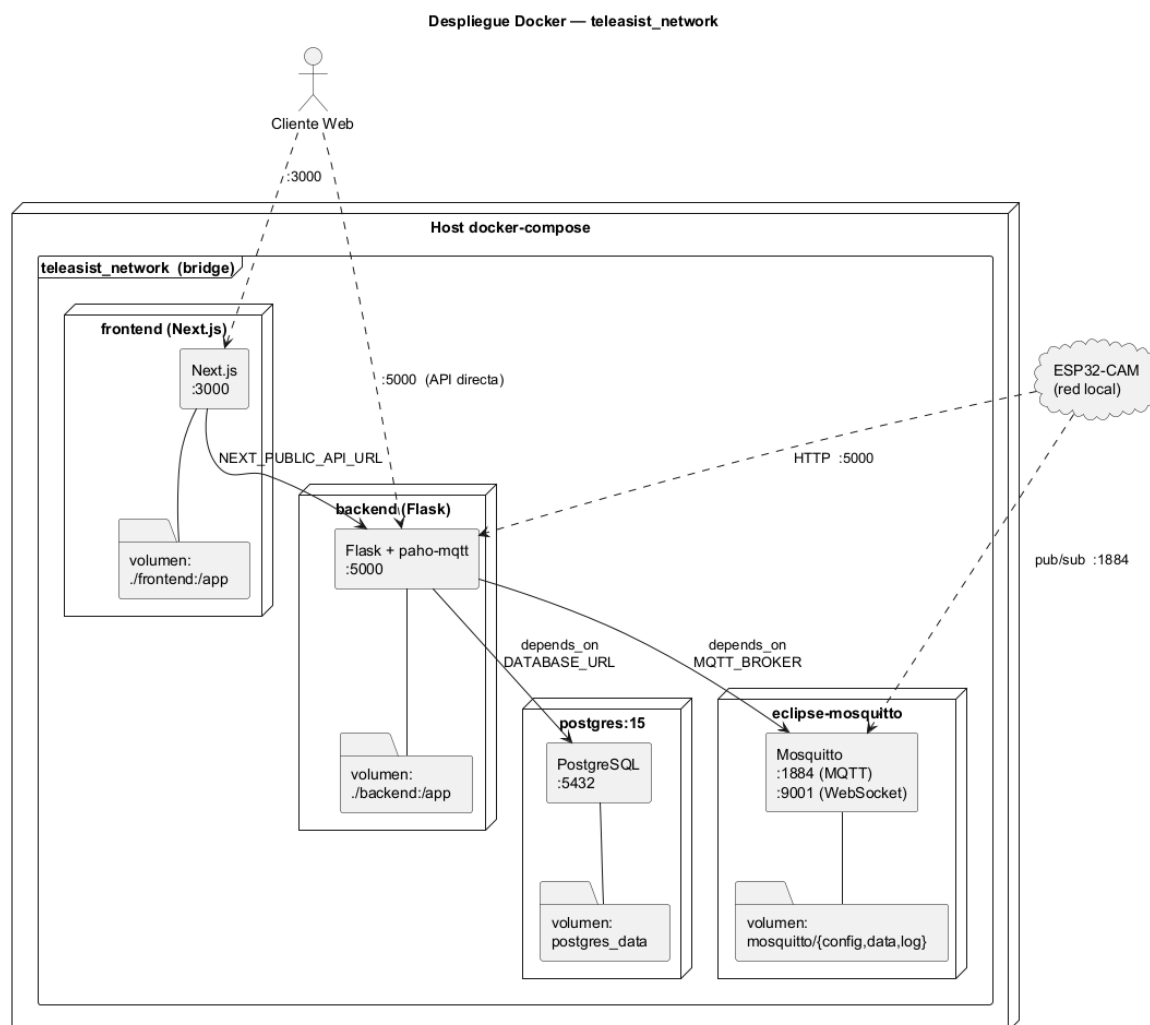


Figura 4.26: Arquitectura de despliegue Docker: servicios, puertos, volúmenes y dependencias en la red `teleasist_network`.

Pruebas

Se diseñaron las siguientes pruebas para validar la comunicación y el backend:

- **Prueba de conexión Wi-Fi y reconexión:** Configurar credenciales válidas en

el ESP32-CAM, verificar que se conecte correctamente y obtenga una IP. Luego, apagar el router, verificar que el dispositivo detecte la desconexión mediante el evento de sistema y active el mecanismo de reintentos. Finalmente, encender el router y verificar que el dispositivo reconecte automáticamente en menos de 30 segundos.

- **Prueba de envío MQTT:** Capturar una imagen desde el ESP32-CAM y publicarla en el tópico `esp32/imagen/registrar`. Verificar que el backend reciba el mensaje, decodifique la imagen, la guarde en disco y publique una respuesta en `esp32/respuesta/facial` que el ESP32-CAM reciba correctamente.
- **Prueba de heartbeat y LWT:** Verificar que, estando el ESP32-CAM conectado, el backend reciba heartbeats periódicos (cada 30 segundos) y actualice el campo `ultimo_heartbeat` en la tabla `dispositivos`. Desconectar el dispositivo abruptamente y verificar que el mensaje LWT se publique y el backend marque el dispositivo como `inactivo` en menos de 10 segundos.
- **Prueba de ping/pong activo:** Conectar el ESP32-CAM al broker MQTT. Verificar que el backend publique `esp32/ping/<MAC>` periódicamente (cada 30 segundos) y que el ESP32-CAM responda con un heartbeat que contenga el campo `"pong": true`. Detener la conectividad del ESP32-CAM y verificar que, tras 60 segundos sin respuesta, el backend marque el dispositivo como `inactivo`.
- **Prueba de recepción de datos en backend vía HTTP:** Enviar solicitudes POST a `/api/personas`, `/api/turnos`, `/api/asignaciones` y `/api/asistencias` desde el ESP32-CAM. Verificar que cada solicitud sea procesada correctamente por el backend (código HTTP 200/201) y que los datos queden persistidos en PostgreSQL.
- **Prueba de sincronización por lotes:** Generar 5 marcaciones offline en el ESP32-CAM, conectar el dispositivo a la red y ejecutar la sincronización. Verificar que los 5 registros lleguen al backend vía POST `/api/asistencias/sync`, que se inserten correctamente y que el dispositivo marque cada registro como `sincronizado = true`.
- **Prueba de inicialización de base de datos:** Ejecutar `init_db()` sobre una base de datos vacía. Verificar que se creen las 14 tablas, los índices y los datos semilla (empresa por defecto y usuario administrador) sin errores.
- **Prueba de latencia de transmisión:** Medir el tiempo total desde que el ESP32-CAM publica una imagen en MQTT hasta que recibe la respuesta de

confirmación del backend. Se espera un tiempo menor a 5 segundos en condiciones de red local.

Nuevos tópicos MQTT para control y notificación En iteraciones posteriores se agregaron los siguientes tópicos al ecosistema MQTT:

- **backend/comando/<MAC>**: Permite al backend enviar comandos remotos al ESP32-CAM. La función `enviar_comando_dispositivo()` en `mqtt_handler.py` publica un JSON con el campo `comando` en este tópico.
- **backend/notificacion/<MAC>**: Utilizado por el módulo `eventos_mqtt.py` para notificar a los dispositivos que deben sincronizar sus datos. Cada vez que se crea, actualiza o elimina una persona, turno o asignación, la función `notificar_sincronizacion()` publica un mensaje indicando el tipo de entidad y la acción realizada.
- **esp32/huella/resultado/<huella_id>**: Tópico en el que el ESP32-CAM publica el resultado del enrolamiento de una huella. El backend se suscribe a `esp32/huella/resultado/#` y procesa la respuesta, actualizando la persona correspondiente y notificando a través de SSE.

Módulo `eventos_mqtt.py` Se implementó el módulo `eventos_mqtt.py` con la función `notificar_sincronizacion(empresa_id, tipo, accion, registro_id)`. Esta función consulta los dispositivos enrolados de la empresa y publica un mensaje en `backend/notificacion/<MAC>` por cada uno, permitiendo que los ESP32-CAM reciban notificaciones en tiempo real sobre cambios en personas, turnos y asignaciones. Es invocada desde `routes/personas.py`, `routes/turnos.py` y `routes/asignaciones.py` tras cada operación de creación, actualización o eliminación.

Streaming SSE de huellas Para mantener actualizados los clientes web durante el enrolamiento de huellas, se implementó el endpoint `GET /sse/huellas` en `app.py`. La función `broadcast_huella_update()` difunde los resultados del enrolamiento (recibidos por MQTT en `esp32/huella/resultado/#`) a todos los clientes conectados al SSE, permitiendo que el panel web muestre en tiempo real el progreso del registro biométrico.

4.2.4 Iteración 4: Reconocimiento facial, anti-spoofing y cifrado biométrico

Esta iteración incorpora el procesamiento biométrico en el backend: reconocimiento facial con DeepFace, validación anti-spoofing y cifrado de embeddings faciales. El sistema pasa de solo transmitir imágenes a realizar verificación e identificación biométrica.

Análisis

Para incorporar el reconocimiento facial al sistema, se evaluaron diversas librerías y modelos de *deep learning*. Se seleccionó **DeepFace** por las siguientes razones:

- Soporta múltiples modelos de reconocimiento facial (Facenet, VGG-Face, ArcFace, etc.) y detectores (RetinaFace, MTCNN, OpenCV, etc.).
- Incluye funcionalidad de **anti-spoofing** integrada, capaz de distinguir entre un rostro real y una fotografía o pantalla, analizando textura, profundidad y reflectancia de la piel.
- Su API en Python es sencilla de integrar con Flask y compatible con las imágenes JPEG capturadas por la cámara OV2640 del ESP32-CAM.
- Facenet genera embeddings de 128 dimensiones, optimizando el almacenamiento y la velocidad de comparación.

Se identificaron los siguientes requisitos funcionales para esta iteración:

- **Registro facial:** Recibir una imagen del ESP32-CAM, extraer el embedding facial, validar que el rostro no esté duplicado en la base de datos (detección de identidades repetidas) y almacenar el embedding cifrado.
- **Identificación 1:N:** Dada una imagen capturada en vivo, comparar su embedding contra todos los embeddings almacenados y retornar la identidad con mayor coincidencia, si supera el umbral de similitud.
- **Verificación 1:1:** Dada una imagen y un `rut`, verificar si la imagen corresponde a la persona declarada.
- **Anti-spoofing:** Detectar intentos de suplantación mediante fotografías impresas, pantallas de dispositivos o máscaras, rechazando la captura si no se detecta un rostro real.
- **Cifrado de embeddings:** Los vectores de características faciales (embeddings) son datos biométricos sensibles. Para protegerlos, se deben cifrar antes de persistirlos en la base de datos, utilizando una clave derivada de una variable de entorno.
- **Consentimiento biométrico:** Todo registro facial debe estar precedido por la aceptación explícita de una política de privacidad por parte del trabajador, en cumplimiento de la normativa de protección de datos.

Diseño

El flujo de procesamiento facial se divide en tres operaciones principales:

1. Registro (POST /api/facial/registrar):

- Recibe `rut` e `imagen` (Base64).
- Verifica que exista consentimiento biométrico en la tabla `consentimientos`. Si no existe, rechaza con HTTP 403.
- Decodifica la imagen, la guarda como `static/previews/<id>.jpg`.
- Aplica un **filtro de calidad de imagen** que mide la nitidez mediante la varianza del Laplaciano. Si la imagen es demasiado plana o borrosa (por debajo de un umbral configurable, 50 por defecto), se rechaza como posible intento de suplantación (foto de pantalla o papel) antes de llegar a DeepFace, ahorrando procesamiento innecesario del modelo.
- Extrae el embedding con DeepFace usando el modelo Facenet y un detector configurable mediante variable de entorno (`FACIAL_DETECTOR`), utilizando MTCNN por defecto por su velocidad (tres veces más rápido que RetinaFace con precisión comparable) y sin anti-spoofing en esta etapa.
- Cifra el embedding con Fernet y lo almacena en la tabla `encodings_faciales`, que permite acumular múltiples fotos de referencia por persona con distintos ángulos e iluminación, mejorando la precisión en subsecuentes identificaciones.

La Figura 4.27 ilustra el flujo completo de enrolamiento biométrico, desde la creación de la persona en el panel web por parte del administrador, pasando por el registro de huella en dos pasos en el ESP32-CAM, hasta la captura facial enviada por MQTT al backend donde se extrae, cifra y almacena el embedding.

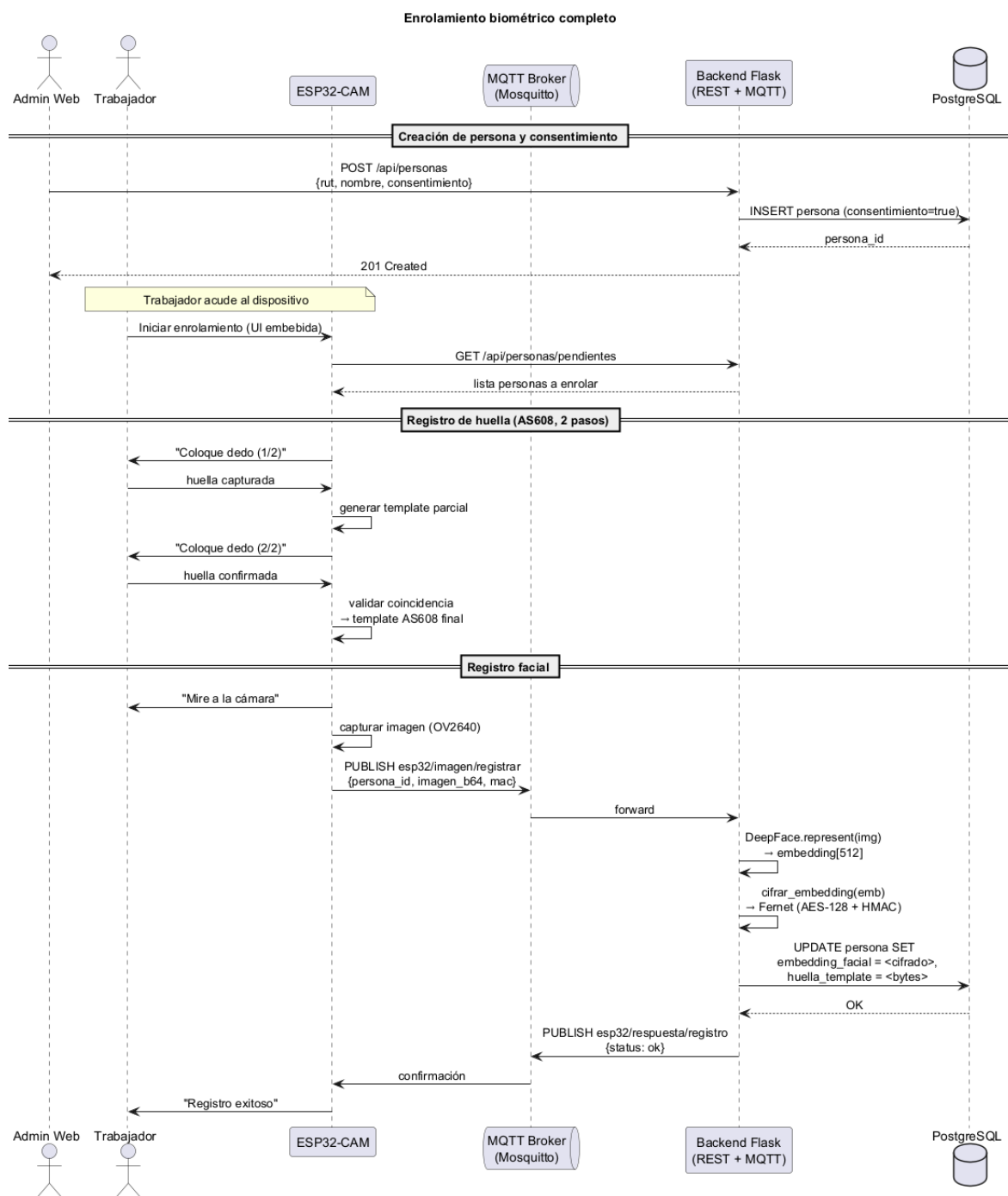


Figura 4.27: Diagrama de secuencia del enrolamiento biométrico completo.

2. Identificación 1:N (POST /api/facial/identificar):

- Recibe la imagen como JPEG crudo (`application/octet-stream`) desde el ESP32-CAM, o como JSON con Base64 desde la web.
- Guarda una copia de auditoría en `static/capturas_prueba/` con timestamp.
- Aplica el mismo filtro de calidad Laplacian como barrera anti-spoofing previa,

rechazando imágenes de baja nitidez antes de invocar DeepFace.

- Extrae el embedding con DeepFace (anti-spoofing activado mediante `anti_spoofing=True`).
- Consulta una **caché en memoria** de embeddings (con TTL configurable, 60 segundos por defecto). Si la caché expiró, recarga todos los embeddings desde la base de datos. Para cada persona, compara contra **todos los embeddings** almacenados en `encodings_faciales` (multi-encoding) y selecciona la menor distancia. Esto mejora la precisión porque el sistema puede reconocer a un trabajador aunque la captura en vivo tenga una iluminación o ángulo diferentes a los de su foto de registro.
- Si la menor distancia es inferior al umbral (10.0 para Facenet), retorna el `rut` correspondiente. En caso contrario, retorna HTTP 404.

La Figura 4.28 presenta el ciclo de marcaje automático por centinela facial. Cada 6 segundos el ESP32-CAM evalúa el sensor PIR, captura una imagen, la envía por HTTP al endpoint de identificación, y según la respuesta del backend registra la asistencia y dispara el push asíncrono a los ERP configurados.

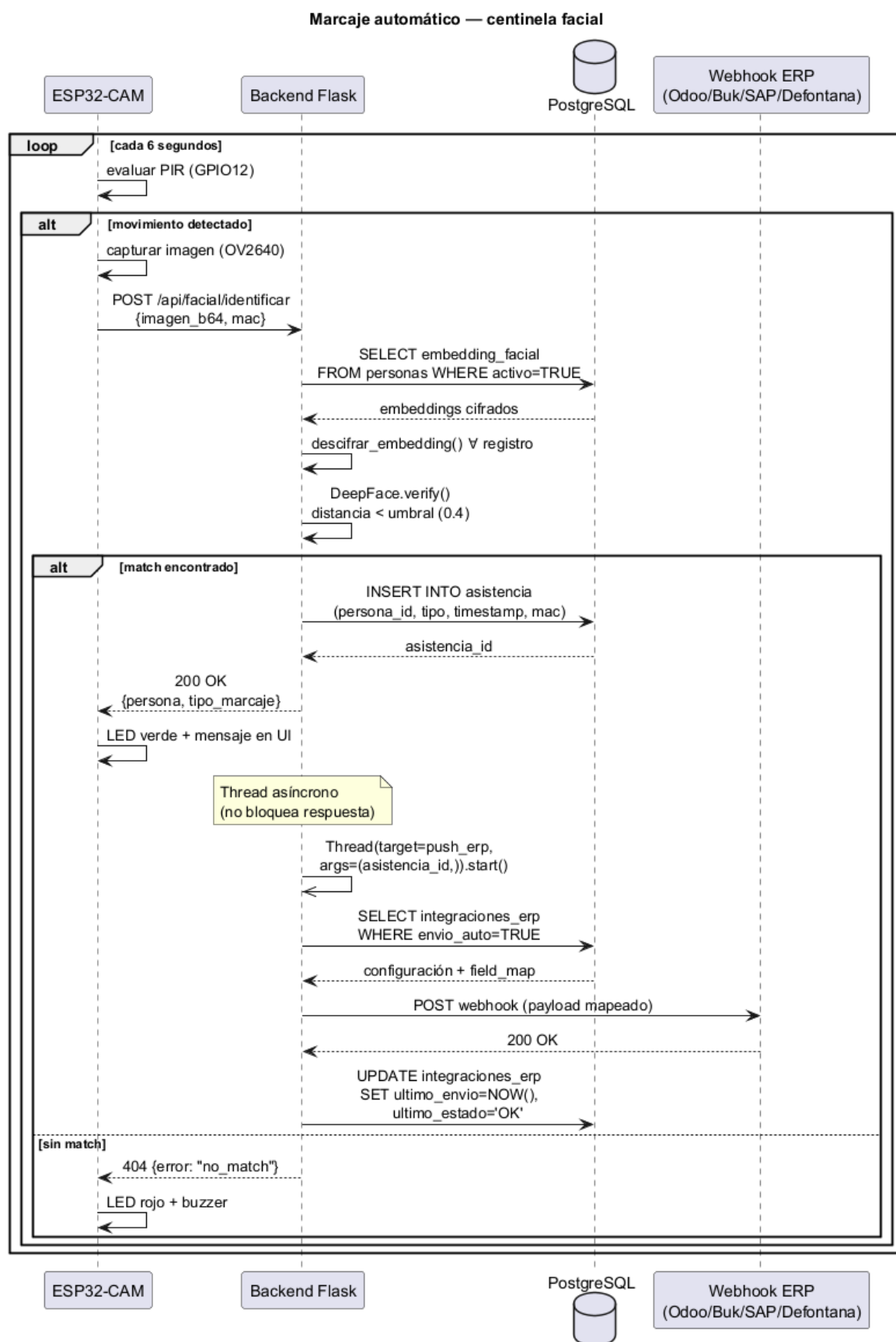


Figura 4.28: Marcaje automático — centinela facial con identificación 1:N y push asíncrono a ERP.

3. Verificación 1:1 (POST /api/facial/verificar):

- Recibe `rut` e `imagen` (Base64).
- Descifra todos los embeddings almacenados para esa persona en `encodings_faciales` y extrae el embedding de la imagen capturada (con anti-spoofing activado).
- Calcula la distancia euclidiana contra cada embedding de referencia y selecciona la menor. Si es menor que 10.0, retorna éxito con el nivel de confianza calculado como $\max(0, (1 - \text{distancia}/20) * 100)$.

El modelo seleccionado es **Facenet** por su equilibrio entre precisión y tamaño de embedding (128 dimensiones). El detector facial es **configurable mediante variable de entorno** (`FACIAL_DETECTOR`), utilizando **MTCNN** por defecto. Si bien RetinaFace ofrece mayor robustez ante variaciones extremas de pose e iluminación, MTCNN es tres veces más rápido y mantiene una precisión comparable para las condiciones típicas de una cámara de control de asistencia (rostro frontal a corta distancia), lo que lo hace más adecuado para un dispositivo embebido como el ESP32-CAM. El umbral de 10.0 para la distancia euclidiana fue elegido con base en la documentación de DeepFace y ajustes empíricos realizados durante el desarrollo.

Para el cifrado de embeddings se utiliza el algoritmo **Fernet** (AES-128 en modo CBC con HMAC-SHA256 para autenticación), parte de la librería `cryptography` de Python. La clave de cifrado se deriva aplicando SHA-256 a una variable de entorno (`BIOMETRIC_KEY`), garantizando que los embeddings nunca se almacenen en texto plano en la base de datos.

Implementación

Procesamiento de imágenes en el backend El módulo `routes/facial.py` implementa los tres endpoints descritos en el diseño. Las funciones auxiliares incluyen:

- `_validar_calidad_imagen(img_path)`: Aplica el filtro Laplaciano (varianza del Laplaciano) sobre la imagen decodificada para medir su nitidez. Si el valor está por debajo del umbral configurable en `FACIAL_NITIDEZ_UMBRAL` (50 por defecto), la imagen se considera borrosa o plana (posible foto de pantalla o papel) y se rechaza antes de llegar a DeepFace, funcionando como una capa adicional de anti-spoofing.
- `extraer_embedding(img_path, anti_spoofing)`: Invoca `DeepFace.represent()` con el modelo Facenet pre-cargado y el detector configurable vía variable de entorno. Si `anti_spoofing=True`, activa la detección de suplantación.

- `guardar_imagen_de_registro(persona_id, imagen_b64)`: Decodifica la imagen Base64, la convierte a RGB con Pillow y la guarda en `static/previews/<id>.jpg`.
- `_verificar_consentimiento(persona_id)`: Consulta la tabla `consentimientos` para verificar que exista un registro de aceptación.
- `_log_biometrico(persona_id, dispositivo_id, tipo_operacion, resultado)`: Inserta un registro de auditoría en `logs_biometricos`.

Además, para optimizar el rendimiento, el modelo **Facenet** se **precarga al iniciar el servidor** mediante `DeepFace.build_model('Facenet', task='facial_recognition')`, eliminando la latencia de carga de pesos en la primera solicitud. Los embeddings almacenados en la base de datos se mantienen en una **caché en memoria** con tiempo de vida configurable (por defecto 60 segundos, mediante `FACIAL_CACHE_TTL`), evitando consultas repetitivas y descifrados a la base de datos en cada identificación. La caché se invalida automáticamente al registrar una nueva foto o al eliminar datos biométricos, garantizando consistencia.

Procesamiento facial vía MQTT El módulo `mqtt_handler.py` complementa los endpoints REST con un canal de procesamiento asíncrono. La función `procesar_imagen_facial(cliente, persona_id, imagen_b64)` implementa el mismo flujo de registro facial pero activado por la recepción de un mensaje MQTT en `esp32/imagen/registrar`. Publica el resultado en `esp32/respuesta/facial` para que el ESP32-CAM pueda reaccionar en tiempo real (confirmación con flash LED verde o error con flash rojo).

Cifrado y descifrado de embeddings El módulo `encryption.py` proporciona dos funciones:

- `cifrar_embedding(embedding)`: Serializa el embedding como JSON, lo cifra con Fernet y codifica el resultado en Base64 URL-safe.
- `descifrar_embedding(cifrado_str)`: Decodifica el Base64, descifra con Fernet y deserializa el JSON para obtener el vector original.

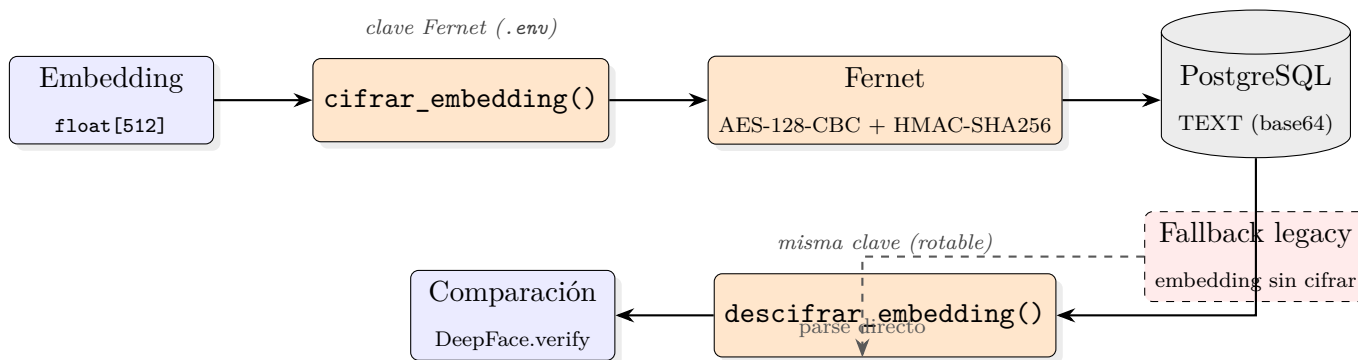


Figura 4.29: Pipeline de cifrado biométrico — Fernet (AES-128-CBC + HMAC-SHA256) con fallback para embeddings legacy almacenados sin cifrar.

Actualización y eliminación de datos faciales Se implementó el endpoint PUT `/api/facial/actualizar/<persona_id>` que permite reemplazar el embedding y la foto de una persona existente. También se implementó DELETE `/api/personas/<id>/datos-biometria` que elimina el embedding facial, la huella asociada, la foto en disco y el consentimiento, pero conserva los registros de asistencia históricos. La eliminación queda registrada en la tabla `eliminaciones_biometricas` para trazabilidad.

Se implementó además el endpoint POST `/api/facial/agregar-foto` que permite **agregar fotos de referencia adicionales** a una persona ya registrada, sin reemplazar las existentes. Cada nueva foto pasa por el mismo filtro de calidad Laplacian y se almacena como un nuevo registro en `encodings_faciales`. Esta funcionalidad de enrolamiento progresivo permite mejorar la precisión del sistema a lo largo del tiempo: si un trabajador experimenta falsos rechazos, se le pueden agregar más fotos de referencia con distintos ángulos o condiciones de iluminación, aumentando la robustez del reconocimiento sin necesidad de rehacer el registro completo.

Pruebas

Se diseñaron las siguientes pruebas para validar el módulo de reconocimiento facial:

- **Prueba de registro facial exitoso:** Enviar una imagen facial válida junto con un rut que tenga consentimiento registrado. Verificar que el backend extraiga el embedding, lo almacene cifrado y responda con HTTP 200. Verificar que la imagen se guarde en `static/previews/`.
- **Prueba de detección de duplicados:** Registrar una segunda imagen del mismo rostro para otra persona. Verificar que el backend detecte la similitud (distancia < 10.0) y rechace el registro con HTTP 409, indicando el nombre de la persona con la que coincide.

- **Prueba de rechazo por falta de consentimiento:** Intentar registrar un rostro para una persona sin consentimiento biométrico. Verificar que el backend rechace la operación con HTTP 403.
- **Prueba de identificación 1:N:** Registrar 3 personas con sus respectivas fotos. Enviar una nueva foto de una de ellas al endpoint de identificación. Verificar que el backend retorne el `rut` correcto y que la copia de auditoría se guarde en `static/capturas_prueba/`.
- **Prueba de identificación con rostro desconocido:** Enviar una foto de una persona no registrada. Verificar que el backend retorne HTTP 404 y registre el intento fallido en `logs_biometricos`.
- **Prueba de anti-spoofing con fotografía impresa:** Capturar una foto de un rostro registrado desde una pantalla o impresión y enviarla al endpoint de identificación. Verificar que DeepFace con `anti_spoofing=True` rechace la imagen por no detectar un rostro real (excepción `ValueError` con mensaje “Spoof detected”).
- **Prueba de resistencia a condiciones de iluminación:** Capturar imágenes del mismo rostro bajo 3 condiciones de iluminación (luz natural, luz artificial tenue, contraluz). Verificar que los 3 embeddings generados mantengan distancia euclidiana entre sí menor a 10.0 y que el sistema identifique correctamente a la persona.
- **Prueba de cifrado y descifrado:** Almacenar un embedding en la base de datos y verificar que el valor en la columna `encoding_facial` no sea legible (no contenga los números del vector en texto plano). Luego, recuperar el registro y descifrar el embedding; verificar que el vector original se reconstruya idénticamente.
- **Prueba de eliminación de datos biométricos:** Ejecutar el endpoint de eliminación de datos biométricos para una persona con rostro registrado. Verificar que el embedding se elimine de la base de datos, que la foto se borre del disco, que el consentimiento se elimine, que los registros de asistencia se conserven y que quede una entrada en `eliminaciones_biometricas`.
- **Prueba de filtro de calidad de imagen:** Enviar al endpoint de registro una imagen deliberadamente borrosa o una foto de una pantalla (baja varianza de Laplacian). Verificar que el backend rechace la imagen antes de invocar DeepFace, retornando un error de calidad insuficiente.

- **Prueba de múltiples fotos de referencia:** Registrar una persona con una foto frontal. Luego, agregar una segunda foto de la misma persona con distinto ángulo mediante `POST /api/facial/agregar-foto`. Realizar una identificación con una tercera foto y verificar que el sistema reconozca a la persona correctamente, comparando contra ambos embeddings de referencia.
- **Prueba de caché de embeddings:** Realizar una identificación exitosa y verificar los tiempos de respuesta. Realizar una segunda identificación inmediatamente después; verificar que el tiempo sea significativamente menor porque los embeddings se sirvieron desde la caché en memoria sin consultar la base de datos.

Endpoint identificar-o-registrar Se implementó el endpoint `POST /api/facial/identificar-o-registrar` que combina ambos flujos en una sola llamada: primero intenta identificar a la persona por su rostro; si la identificación es exitosa, retorna los datos de la persona; si falla y se proporciona un RUT con consentimiento biométrico, registra automáticamente la nueva persona con el embedding facial extraído. Este endpoint facilita el enrolamiento progresivo en el dispositivo: un trabajador puede colocarse frente al ESP32-CAM y, si ya está registrado, se marca su asistencia; si no, el sistema lo registra sobre la marcha. El endpoint incluye validación de calidad de imagen, anti-spoofing y log biométrico tanto para identificación como para registro.

4.2.5 Iteración 5: Autenticación, multi-tenant y enrolamiento de dispositivos

La quinta iteración incorpora la capa de seguridad y administración del sistema: autenticación de usuarios del panel web mediante JWT, un modelo multi-tenant que permite a múltiples empresas operar sobre la misma instancia del backend con datos aislados, y un mecanismo seguro de enrolamiento de dispositivos ESP32-CAM mediante PIN de un solo uso.

Análisis

El sistema, al estar orientado a un entorno empresarial, requiere que diferentes organizaciones puedan utilizar la misma plataforma sin que sus datos se mezclen. Asimismo, el panel web administrativo debe restringir el acceso según el rol del usuario. Finalmente, cada dispositivo ESP32-CAM debe ser vinculado de forma segura a una empresa específica, evitando que dispositivos no autorizados inyecten datos en el sistema.

Los requisitos identificados son:

- **Multi-tenant:** Cada entidad (persona, turno, asignación, asistencia, dispositivo) debe pertenecer a una empresa. Las consultas deben filtrarse por empresa según el contexto del usuario autenticado. El administrador global (rol `admin`) tiene visibilidad sobre todas las empresas.
- **Autenticación JWT:** El panel web backend debe requerir autenticación para operaciones sensibles. Los tokens JWT (JSON Web Tokens), firmados con HMAC-SHA256, transportan el ID de usuario, la empresa seleccionada y su rol. Tienen expiración de 24 horas.
- **Roles de usuario:** Se definen tres roles jerárquicos:
 - `admin`: Acceso total al sistema, puede crear empresas, gestionar todos los usuarios y dispositivos.
 - `empleador`: Acceso limitado a los datos de su empresa; puede gestionar trabajadores, turnos y ver asistencias.
 - `trabajador`: Acceso restringido a sus propios datos de asistencia.
- **Enrolamiento de dispositivos:** Un administrador genera un PIN de 8 caracteres en el panel web. Ese PIN se ingresa en el ESP32-CAM (vía web embebida), que lo envía al backend junto con su MAC address. Si el PIN es válido y no ha sido usado, el dispositivo queda enrolado y asociado a la empresa correspondiente.
- **Auto-registro de empresas:** Se requiere un endpoint público que permita a una nueva organización registrarse de forma autónoma en el sistema, sin intervención del administrador central. La empresa proporciona sus datos (nombre, RUT, email, etc.) y los datos del usuario administrador; el sistema crea la empresa, el usuario, y la asociación entre ambos con rol `admin` en una sola transacción atómica, retornando un token JWT para que el nuevo administrador quede autenticado inmediatamente. Esto facilita el despliegue del sistema en nuevos clientes sin requerir configuración manual previa.
- **Monitorización de dispositivos:** Una vez enrolado, el ESP32-CAM envía heartbeats periódicos vía MQTT. El backend actualiza el estado del dispositivo (`activo`) y registra su última actividad. Un proceso watchdog marca como `inactivo` a los dispositivos que no han enviado heartbeat en más de 90 segundos.

Diseño

Modelo de datos multi-tenant La estrategia multi-tenant se implementa a nivel de base de datos mediante una columna `empresa_id` en las tablas `personas`, `turnos`,

dispositivos, `integraciones_erp` y `asistencias` (esta última a través de la relación con `personas`). La tabla `empresas` actúa como entidad raíz del tenant. La tabla asociativa `usuario_empresa` vincula usuarios con empresas y define el rol que cada usuario tiene en cada empresa, permitiendo que un mismo usuario tenga roles distintos en diferentes organizaciones.

Flujo de autenticación

1. El usuario envía email y contraseña al endpoint `POST /api/auth/login`.
2. El backend valida las credenciales contra la tabla `usuarios_web` usando `bcrypt`.
3. Se consultan las empresas asociadas al usuario en `usuario_empresa`.
4. Si el usuario pertenece a una sola empresa, se selecciona automáticamente. Si pertenece a varias, se retorna la lista para que elija.
5. Se genera un token JWT con payload: `user_id`, `empresa_id`, `rol`, `persona_id` (si es trabajador) y `exp` (24 horas).
6. Las peticiones subsecuentes deben incluir el header `Authorization: Bearer <token>`.

Se implementan dos niveles de protección para los endpoints:

- `@token_required`: Exige token JWT válido.
- `@requiere_rol('admin', 'empleador')`: Además del token, exige que el rol del usuario esté en la lista permitida.
- `@token_opcional`: Intenta extraer el token si está presente; si no, permite el acceso sin autenticación (útil para endpoints consumidos por el ESP32-CAM, que en su lugar envían el header `X-Device-MAC` para identificar la empresa).

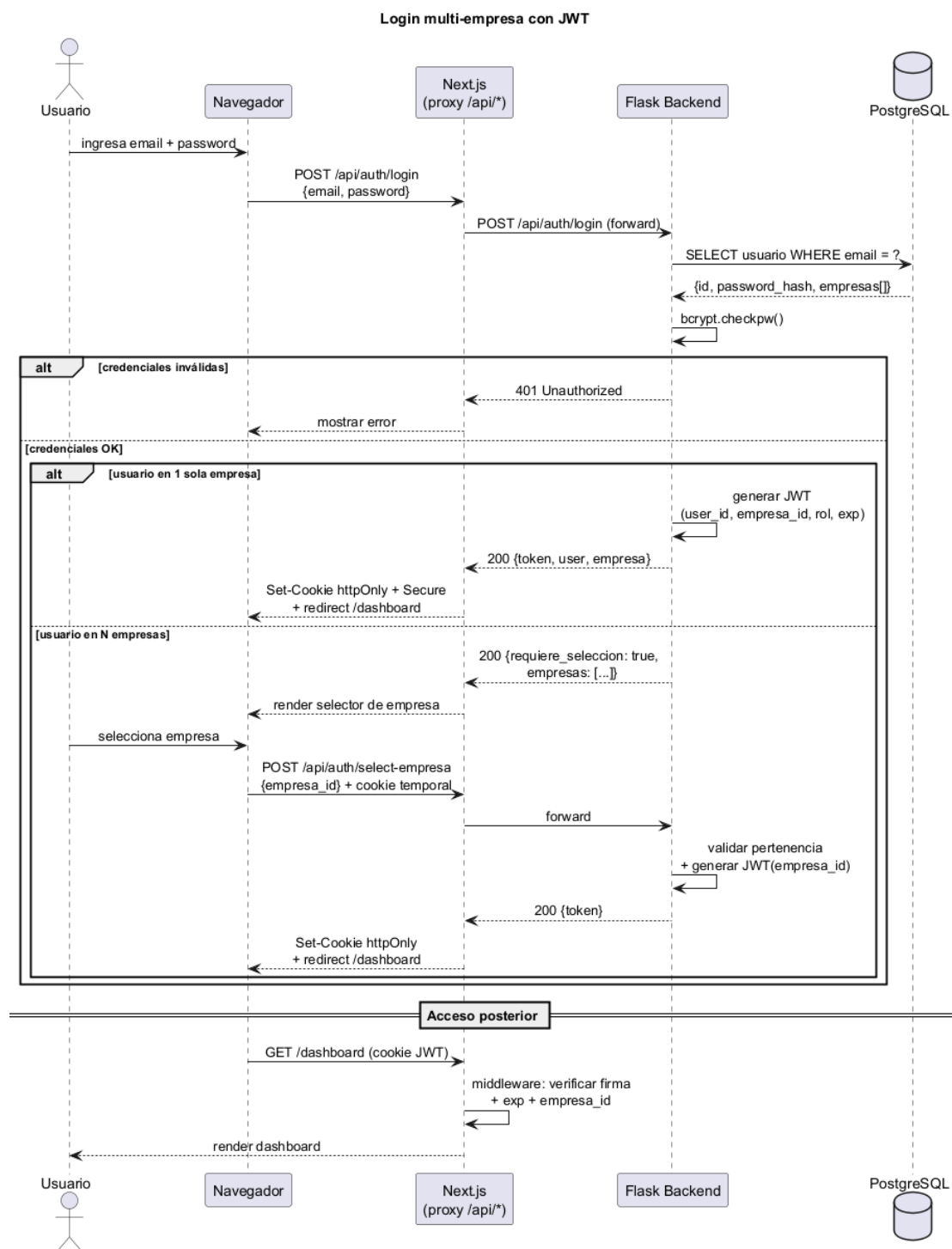


Figura 4.30: Diagrama de secuencia del login multi-empresa con JWT.

Flujo de enrolamiento de dispositivos

1. Un administrador o empleador accede a `POST /api/dispositivos/generar-pin` desde el panel web, especificando un nombre para el dispositivo. El backend genera un PIN alfanumérico de 8 caracteres (ej. `A3B9XK2M`) usando `secrets.choice()`

- y lo inserta en la tabla `dispositivos` con `enrolado = FALSE`.
2. El administrador ingresa el PIN en la interfaz web del ESP32-CAM (`/wifi-setup`).
 3. Al guardar la configuración, el ESP32-CAM envía `POST /api/dispositivos/enrolar` con el PIN, su MAC address y su IP local.
 4. El backend valida el PIN, asocia la MAC y la IP al dispositivo, marca `enrolado = TRUE` y limpia el PIN para que no pueda ser reutilizado.
 5. A partir de este momento, el ESP32-CAM envía heartbeats periódicos y el backend lo reconoce como dispositivo autorizado de la empresa.

Heartbeat y Last Will Testament (LWT) El ESP32-CAM publica periódicamente (cada 30 segundos) un mensaje JSON en el tópico `esp32/heartbeat/<MAC>` con su MAC, timestamp e IP. El backend actualiza el campo `ultimo_heartbeat` y marca el estado como `activo`. Al conectarse al broker MQTT, el ESP32-CAM configura un mensaje LWT en el tópico `esp32/lwt/<MAC>` con `{"estado": "inactivo"}`. Si el dispositivo se desconecta abruptamente, el broker publica automáticamente el LWT, y el backend marca el dispositivo como `inactivo`.

Además del LWT y el watchdog pasivo, se implementó un **ping activo** como tercera capa de detección: un hilo `device_pinger()` publica periódicamente (cada 30 segundos) un mensaje en el tópico `esp32/ping/<MAC>`. El ESP32-CAM, al recibirlo, responde inmediatamente publicando un heartbeat con el campo `"pong": true`. Si el backend no recibe respuesta dentro de 60 segundos, marca el dispositivo como inactivo. Este mecanismo permite detectar desconexiones incluso cuando el mensaje LWT no llega al broker (por ejemplo, por pérdida abrupta de conectividad de red). Adicionalmente, un hilo `device_watchdog` ejecutado cada 60 segundos verifica qué dispositivos no han enviado heartbeat en más de 90 segundos y los marca como inactivos, constituyendo una capa de respaldo adicional.

Implementación

Módulo de autenticación El módulo `routes/auth.py` implementa:

- `login()`: Valida credenciales con `bcrypt`, determina la empresa (con selección múltiple si aplica), vincula al trabajador con su `persona_id` si el rol es `trabajador`, y genera el token JWT.
- `register()`: Permite a administradores y empleadores crear nuevos usuarios, asignándoles rol y empresa. Si el email ya existe, se agrega la nueva asociación `usuario_empresa` en lugar de duplicar el usuario.

- `me()`: Retorna los datos del usuario autenticado, incluyendo sus empresas y rol actual.
- `cambiar_password()`: Permite al usuario cambiar su contraseña validando la actual.
- `generar_pin_enrolamiento()`: Crea un PIN para enrolar un nuevo dispositivo.
- `enrolar_dispositivo()`: Valida el PIN y completa el enrolamiento del ESP32-CAM.
- `listar_usuarios()`, `eliminar_usuario()`, `listar_empresas()`, `crear_empresa()`: Operaciones administrativas con control de roles.
- `register_company()`: Endpoint público `POST /api/auth/register-company` que permite el auto-registro de una nueva empresa. Recibe los datos de la organización (nombre, RUT, email, teléfono, dirección) y del usuario administrador (nombre, email, contraseña). En una única transacción atómica crea la empresa, el usuario con hash `bcrypt`, y la asociación `usuario_empresa` con rol `admin`. Retorna un token JWT para autenticar inmediatamente al nuevo administrador.

Los decoradores `@token_required`, `@token_opcional` y `@requiere_rol` se definen en el mismo módulo. El decorador `@token_opcional` incluye lógica para inferir la empresa a partir del header `X-Device-MAC` cuando no hay token JWT presente, buscando la MAC en la tabla de dispositivos. Si la MAC no existe, automáticamente crea un nuevo registro en `dispositivos` con estado `pendiente`, lo que permite que los ESP32-CAM comiencen a enviar datos incluso antes de ser formalmente enrolados.

Gestión de dispositivos El módulo `routes/dispositivos.py` implementa:

- `GET /api/dispositivos`: Lista los dispositivos con su estado, MAC, IP y empresa.
- `PUT /api/dispositivos/<id>`: Actualiza el nombre del dispositivo.
- `DELETE /api/dispositivos/<id>`: Elimina un dispositivo (admin) o lo desvincula de la empresa (empleador).
- `POST /api/dispositivos/verificar`: Prueba conectividad con un dispositivo dada su IP, consultando el endpoint `/estado` del ESP32-CAM.

Monitorización vía MQTT En el módulo `mqtt_handler.py`, la función `on_message` maneja los tópicos de heartbeat y LWT:

- `esp32/heartbeat/<MAC>`: Actualiza `ultimo_heartbeat`, `estado = 'activo'` y opcionalmente `ip_local` del dispositivo.
- `esp32/lwt/<MAC>`: Marca `estado = 'inactivo'` en la base de datos.

El watchdog (`device_watchdog`) ejecuta un barrido inicial al arrancar (todos los dispositivos se marcan inactivos hasta que envíen su primer heartbeat) y luego verifica cada 60 segundos los heartbeats vencidos.

Pruebas

Se diseñaron las siguientes pruebas para validar la autenticación, el multi-tenant y el enrolamiento:

- **Prueba de login exitoso:** Enviar credenciales válidas del administrador por defecto (`admin@empresa.cl / admin123`). Verificar que se reciba un token JWT, los datos del usuario y su empresa. Decodificar el token y verificar que contenga `user_id`, `empresa_id` y `rol = admin`.
- **Prueba de login con múltiples empresas:** Crear una segunda empresa y asignar el mismo usuario a ella con rol `empleador`. Iniciar sesión sin especificar empresa. Verificar que el backend retorne la lista de empresas disponibles. Luego, iniciar sesión especificando una empresa y verificar que el token refleje esa selección.
- **Prueba de rechazo de credenciales inválidas:** Intentar login con contraseña incorrecta. Verificar HTTP 401 y mensaje “Credenciales inválidas”.
- **Prueba de acceso sin token:** Intentar acceder a `GET /api/personas` sin enviar header `Authorization`, desde un cliente sin `X-Device-MAC`. Verificar que el endpoint responda correctamente (acceso público o con datos limitados según el decorador).
- **Prueba de acceso con rol insuficiente:** Autenticarse como trabajador e intentar crear un turno (`POST /api/turnos`). Verificar HTTP 403 y mensaje “Permisos insuficientes”.
- **Prueba de aislamiento multi-tenant:** Crear dos empresas (A y B), registrar una persona en cada una. Autenticarse como empleador de la empresa A y solicitar `GET /api/personas`. Verificar que solo se retornen las personas de la empresa A.

- **Prueba de generación de PIN:** Autenticarse como empleador y solicitar `POST /api/dispositivos/generar-pin`. Verificar que se reciba un PIN de 8 caracteres alfanuméricos y que en la base de datos exista un dispositivo con `enrolado = FALSE`.
- **Prueba de enrolamiento exitoso:** Con un PIN generado, enviar `POST /api/dispositivos/enrolar` con el PIN, una MAC y una IP. Verificar que el backend marque el dispositivo como enrolado, asocie la MAC y limpie el PIN. Verificar que el mismo PIN no pueda usarse una segunda vez (HTTP 409).
- **Prueba de heartbeat y estado:** Simular heartbeats desde un dispositivo enrolado. Verificar que el estado en la base de datos sea `activo`. Detener los heartbeats por más de 90 segundos y verificar que el watchdog marque el dispositivo como `inactivo`.
- **Prueba de LWT:** Conectar el ESP32-CAM al broker MQTT, verificar que el backend reciba el heartbeat y lo marque activo. Desconectar el dispositivo abruptamente (corte de energía). Verificar que el broker publique el LWT y el backend actualice el estado a `inactivo` en menos de 10 segundos.
- **Prueba de auto-registro de empresa:** Enviar una solicitud a `POST /api/auth/register-company` con los datos de una nueva empresa y su administrador. Verificar que se cree la empresa en la base de datos, que el usuario administrador se cree con contraseña encriptada (bcrypt), que exista la asociación `usuario_empresa` con rol `admin`, y que el backend retorne un token JWT válido que permita acceder a endpoints protegidos. Verificar que un segundo intento con el mismo email de administrador retorne HTTP 409 (Conflicto).

4.2.6 Iteración 6: Módulo antifraude y validación previa a la captura

Esta iteración implementa la prevención de suplantación de identidad en el ESP32-CAM mediante detección física de presencia, control de iluminación y procesamiento biométrico. Si bien el plan original contemplaba un sensor infrarrojo (IR) dedicado para detectar intentos de suplantación con fotografías, el enfoque implementado evolucionó hacia una solución de múltiples capas que resultó más robusta y prescindió del sensor IR adicional.

Análisis

El análisis de escenarios de suplantación identificó los siguientes vectores de ataque potenciales contra el sistema de reconocimiento facial del ESP32-CAM:

- **Fotografía impresa:** Un atacante presenta una foto del rostro de un trabajador legítimo frente a la cámara.
- **Pantalla de dispositivo:** Un atacante muestra un video o imagen del trabajador en la pantalla de un teléfono o tablet.
- **Ausencia de presencia real:** La cámara captura sin que haya una persona físicamente frente al dispositivo (falsos positivos del sensor de movimiento).

Inicialmente se planificó el uso de un sensor IR dedicado que permitiera detectar la presencia de un rostro real mediante la emisión y recepción de luz infrarroja, cuya reflectancia difiere entre piel humana y superficies planas como papel o pantallas. Sin embargo, durante el desarrollo se identificaron dos factores que motivaron el cambio de estrategia:

1. **Disponibilidad de GPIO:** El ESP32-CAM tiene una cantidad muy limitada de pines GPIO disponibles tras la asignación del bus paralelo de la cámara. Destinar un pin adicional al sensor IR hubiera requerido sacrificar otra funcionalidad (como el sensor PIR de presencia o el control PWM del flash).
2. **Madurez del anti-spoofing por software:** DeepFace, la librería seleccionada para el reconocimiento facial, incorpora un detector de suplantación (*anti-spoofing*) basado en redes neuronales convolucionales que analiza micro-texturas, reflectancia de la piel y profundidad en la imagen capturada. Este detector es capaz de distinguir entre un rostro real tridimensional y una superficie plana (foto, pantalla) con una precisión comparable o superior a la de sensores IR de bajo costo como los inicialmente considerados.

En consecuencia, se adoptó un enfoque de múltiples capas que combina:

1. **Detección física de presencia (PIR):** Un sensor infrarrojo pasivo (PIR) que detecta el calor corporal, activando el sistema de captura solo cuando hay una persona real frente al dispositivo. Esto elimina falsos positivos del software y reduce el consumo energético del sensor de huella y la cámara.
2. **Iluminación controlada (flash PWM):** Un destello breve y de intensidad moderada (50% de duty cycle a 5 kHz) que ilumina el rostro durante la captura

sin encenegecer a la persona. La respuesta del rostro real a esta fuente de luz puntual difiere de la de una superficie plana, aportando una capa adicional de discriminación.

3. **Anti-spoofing por software (DeepFace):** La capa más potente, que analiza la imagen capturada con un modelo entrenado para detectar intentos de suplantación. Las imágenes que no superan esta validación son rechazadas con una excepción `ValueError` que el backend traduce en un mensaje de error al ESP32-CAM.

Diseño

Sensor PIR como portero del sistema El PIR conectado al GPIO12 actúa como “portero”: el sistema permanece en reposo hasta que el sensor detecta movimiento (cambio en la radiación infrarroja del entorno). Una vez activado, se inicia un período de “alerta” de 15 segundos durante el cual:

- Se habilita el escaneo de huella cada 2 segundos mediante el AS608.
- Si hay conectividad, se habilita la identificación facial cada 6 segundos.
- Si el PIR no vuelve a detectar movimiento en 15 segundos, se asume que fue una falsa alarma o que la persona se retiró, y el sistema vuelve a reposo.

Esta arquitectura reduce drásticamente el consumo energético y el desgaste del sensor de huellas, ya que el AS608 solo se activa cuando hay una alta probabilidad de que haya una persona frente al dispositivo.

Control de acceso con cooldown y bloqueo Para evitar marcaciones múltiples accidentales y garantizar que cada interacción sea deliberada, se implementan tres mecanismos de control:

1. **Cooldown entre marcaciones** (`COOLDOWN_TIEMPO = 8000 ms`): Tras una marcación exitosa, el sistema ignora nuevos intentos durante 8 segundos.
2. **Debounce de huella** (`FINGER_DEBOUNCE = 4000 ms`): Una misma huella no puede gatillar dos marcaciones en un intervalo menor a 4 segundos, evitando lecturas múltiples del mismo dedo.
3. **Bloqueo por menú** (`BLOQUEO_MENU_MS = 30000 ms`): Cuando se utiliza la interfaz web para operaciones administrativas (registro, edición, configuración), se activa un bloqueo de 30 segundos que impide la marcación automática, evitando que la persona que está operando el menú sea registrada inadvertidamente.

Firma de movimiento Para detectar si la escena frente a la cámara cambió entre dos capturas consecutivas (lo que indicaría una persona real moviéndose versus una foto estática), se implementa una **firma de movimiento** que calcula un hash reducido del buffer JPEG:

- Se muestrean 128 bytes del frame capturado, distribuidos uniformemente a lo largo del buffer.
- Se calcula la suma acumulada de esos bytes, produciendo una firma de 32 bits.
- Si la diferencia entre la firma actual y la anterior supera un umbral (`UMBRAL_MOVIMIENTO = 1800`), se considera que hubo movimiento real en la escena.

Implementación

Integración del PIR El sensor PIR se conecta al GPIO12 del ESP32-CAM, configurado con resistencia de pull-down interna para evitar lecturas flotantes. En el `setup()` se incluye una fase de calibración de 3 segundos (`delay(3000)`) para estabilizar el sensor antes de comenzar el bucle principal.

En el `loop()`, la lógica de detección sigue el siguiente flujo:

- Si el PIR detecta `HIGH` y el flag `hayAlguienFrenteAlSensor` es falso, se activa el modo alerta y se esperan 800 ms para estabilizar la fuente de alimentación (el capacitor de 1000 μ F del ESP32-CAM necesita recuperarse antes de encender la cámara).
- Si el PIR detecta `HIGH` estando ya en modo alerta, se renueva el timer de actividad.
- Si pasan más de 15 segundos sin detección, se desactiva el modo alerta y se retorna a reposo.

Flash PWM para iluminación facial El flash LED integrado del ESP32-CAM se controla mediante PWM en el GPIO4, configurado como salida LED PWM con frecuencia de 5 kHz y resolución de 8 bits. Durante la captura de una imagen (tanto para registro como para identificación), el flash se activa con un duty cycle del 50% (`FLASH_PWM_DUTY_50 = 128`) durante 150-200 milisegundos, tiempo suficiente para que el sensor OV2640 ajuste su exposición.

La secuencia de captura es la siguiente:

1. Encender flash al 50% (`ledcWrite(FLASH_PIN, 128)`).
2. Esperar 150 ms para estabilización del sensor.

3. Capturar el frame (`esp_camera_fb_get()`).
4. Apagar el flash inmediatamente (`ledcWrite(FLASH_PIN, 0)`).
5. Procesar o transmitir el frame.

Este enfoque minimiza el tiempo total de encendido del flash y, por tanto, el consumo eléctrico y la molestia para el usuario.

Señalización mediante flash Además de la iluminación para captura, el flash se utiliza para señalar el resultado de las operaciones:

- **Éxito** (`flashExito()`): Dos destellos breves (150 ms encendido, 150 ms apagado).
- **Error** (`flashError()`): Un destello largo (800 ms encendido).

Esta señalización permite al usuario recibir retroalimentación inmediata sin necesidad de mirar una pantalla.

Validación anti-spoofing en el backend La validación anti-spoofing se ejecuta en el backend, no en el ESP32-CAM, debido a las limitaciones de procesamiento del microcontrolador. La función `extraer_embedding()` en `routes/facial.py` recibe el parámetro `anti_spoofing=True` para los endpoints de identificación y verificación, lo que activa el detector de suplantación de DeepFace. Si la imagen no supera la validación (por ser una foto, pantalla o carecer de características faciales reales), DeepFace lanza una excepción `ValueError` que el backend captura y traduce en una respuesta HTTP 400 con el mensaje “Spoof detected”.

Pruebas

Se diseñaron las siguientes pruebas para validar el módulo antifraude:

- **Prueba de detección PIR:** Situar a una persona a 50 cm del dispositivo. Verificar que el PIR detecte el movimiento en menos de 1 segundo y que el sistema transicione a modo activo. Retirar a la persona y verificar que tras 15 segundos sin detección, el sistema retorne a reposo.
- **Prueba de falsos positivos del PIR:** Dejar el dispositivo en una habitación vacía durante 10 minutos y monitorear los logs. Verificar que no se active el modo de captura sin presencia humana real (tolerancia máxima: 1 falso positivo por hora).

- **Prueba de cooldown:** Realizar una marcación exitosa por huella. Intentar inmediatamente una segunda marcación. Verificar que el sistema rechace el intento por estar en período de cooldown (8 segundos).
- **Prueba de debounce de huella:** Mantener el dedo presionado sobre el sensor AS608 durante 5 segundos. Verificar que solo se registre una marcación, no múltiples.
- **Prueba de bloqueo por menú:** Acceder a la vista `/register` desde el navegador. Mientras la página está abierta, intentar provocar una marcación automática con huella. Verificar que el sistema esté bloqueado durante 30 segundos.
- **Prueba de anti-spoofing con fotografía:** Imprimir una foto de un rostro registrado y presentarla frente al ESP32-CAM con el PIR activado (para ello, mover la foto simulando una persona). Verificar que DeepFace rechace la imagen con `anti_spoofing=True` y que el backend retorne HTTP 400.
- **Prueba de anti-spoofing con pantalla:** Reproducir un video del rostro de una persona registrada en un teléfono móvil y presentarlo frente al ESP32-CAM. Verificar que el sistema rechace la identificación.
- **Prueba de flash PWM:** Durante una captura, medir con un osciloscopio o cámara de alta velocidad que el flash se encienda al 50% de duty cycle durante no más de 200 ms. Verificar que la iluminación sea suficiente para obtener una imagen nítida del rostro.
- **Prueba de señalización:** Provocar una marcación exitosa y verificar que el flash emita dos destellos breves. Provocar un error (huella no reconocida) y verificar que el flash emita un destello largo.

4.2.7 Iteración 7: Panel web backend e integración con ERP

La séptima iteración incorpora el panel de administración web del backend y el módulo de integración con sistemas ERP externos, permitiendo que los datos de asistencia capturados por el dispositivo IoT sean automáticamente transmitidos a plataformas empresariales de recursos humanos o nómina.

Análisis

El sistema, hasta esta iteración, permite registrar asistencias y almacenarlas en la base de datos, pero carece de una interfaz administrativa centralizada y de un mecanismo

para integrar los datos con sistemas ERP existentes. Se identificaron los siguientes requisitos:

- **Panel web administrativo:** Una interfaz web que permita a administradores y empleadores gestionar personas, turnos, asignaciones, visualizar asistencias, administrar dispositivos enrolados y configurar integraciones ERP. Debe ser accesible vía navegador sin dependencia del ESP32-CAM.
- **API REST completa y documentada:** Todos los blueprints implementados en iteraciones anteriores deben consolidarse como una API REST coherente, con métodos HTTP estándar, respuestas JSON estructuradas y manejo de autenticación por JWT.
- **Integración con ERP vía webhooks:** Permitir configurar múltiples destinos ERP, cada uno con su propia URL de webhook, headers personalizados (ej. tokens de autenticación) y mapeo de campos (field mapping) para adaptar el formato de los datos de asistencia al esquema esperado por cada ERP.
- **Envío automático:** Las asistencias registradas deben enviarse automáticamente a los ERP configurados, en un hilo separado para no bloquear la respuesta HTTP al dispositivo.
- **Test de conectividad:** Cada integración ERP debe poder probarse manualmente desde el panel web antes de activar el envío automático.

Diseño

Arquitectura del backend El backend se organiza en 9 blueprints de Flask, cada uno con responsabilidades bien definidas:

1. **auth_bp:** Autenticación JWT, registro de usuarios, gestión de empresas, enrolamiento de dispositivos.
2. **personas_bp:** CRUD de personas con consentimiento biométrico y eliminación de datos sensibles.
3. **turnos_bp:** CRUD de turnos con filtro por empresa.
4. **asignaciones_bp:** CRUD de asignaciones persona-turno.
5. **asistencias_bp:** Registro de asistencias individuales y sincronización por lotes.
6. **facial_bp:** Registro, verificación e identificación facial con anti-spoofing y cifrado.

7. `dispositivos_bp`: Gestión de dispositivos enrolados y verificación de conectividad.
8. `logs_bp`: Consulta de logs de sincronización y logs biométricos.
9. `erp_bp`: CRUD de integraciones ERP, envío de datos y test de conectividad.

Diseño del módulo ERP La integración con ERP se basa en el patrón **webhook saliente**: cuando se registra una asistencia, el backend notifica a cada ERP configurado mediante una solicitud HTTP POST al webhook correspondiente, con los datos de la marcación en el cuerpo JSON.

Cada integración ERP se configura con los siguientes parámetros:

- **nombre**: Identificador descriptivo (ej. “Softland RRHH”).
- **tipo**: Clasificación del ERP (`generic`, `softland`, `defontana`, `sap`).
- **webhook_url**: Endpoint HTTP donde el ERP espera recibir los datos.
- **headers**: Objeto JSON con headers HTTP personalizados (ej. `{"Authorization": "Bearer <token>"}`).
- **field_map**: Objeto JSON que mapea los campos estándar del sistema (`rut`, `nombre`, `tipo`, `metodo`, `fecha_hora`) a los nombres de campo esperados por el ERP.
- **envio_auto**: Booleano que indica si el envío se realiza automáticamente al registrar cada asistencia, o solo manualmente.
- **activo**: Booleano para habilitar o deshabilitar temporalmente la integración sin eliminarla.

El field mapping permite adaptar el formato de salida sin modificar el código del backend. Por ejemplo, un ERP que espera los campos `employee_id`, `timestamp` y `event_type` puede configurarse con:

```
{"rut": "employee_id", "fecha_hora": "timestamp", "tipo": "event_type"}
```

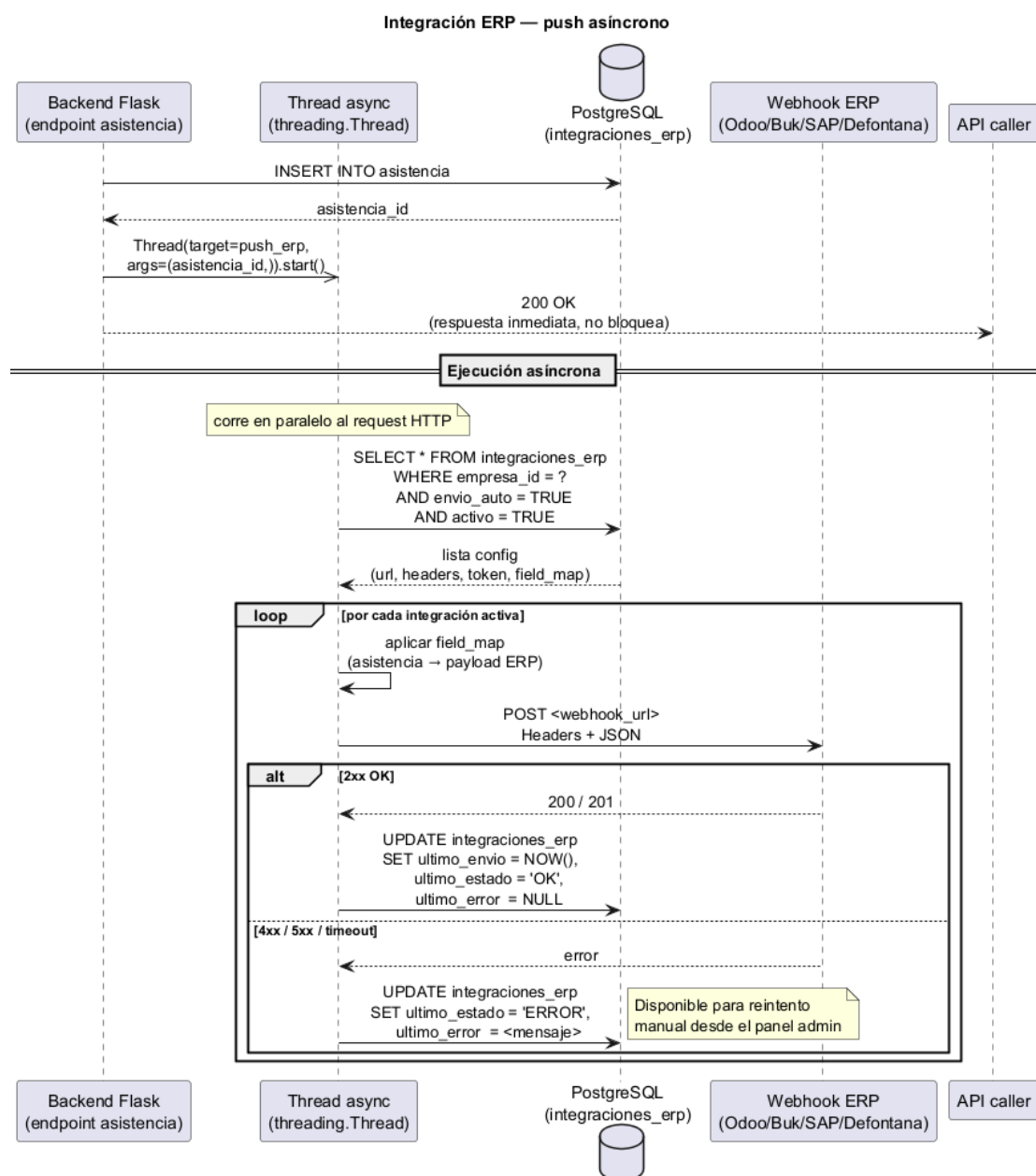


Figura 4.31: Diagrama de secuencia de la integración ERP — push asíncrono con field mapping.

Implementación

Panel web del backend Para complementar el backend, se desarrolló un panel web independiente (Next.js 16 con React 19 y TypeScript) cuyo módulo principal es la gestión del dispositivo IoT. Este panel, accesible desde cualquier navegador moderno, permite al administrador:

- Visualizar el estado online/offline de cada dispositivo enrolado en tiempo real

mediante tarjetas con código de colores, indicador animado (live-dot) y la IP como enlace clickable con la leyenda “Misma red requerida”.

- Generar el PIN de 8 caracteres necesario para enrolar un nuevo ESP32-CAM.
- Probar la conectividad activa con un dispositivo específico consultando su endpoint `/estado` (variable `NEXT_PUBLIC_DEVICE_BASE_URL`, por defecto `http://192.168.4.1`).
- Renombrar y eliminar dispositivos registrados.
- Acceder a los logs de sincronización generados por el dispositivo.
- Acceder a las funcionalidades complementarias de gestión: personas, turnos, asignaciones, asistencias, integración ERP, usuarios y empresas.

El estado en tiempo real se obtiene mediante dos mecanismos complementarios. El principal es un hook `useDeviceWebSocket` que utiliza `EventSource` para conectarse directamente al endpoint SSE del backend (`http://localhost:5000/sse/devices`), recibiendo eventos con el id, nombre, IP, estado y timestamp del último heartbeat de cada dispositivo. Este streaming sustituye la antigua dependencia de `flask-socketio` por una conexión nativa del navegador, sin necesidad de librerías adicionales. Como respaldo, un temporizador ejecuta una consulta REST cada 15 segundos (`GET /api/dispositivos`) para actualizar el estado si la conexión SSE se interrumpe. El estado `online` se calcula verificando que el campo `estado` sea `'activo'` y que el `ultimo_heartbeat` esté dentro de los últimos 5 minutos; si alguna condición falla, el dispositivo se considera `offline`.

El panel web se comunica con el backend Flask mediante un proxy interno (`app/api/_proxy.ts`) que reenvía las peticiones REST a la API, evitando problemas de CORS. La conexión SSE, por su parte, se realiza directamente al backend en el puerto 5000 sin pasar por el proxy, ya que requiere una conexión persistente `text/event-stream`. La aplicación está protegida por un middleware de autenticación que valida el token JWT y restringe el acceso según el rol del usuario (admin, empleador, trabajador). El frontend se compila con Docker en un build multi-etapa usando `output: 'standalone'` para producir una imagen optimizada lista para producción.

Para mantener la consistencia visual entre el panel web y la interfaz embebida del ESP32-CAM, las 9 vistas HTML del dispositivo (`index.html`, `register.html`, `gestion.html`, `personas.html`, `turnos.html`, `asignaciones.html`, `asistencias.html`, `logs.html` y `wifi-setup.html`) fueron actualizadas para adoptar la misma paleta oscura, tipografía y sistema de componentes del panel SAS. Se utilizaron variables CSS para la paleta cromática, diseño basado en tarjetas (cards) para la disposición del contenido, y componentes reutilizables para botones, tablas y formularios, logrando que ambas interfaces presenten una experiencia de usuario unificada.

Adicionalmente, el panel web incorpora las siguientes funcionalidades complementarias:

- **Auto-registro de empresas:** La pantalla de inicio de sesión incluye un modo de registro que permite a nuevas empresas crear su cuenta directamente, invocando al endpoint público `POST /api/auth/register-company`. El formulario solicita nombre de la empresa y datos del administrador (nombre, email, contraseña), y al enviarlo crea la organización, el usuario administrador y retorna un token JWT para acceso inmediato.
- **Captura facial por webcam:** El formulario de registro facial en el panel web utiliza la API `navigator.mediaDevices.getUserMedia()` para capturar fotografías directamente desde la cámara del navegador, eliminando la necesidad de subir archivos desde el sistema de archivos local. La interfaz permite visualizar la vista previa en vivo, capturar, previsualizar y confirmar o repetir la toma, con liberación adecuada del stream al cancelar.
- **Edición de usuarios del sistema:** Los administradores y empleadores pueden editar los datos de los usuarios del sistema (nombre, email, rol, contraseña, estado activo/inactivo) mediante un formulario modal que reutiliza la interfaz de registro en modo edición. Las actualizaciones son parciales: solo se envían los campos modificados.

Endpoint de configuración ERP para el ESP32-CAM Para que el ESP32-CAM pueda notificar asistencias directamente a los ERP configurados (incluso si el backend central no está disponible), se implementó el endpoint `GET /api/dispositivos/erp-config`, que retorna la lista de integraciones ERP activas con envío automático habilitado. El ESP32-CAM consulta este endpoint cada hora y almacena la configuración en `/erp-config.json`, permitiéndole enviar marcaciones a los ERP incluso durante desconexiones parciales.

Contraseñas para autenticación de dispositivos Para reforzar la seguridad en la comunicación entre el backend y los ESP32-CAM enrolados, se implementó un sistema de contraseñas de dispositivo que permite al administrador generar, gestionar y verificar credenciales específicas para cada terminal IoT. El flujo contempla cuatro operaciones:

- **Generar contraseña** (`POST /api/dispositivos/<id>/generar-password`): El administrador solicita una nueva contraseña de 12 caracteres alfanuméricos, generada mediante `secrets.choice()`. El backend almacena el hash SHA256 de la contraseña, la versión en texto plano (temporal) y marca la contraseña como pendiente (`password_pendiente = TRUE`). La respuesta incluye la contraseña en texto plano para que el administrador la proporcione al dispositivo.

- **Verificar contraseña pendiente** (GET /api/dispositivos/check-password): El ESP32-CAM, al iniciar, consulta este endpoint enviando su MAC address como header X-Device-MAC. Si hay una contraseña pendiente, el backend la retorna en texto plano para que el dispositivo la aplique localmente.
- **Confirmar contraseña** (POST /api/dispositivos/confirmar-password): Una vez que el dispositivo ha aplicado la contraseña localmente, confirma al backend, que limpia el texto plano (`password_plain = NULL`) y desmarca el estado pendiente. A partir de este momento, solo el hash SHA256 permanece almacenado.
- **Eliminar contraseña** (DELETE /api/dispositivos/<id>/password): El administrador puede eliminar la contraseña de un dispositivo, limpiando el hash y el texto plano.

En el panel web, la sección de gestión de dispositivos incorpora botones para generar, regenerar y eliminar contraseñas, así como indicadores visuales del estado de la contraseña (tiene o no tiene, pendiente o confirmada).

Módulo de integración ERP El módulo `routes/erp.py` implementa:

- **enviar_asistencia_a_erps()**: Función central que itera sobre las integraciones ERP activas de una empresa, transforma los datos de la asistencia según el field mapping configurado, y envía una solicitud POST al webhook con los headers personalizados. Registra el resultado (ok o error) en la tabla `integraciones_erp` (columnas `ultimo_envio` y `ultimo_estado`).
- **_disparar_erp_push()**: Wrapper que ejecuta `enviar_asistencia_a_erps()` en un hilo daemon separado, evitando que la latencia de los ERP externos afecte el tiempo de respuesta al ESP32-CAM. Esta función se invoca automáticamente desde `create_asistencia()` y `sync_asistencias()` cada vez que se registra una asistencia.
- **CRUD de integraciones**: Endpoints REST para crear, listar y eliminar configuraciones ERP.
- **Test de conectividad** (POST /api/erp/<id>/test): Envía un payload de prueba con datos ficticios al webhook configurado y retorna el código de respuesta y el cuerpo recibido.
- **Envío manual** (POST /api/erp/<id>/enviar): Envía las últimas 200 asistencias de la empresa al ERP seleccionado, útil para cargas históricas o reprocesamiento.
- **Estado de integración** (GET /api/erp/<id>/estado): Retorna el último envío y su estado.

Control remoto de dispositivos ESP32 Se implementaron tres endpoints en `routes/dispositivos.py` que permiten controlar remotamente los ESP32-CAM enrolados mediante comandos MQTT:

- **Registrar huella** (POST `/api/dispositivos/<id>/registrar-huella`): Recibe un `persona_id` y envía al dispositivo el comando para iniciar el enrolamiento de una huella digital. Verifica que el dispositivo exista, que pertenezca a la empresa del usuario autenticado, y que la persona exista en la misma empresa. Utiliza `enviar_comando_dispositivo(mac, 'registrar_huella')` para publicar el comando en `backend/comando/<MAC>/registrar_huella`. El resultado del enrolamiento se recibe luego por el tópico `esp32/huella/resultado/#`.
- **Reiniciar dispositivo** (POST `/api/dispositivos/<id>/reiniciar`): Envía el comando `reiniciar` al ESP32-CAM, provocando un reinicio remoto del dispositivo. Útil para recuperar un dispositivo que presenta fallos de conectividad o para aplicar cambios de configuración. Si el cliente MQTT no está disponible, retorna HTTP 503.
- **Reconectar Wi-Fi** (POST `/api/dispositivos/<id>/wifi-reconnect`): Envía el comando `wifi-reconnect` para que el dispositivo reinicie su conexión Wi-Fi. Permite recuperar un dispositivo que ha perdido la conectividad sin necesidad de intervención física.

Los tres endpoints están protegidos con `@requiere_rol('admin', 'empleador')` y verifican que el dispositivo pertenezca a la empresa del usuario autenticado. Los comandos se publican en el tópico `backend/comando/<MAC>/<comando>` mediante la función `enviar_comando_dispositivo()` del módulo `mqtt_handler.py`.

Pruebas

Se diseñaron las siguientes pruebas para validar el panel web para la gestión del dispositivo y la integración ERP:

- **Prueba de streaming SSE:** Conectar el navegador a `http://localhost:5000/sse/devices` y verificar que se reciban eventos `text/event-stream` con datos JSON de los dispositivos. Simular un heartbeat desde un ESP32-CAM y verificar que el evento SSE refleje el cambio a estado activo. Desconectar el dispositivo y verificar que el evento SSE refleje el estado inactivo.
- **Prueba de fallback polling:** Interrumpir la conexión SSE (deteniendo el servidor backend momentáneamente). Verificar que el panel web continúe mostrando el

estado de los dispositivos mediante las consultas REST periódicas (cada 15 segundos) y que al restaurar la conexión SSE se retome el streaming.

- **Prueba de indicador visual de dispositivo:** Verificar que cada tarjeta de dispositivo en el panel web muestre: la dirección IP como enlace clickable que abre `http://<ip>/` en una nueva pestaña, un indicador animado verde/rojo (live-dot) que refleje el estado online/offline, y la leyenda “Misma red requerida” como sugerencia contextual.
- **Prueba de navegación en panel web:** Acceder a las rutas principales del backend (`/health`, `/api/personas`, `/api/asistencias`) sin autenticación y verificar que respondan correctamente (HTTP 200 con acceso público o HTTP 401 cuando se requiera token).
- **Prueba de creación de integración ERP:** Autenticarse como empleador y enviar `POST /api/erp` con nombre, tipo, webhook URL (usando un servidor de prueba como `https://webhook.site`) y field mapping. Verificar que la integración se cree con `activo = TRUE` y `envio_auto = TRUE`.
- **Prueba de test de webhook:** Ejecutar `POST /api/erp/<id>/test` para la integración creada. Verificar que el webhook de prueba reciba un payload JSON con datos ficticios (`rut = '11.111.111-1'`, `nombre = "Test Usuario"`, `tipo = "entrada"`). Verificar que el estado se actualice en la base de datos.
- **Prueba de envío automático:** Registrar una asistencia real (`POST` a `/api/asistencias`) para una persona de la empresa configurada. Verificar que el servidor de prueba del webhook reciba automáticamente el payload con los datos de la asistencia.
- **Prueba de field mapping:** Configurar un ERP con field mapping `{"rut": "employee_id", "tipo": "event"}`. Registrar una asistencia y verificar que el webhook reciba el payload con los campos renombrados (`employee_id` en lugar de `rut`, `event` en lugar de `tipo`).
- **Prueba de envío asíncrono:** Medir el tiempo de respuesta del endpoint `POST /api/asistencias` cuando hay un ERP configurado cuyo webhook tarda 5 segundos en responder. Verificar que la respuesta al cliente sea inmediata (menor a 200 ms) y que el envío al ERP ocurra en segundo plano.
- **Prueba de tolerancia a fallos ERP:** Configurar un webhook que retorne HTTP 500. Registrar una asistencia y verificar que el sistema no falle; el error debe quedar registrado en `ultimo_estado` de la integración y la asistencia debe persistirse correctamente en la base de datos.

- **Prueba de sincronización de configuración ERP al ESP32-CAM:** Simular la consulta `GET /api/dispositivos/erp-config` desde el ESP32-CAM (con header `X-Device-MAC`). Verificar que retorne solo las integraciones activas con envío automático de la empresa correspondiente.
- **Prueba de envío manual por lote:** Ejecutar `POST /api/erp/<id>/enviar` y verificar que se envíen correctamente los registros de asistencia recientes al webhook del ERP.

4.2.8 Iteración 8: Sincronización diferida, logs y cierre del proyecto

Esta iteración integra la sincronización diferida entre el ESP32-CAM y el backend, la capa de auditoría y logs del sistema, y las herramientas de diagnóstico.

Análisis

Para que el prototipo sea viable en entornos reales, debe garantizar que ningún dato se pierda durante períodos de desconexión y que todas las operaciones críticas queden registradas para auditoría. Los requisitos identificados son:

- **Sincronización diferida de asistencias:** Los registros de asistencia generados offline (marcados con `sincronizado = false`) deben enviarse automáticamente al backend cuando se restablezca la conectividad. El proceso debe ser idempotente: si un registro ya existe en el backend, no debe duplicarse.
- **Sincronización diferida de entidades:** Las personas, turnos y asignaciones creados localmente durante el modo offline deben sincronizarse con el backend, resolviendo conflictos de identificadores (IDs locales con prefijo `local-` versus IDs del backend).
- **Sincronización de configuración ERP:** La configuración de integraciones ERP debe poder descargarse desde el backend para que el ESP32-CAM pueda enviar datos de asistencia directamente a los ERP configurados.
- **Logs de sincronización:** Cada operación de sincronización debe registrarse en la base de datos con el conteo de registros enviados y procesados correctamente, permitiendo auditoría posterior.
- **Logs biométricos:** Todas las operaciones de registro, verificación e identificación facial deben quedar registradas con timestamp, resultado e IP de origen, para cumplimiento normativo.

- **Watchdog de dispositivos:** El backend debe monitorear continuamente el estado de los dispositivos enrolados, detectando aquellos que han dejado de comunicarse y marcándolos como inactivos.
- **Herramientas de diagnóstico:** Se requiere un script de prueba que permita simular asistencias faciales usando fotos almacenadas, facilitando las pruebas del sistema sin necesidad del hardware.

Diseño

Proceso de sincronización diferida El ESP32-CAM ejecuta la sincronización en dos momentos:

1. **Al iniciar:** Si hay conectividad, llama a `sincronizarPendientes()` que ejecuta en secuencia:
 - `sincronizarTurnosPendientes()`: Envía los turnos con `sincronizado = false` al backend vía POST, y actualiza sus `backend_id`.
 - `sincronizarAsignacionesPendientes()`: Similar para asignaciones, dependiendo de que los turnos referenciados ya tengan `backend_id`.
 - `sincronizarAsistencias()`: Envía un lote de asistencias con `sincronizado = false` vía POST `/api/asistencias/sync`.
2. **Periódicamente:** Cada 5 minutos (300,000 ms), se ejecuta `sincronizarPendientes()` en el bucle principal.

Además, el ESP32-CAM consulta `GET /api/dispositivos/erp-config` cada hora (360,000 ms) para mantener actualizada su configuración local de webhooks ERP.

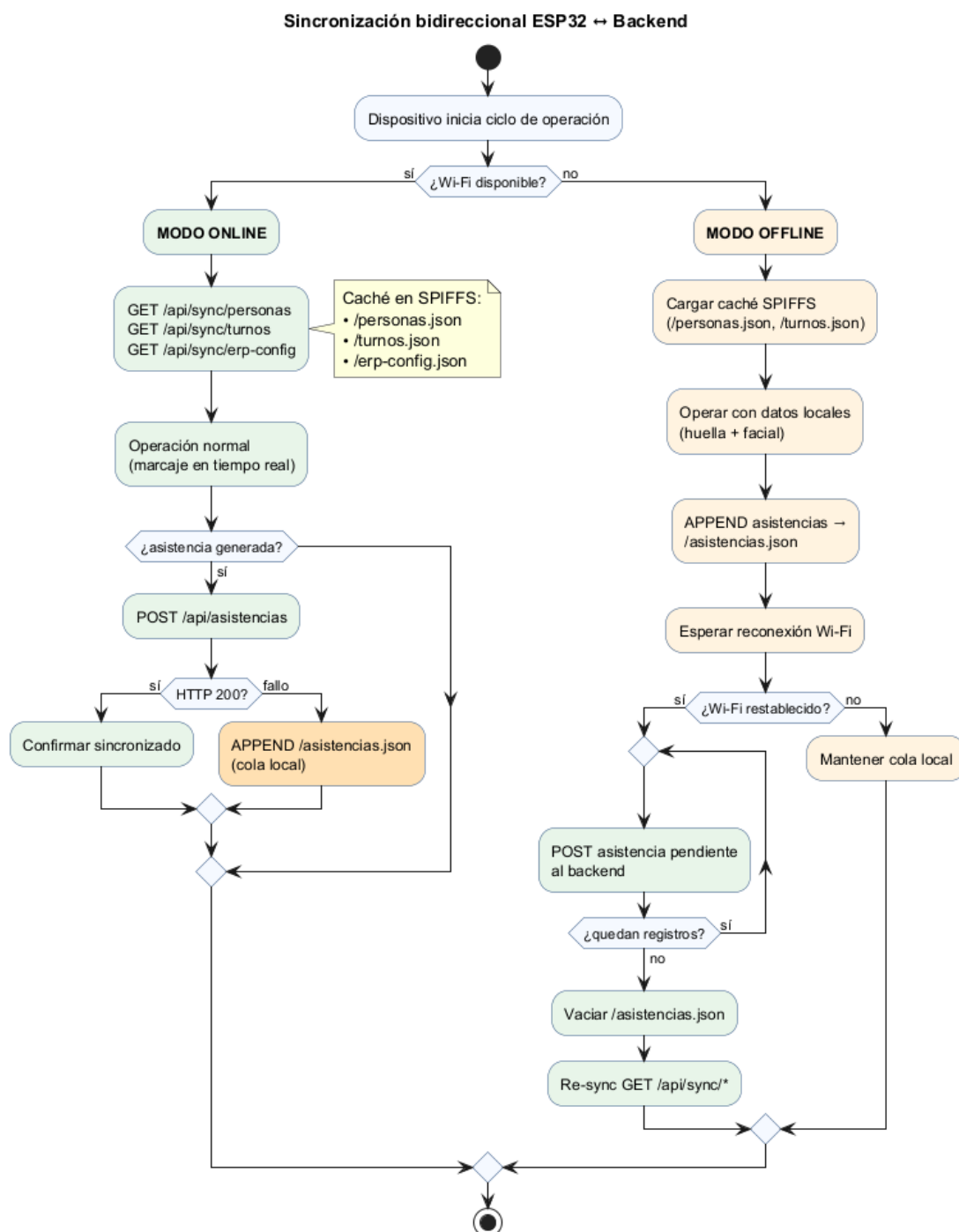


Figura 4.32: Diagrama de actividad: sincronización bidireccional entre los modos online y offline del ESP32-CAM.

Deduplicación en el backend El endpoint `POST /api/asistencias/sync` implementa una verificación previa a la inserción: para cada registro del lote, consulta si ya existe una asistencia de la misma persona (`persona_id`), mismo tipo (`entrada/salida`), mismo turno_id y misma fecha (`DATE(fecha_hora) = CURRENT_DATE`). Si existe, omite la

inserción. Esta verificación por `turno_id` permite que un trabajador registre múltiples asistencias en un mismo día si pertenece a distintos turnos (por ejemplo, jornada diurna y jornada nocturna), sin generar duplicados dentro del mismo turno. Garantiza idempotencia ante reenvíos.

Tablas de auditoría

- **sincronizacion_log**: Registra cada operación de sincronización con: dispositivo de origen, cantidad de registros enviados, cantidad procesada correctamente, estado y detalle.
- **logs_biometricos**: Registra cada operación biométrica con: persona, dispositivo, timestamp, tipo de operación (registro, verificación, identificación), resultado (éxito, fallo, duplicado, no_encontrado) e IP de origen.
- **eliminaciones_biometricas**: Registra cada eliminación de datos biométricos con: persona, embedding anterior (cifrado), ruta de la foto eliminada y usuario solicitante.

Panel de visualización de logs El backend expone dos conjuntos de endpoints para consulta de logs:

- **GET /api/logs**: Lista los registros de sincronización, con filtro por empresa para roles no-admin.
- **DELETE /api/logs**: Permite limpiar los logs (admin limpia todo, empleador limpia los de su empresa).
- El ESP32-CAM expone **GET /api/logs** que retorna el buffer de logs en formato HTML, y **/api/logs/clear** para limpiarlo.

Script de simulación de asistencia facial Se desarrolló `deteccion.py`, un script de línea de comandos que permite probar el flujo de reconocimiento facial sin necesidad del ESP32-CAM. El script escanea la carpeta `static/capturas_prueba/`, presenta un menú interactivo para seleccionar una foto, ejecuta el proceso de identificación 1:N contra la base de datos y registra la asistencia correspondiente. Es útil para pruebas de regresión rápida y demostraciones.

Implementación

Sincronización en el ESP32-CAM Las funciones de sincronización siguen un patrón común:

1. Verificar conectividad (`isOnline` y `WiFi.status() == WL_CONNECTED`).
2. Cargar los datos locales desde LittleFS.
3. Iterar sobre los registros con `sincronizado = false`.
4. Para cada uno, construir el payload JSON y enviarlo al backend vía HTTP POST.
5. Si el backend responde con éxito (HTTP 200/201), marcar `sincronizado = true` y actualizar el `backend_id` si corresponde.
6. Persistir los cambios en LittleFS.

Para las personas, además del envío de pendientes, se implementa la descarga completa desde el backend (`sincronizarPersonasDesdeBackend()`) que sobrescribe el archivo local si no hay registros locales pendientes, manteniendo el ESP32-CAM actualizado con los datos del servidor.

Campo colación en turnos Los turnos incorporaron los campos `con_colacion` (booleano), `colacion_inicio` (TIME) y `colacion_fin` (TIME) para modelar el horario de colación de los trabajadores, en cumplimiento del artículo 34 del Código del Trabajo chileno. En la tabla `turnos` de PostgreSQL se almacenan como TIME (o NULL si `con_colacion = FALSE`). Los endpoints CRUD de `routes/turnos.py` gestionan estos campos: al crear un turno, se envían `con_colacion`, `colacion_inicio` y `colacion_fin`; al actualizar, se usa `COALESCE` para preservar valores existentes si no se envían. Las consultas GET retornan los horarios como string o null según corresponda. El campo `turno_id` se agregó también a la tabla `asistencias` para asociar cada marcación al turno bajo el cual se registró, permitiendo el cálculo preciso de horas trabajadas descontando la colación.

Sincronización de asistencias por lote La función `sincronizarAsistencias()` en el ESP32-CAM es especialmente relevante porque maneja el caso más frecuente de datos offline: un trabajador marcó entrada y salida durante el día sin conexión. La función:

1. Carga `asistencias.json` y filtra las que tengan `sincronizado = false`.
2. Construye un JSON con estructura `{"registros": [...]}`.
3. Lo envía a POST `/api/asistencias/sync`.
4. Si el backend retorna HTTP 200, marca todos los registros enviados como sincronizados.

Sincronización en el backend El endpoint `POST /api/asistencias/sync` en `routes/asistencias.py` recibe el lote e itera sobre cada registro, aplicando la verificación de duplicados y realizando la inserción. También dispara el envío a ERP para cada asistencia nueva.

Watchdog y ping activo de dispositivos La detección de desconexión de dispositivos se implementó con tres capas complementarias:

1. **LWT (instantáneo):** Al conectarse al broker MQTT, el ESP32-CAM configura un mensaje Last Will en `esp32/lwt/<MAC>`. Si el dispositivo se desconecta abruptamente, el broker publica automáticamente el LWT y el backend marca el dispositivo como `inactivo` de inmediato.
2. **Ping activo (`device_pinger`):** Un hilo separado publica periódicamente (cada 30 segundos) un mensaje en el tópico `esp32/ping/<MAC>`. El ESP32-CAM, al recibirlo, responde publicando un heartbeat con el campo `"pong": true`. Si el backend no recibe respuesta dentro de 60 segundos, marca el dispositivo como inactivo. Este mecanismo funciona incluso si el mensaje LWT no fue entregado.
3. **Watchdog pasivo (`device_watchdog`):** Un hilo ejecutado cada 60 segundos realiza un barrido inicial al arrancar el backend (todos los dispositivos se marcan `inactivo`) y luego verifica periódicamente qué dispositivos no han enviado heartbeat en más de 90 segundos, marcándolos como inactivos.

Esta arquitectura de tres capas asegura que ninguna desconexión pase desapercibida: el LWT atrapa las desconexiones abruptas detectadas por el broker; el ping activo detecta silencios de red donde el LWT no alcanza a publicarse; y el watchdog pasivo actúa como respaldo final para cualquier caso no cubierto por las capas anteriores.

Herramienta de simulación facial Como complemento durante el desarrollo se implementó `deteccion.py`, un script de línea de comandos que permitió probar el flujo de reconocimiento facial sin necesidad del ESP32-CAM. El script se conectaba directamente a PostgreSQL, ejecutaba DeepFace (Facenet + detector configurable + anti-spoofing) sobre imágenes almacenadas en `static/capturas_prueba/`, y registraba asistencias de prueba en la base de datos. En las etapas finales del proyecto, esta funcionalidad de simulación fue absorbida por la suite de pruebas automatizadas (con mocks de DeepFace, OpenCV y PostgreSQL en Docker), que ofrece mayor repetibilidad y cobertura sin dependencia del sistema de archivos local.

Pruebas

Se diseñaron las siguientes pruebas para validar la sincronización diferida, los logs y la integración final:

- **Prueba de sincronización de asistencias offline:** Desconectar el ESP32-CAM de la red, registrar 5 marcaciones de entrada/salida para 2 personas diferentes, reconectar y ejecutar sincronización. Verificar que los 5 registros aparezcan en la base de datos del backend y que el archivo `asistencias.json` local los marque como sincronizados.
- **Prueba de idempotencia de sincronización:** Tras una sincronización exitosa, ejecutar nuevamente la sincronización sin nuevos registros. Verificar que no se inserten registros duplicados en el backend.
- **Prueba de sincronización de personas creadas offline:** Desconectar el dispositivo, registrar una nueva persona (con huella y datos), reconectar y sincronizar. Verificar que la persona aparezca en el backend con un ID definitivo (no `local-`).
- **Prueba de sincronización de turnos y asignaciones:** Desconectar el dispositivo, crear un turno y asignarlo a una persona existente, reconectar y sincronizar. Verificar que el turno y la asignación se repliquen en el backend y que los `backend_id` locales se actualicen correctamente.
- **Prueba de persistencia tras corte de energía:** Durante una sincronización en curso, cortar la alimentación del ESP32-CAM. Al reiniciar, verificar que los archivos JSON no estén corruptos y que los registros con `sincronizado = false` sigan estando disponibles para una nueva sincronización.
- **Prueba de duplicados por turno:** Registrar dos asistencias con la misma persona, tipo, turno y fecha. Verificar que el sistema rechace la segunda inserción por duplicado. Registrar asistencias con diferentes `turno_id` para un mismo día y verificar que ambas se registren sin conflictos.
- **Prueba de colación en turnos:** Crear un turno con colación habilitada (`con_colación = true`) especificando horario de colación. Verificar que los campos `colacion_inicio` y `colacion_fin` se almacenen correctamente como `TIME` en la base de datos. Crear un turno sin colación y verificar que ambos campos sean `NULL`.
- **Prueba de logs de sincronización:** Ejecutar 3 sincronizaciones con diferentes cantidades de registros. Verificar que en la tabla `sincronizacion_log` del backend aparezcan 3 entradas con los conteos correctos de registros enviados y procesados.

- **Prueba de logs biométricos:** Realizar operaciones de registro facial, identificación exitosa, identificación fallida y verificación. Verificar que cada operación genere una entrada en `logs_biometricos` con el tipo de operación y resultado correctos.
- **Prueba de watchdog:** Iniciar el backend, verificar que todos los dispositivos se marquen como inactivos. Conectar un ESP32-CAM, verificar que tras el primer heartbeat cambie a activo. Desconectar el ESP32-CAM y verificar que tras 90 segundos el watchdog lo marque como inactivo.
- **Prueba de ping activo:** Conectar un ESP32-CAM, verificar que el backend publique pings en `esp32/ping/<MAC>` y que el dispositivo responda. Detener el ESP32-CAM (corte de red) y verificar que el ping activo marque el dispositivo como inactivo antes de que el watchdog de 90 segundos actúe (tiempo de espera de 60 segundos).
- **Prueba de simulación facial automatizada:** Utilizar la suite de pruebas `test_routes_facial.py` y el emulador `test_identificacion_facial.py` para simular el flujo completo de identificación 1:N con imágenes sintéticas generadas en memoria, cubriendo casos de coincidencia exitosa, rostro no registrado y calidad insuficiente de imagen.
- **Prueba de flujo completo punta a punta:** Ejecutar el flujo completo: registrar persona con huella y rostro en el ESP32-CAM → sincronizar con backend → verificar que la persona aparezca en el panel web → marcar asistencia por huella offline → reconectar y sincronizar → verificar que la asistencia aparezca en el panel web y se envíe al ERP de prueba.

Pruebas automatizadas Además de las pruebas funcionales descritas, se implementó una suite de **334 pruebas automatizadas** para el backend y el firmware, utilizando pytest con PostgreSQL en Docker y mocks para DeepFace, OpenCV y MQTT. La suite se organiza en 19 archivos de prueba (10 en `Backend/tests/` y 9 en `Backend/tests/esp32_emulator/`) y alcanza una cobertura de código del **90%** sobre 4.881 líneas de código fuente, sin fallos en ninguna de las 334 pruebas.

Las pruebas cubren los flujos de todas las iteraciones: inicialización de la aplicación, esquema de base de datos, cifrado de embeddings, servicio de correo electrónico (con mock de SMTP), handlers MQTT, autenticación JWT con roles jerárquicos y flujo multi-empresa, endpoints faciales, CRUD de personas/turnos/asignaciones/asistencias, verificación de dispositivos, integraciones ERP con tolerancia a fallos de conexión y timeout, enrolamiento de dispositivos mediante PIN, sincronización offline, y logs de auditoría. Entre los módulos con mayor cobertura destacan el cifrado de embeddings

(100%), el servicio de correo (100%), la gestión de turnos (100%), la base de datos (99%) y el módulo de asignaciones (96%). El detalle completo de los casos de prueba se incluye en el Anexo correspondiente.

4.3 Resumen del capítulo

El capítulo 4 presenta el desarrollo completo del prototipo a lo largo de ocho iteraciones, siguiendo la metodología de prototipado evolutivo incremental. Se documenta la integración del hardware (ESP32-CAM, lector AS608, sensor PIR, cámara OV2640), la implementación del almacenamiento local con LittleFS y JSON, el desarrollo del backend con Flask y PostgreSQL (14 tablas con soporte multi-tenant), la comunicación mediante HTTP y MQTT, el reconocimiento facial con DeepFace (modelo Facenet, detector configurable MTCNN/RetinaFace y anti-spoofing), el cifrado de embeddings biométricos con Fernet, la autenticación JWT con roles jerárquicos, el enrolamiento seguro de dispositivos mediante PIN, el sistema antifraude multicapa (PIR + flash PWM + anti-spoofing + cooldown), el panel web para la gestión del dispositivo con streaming SSE de estado en tiempo real, la integración con ERP externos vía webhooks con field mapping, los mecanismos de sincronización diferida con logs de auditoría, y la detección de desconexión mediante un sistema de tres capas (LWT, ping activo y watchdog pasivo).

Cada iteración incluyó su respectivo análisis de requisitos, diseño de la solución, implementación detallada y un plan de pruebas de integración para validar el correcto funcionamiento de los componentes desarrollados. Adicionalmente, se documenta la suite de 334 pruebas unitarias automatizadas con pytest, que alcanzó una cobertura de código del 90% sobre 4.881 líneas del backend, incluyendo pruebas de caja negra sobre los 28 endpoints de la API REST y verificación exhaustiva de todos los caminos de error.

5. Análisis de resultados

En el siguiente capítulo se presentan los resultados obtenidos tras la implementación y evaluación del prototipo, organizados según las dimensiones definidas en la metodología de evaluación del capítulo 3. Se analiza el costo final del prototipo, los puntajes de usabilidad SUS obtenidos de usuarios técnicos y no técnicos, las métricas de desempeño del reconocimiento facial y los resultados de las pruebas de estrés offline, con el objetivo de determinar la viabilidad técnica y operativa del sistema propuesto.

5.1 Costo total del prototipo

La siguiente tabla desglosa los costos de los componentes utilizados en el prototipo final, considerando precios de mercado local e importación según corresponda.

Componente	Cantidad	Costo (USD)
ESP32-CAM (cámara OV2640 + microcontrolador)	1	Pendiente
Lector de huellas AS608	1	Pendiente
Sensor PIR HC-SR501	1	Pendiente
Fuente de alimentación 5V / 2A	1	Pendiente
Cables y conectores Dupont	1 kit	Pendiente
Placa de pruebas (protoboard)	1	Pendiente
Carcasa / soporte impreso en 3D	1	Pendiente
Total		Pendiente

Cuadro 5.1: Desglose de costos del prototipo (pendiente de completar con precios de mercado)

5.2 Resultados de encuestas SUS

Se aplicó el cuestionario System Usability Scale estándar a dos grupos de usuarios: *no técnicos* (trabajadores y empleadores) y *técnicos* (desarrolladores e integradores ERP).

Ambos grupos respondieron las mismas 10 preguntas en escala Likert de 5 puntos (1 = Totalmente en desacuerdo, 5 = Totalmente de acuerdo). La puntuación SUS se calcula según el método estándar descrito por Brooke [31]: para cada ítem impar se resta 1, para cada ítem par se resta de 5, y el resultado se multiplica por 2.5, obteniendo un valor entre 0 y 100.

5.2.1 Usuarios no técnicos

N.º	Pregunta	Promedio	Tipo
1	Creo que me gustaría usar este sistema con frecuencia.	Pendiente	Positiva
2	Encontré el sistema innecesariamente complejo.	Pendiente	Invertida
3	Pensé que el sistema era fácil de usar.	Pendiente	Positiva
4	Creo que necesitaría apoyo de una persona técnica para poder usar este sistema.	Pendiente	Invertida
5	Consideré que las funciones del sistema estaban bien integradas.	Pendiente	Positiva
6	Pensé que había demasiada inconsistencia en el sistema.	Pendiente	Invertida
7	Imagino que la mayoría de las personas aprenderían a usar este sistema rápidamente.	Pendiente	Positiva
8	Encontré el sistema muy engorroso de usar.	Pendiente	Invertida
9	Me sentí seguro al usar el sistema.	Pendiente	Positiva
10	Necesité aprender muchas cosas antes de poder empezar a usar este sistema.	Pendiente	Invertida
Puntuación SUS total			Pendiente

Cuadro 5.2: Resultados de encuesta SUS - usuarios no técnicos (pendiente de aplicar encuestas)

5.2.2 Usuarios técnicos

N.º	Pregunta	Promedio	Tipo
1	Creo que me gustaría usar este sistema con frecuencia.	Pendiente	Positiva
2	Encontré el sistema innecesariamente complejo.	Pendiente	Invertida
3	Pensé que el sistema era fácil de usar.	Pendiente	Positiva
4	Creo que necesitaría apoyo de una persona técnica para poder usar este sistema.	Pendiente	Invertida
5	Consideré que las funciones del sistema estaban bien integradas.	Pendiente	Positiva
6	Pensé que había demasiada inconsistencia en el sistema.	Pendiente	Invertida
7	Imagino que la mayoría de las personas aprenderían a usar este sistema rápidamente.	Pendiente	Positiva
8	Encontré el sistema muy engorroso de usar.	Pendiente	Invertida
9	Me sentí seguro al usar el sistema.	Pendiente	Positiva
10	Necesité aprender muchas cosas antes de poder empezar a usar este sistema.	Pendiente	Invertida
Puntuación SUS total			Pendiente

Cuadro 5.3: Resultados de encuesta SUS - usuarios técnicos (pendiente de aplicar encuestas)

5.2.3 Análisis comparativo

Se presentará aquí la comparación entre ambos grupos, el promedio general y la interpretación según la escala de Bangor et al. [32] (no aceptable < 50, marginal 50–70, aceptable > 70).

5.3 Métricas de reconocimiento facial

5.3.1 Tiempo de respuesta del ESP32-CAM

Se midió el tiempo total del ciclo de marcación facial: captura de imagen, codificación JPEG, transmisión HTTP (octet-stream) al endpoint `/api/facial/identificar`, procesamiento DeepFace en backend y respuesta al dispositivo. Las mediciones se realizaron sobre 20 ciclos en condiciones de laboratorio.

Métrica	Valor
Tiempo promedio de captura + codificación	Pendiente
Tiempo promedio de transmisión HTTP (POST octet-stream)	Pendiente
Tiempo promedio de procesamiento DeepFace (identificación 1:N)	Pendiente
Tiempo total promedio del ciclo	Pendiente
Desviación estándar	Pendiente

Cuadro 5.4: Tiempos de respuesta del ciclo de marcación facial (pendiente de medir)

5.3.2 Precisión del reconocimiento

Se evaluó la precisión del sistema con un conjunto de pruebas de registro e identificación facial bajo distintas condiciones.

Métrica	Valor
Tasa de reconocimiento exitoso (True Positive Rate)	Pendiente
Tasa de falsos positivos	Pendiente
Tasa de falsos negativos	Pendiente
Efectividad del anti-spoofing (fotografías / pantallas rechazadas)	Pendiente
Precisión con iluminación favorable	Pendiente
Precisión con iluminación desfavorable	Pendiente

Cuadro 5.5: Métricas de precisión del reconocimiento facial (pendiente de evaluar)

5.4 Resultados de pruebas de estrés offline

Se sometió el dispositivo a tres escenarios de conectividad para evaluar la confiabilidad del almacenamiento local y la sincronización diferida.

5.4.1 Escenario 1: Desconexión total

El dispositivo operó sin conexión a internet durante un período Pendiente de definir, registrando Pendiente de definir marcaciones. Al restablecer la conectividad, se verificó la sincronización del 100% de los registros.

5.4.2 Escenario 2: Conectividad intermitente

Se simuló Pendiente de definir ciclos de conexión/desconexión con intervalos aleatorios. Se midió la cantidad de reintentos automáticos y la consistencia del log de eventos.

5.4.3 Escenario 3: Reconexión prolongada

Tras Pendiente de definir días de operación offline, se reconectó el dispositivo y se midió el tiempo total de sincronización y la integridad de los datos transferidos.

Métrica	Sin conexión	Intermitente	Reconexión
Registros generados offline	Pendiente	Pendiente	Pendiente
Registros sincronizados correctamente	Pendiente	Pendiente	Pendiente
Porcentaje de sincronización	Pendiente	Pendiente	Pendiente
Tiempo de sincronización	Pendiente	Pendiente	Pendiente
Reintentos automáticos	N/A	Pendiente	Pendiente

Cuadro 5.6: Resultados de pruebas de estrés offline (pendiente de ejecutar)

5.5 Resumen del capítulo

El capítulo 5 presentó los resultados obtenidos en las cuatro dimensiones de evaluación definidas en la metodología. El análisis de costos, usabilidad, desempeño biométrico y confiabilidad offline permite validar el cumplimiento de los objetivos específicos planteados en el capítulo 1 y determinar la viabilidad técnica y operativa del prototipo propuesto.

6. Bibliografía

1. Ministerio del Trabajo y Previsión Social, Subsecretaría del Trabajo, Dirección del Trabajo. (2025, abril 25). Resolución 38 Exenta: Establece requisitos obligatorios y procedimiento de autorización para los sistemas electrónicos de registro y control de la asistencia y determinación de las horas de trabajo (modificada por Resolución 9453 Exenta). Biblioteca del Congreso Nacional de Chile. <https://www.bcn.cl/leychile/navegar?idNorma=1203415&idParte=10499950&idVersion=04-25>
2. Defontana Chile. (2024, mayo 9). ¿Qué es la Resolución Exenta N.º 38 en la gestión de personas? Defontana Blog. <https://www.defontana.com/blog/cl/que-es-la-resolucion-exenta-38-en-la-gestion-de-personas>
3. Redacción APD. (2025, febrero 3). Qué es un software ERP y para qué sirve: Ventajas y desventajas. APD. <https://www.apd.es/ventajas-y-desventajas-sistema-ERP>
4. Bernal, A. (2025, marzo 5). Control de asistencia integrado con ERP: Beneficios para empresas. GeoVictoria. <https://www.geovictoria.com/es-cl/blog/control-de-asistencia-integrado-con-ERP-beneficios-para-empresas>
5. Fernández, A. (2025, abril). Reloj control: Qué es, para qué sirve y qué debes tener en cuenta. Nubox Blog. <https://blog.nubox.com/empresas/reloj-control>
6. Pachon Robayo, S. (s.f.). Monitoreo de transmisión de video en vivo utilizando ESP32-CAM y Ubidots. Ubidots Centro de Ayuda. Recuperado el 17 de octubre de 2025, de <https://help.ubidots.com/es/articles/7156830-monitoreo-de-transmision-de-video-en-vivo-utilizando-ESP32-CAM-y-ubidots>
7. Amazon Web Services. (s.f.). ¿Qué es una API? Explicación de qué es una interfaz de programación de aplicaciones. Recuperado el 9 de noviembre de 2025, de <https://aws.amazon.com/es/what-is/api>

8. Ken, A. (2023, abril 10). Backend: ¿Qué es y para qué sirve? Recuperado de <https://blog.hubspot.es/website/backend>
9. IONOS. (s.f.). ¿Qué es un frontend? Definición y explicación. Recuperado el 28 de octubre de 2025, de <https://www.ionos.com/es-us/digitalguide/paginas-web/creacion-de-paginas-web/que-es-el-frontend>
10. Amazon Web Services. (s.f.). ¿Qué es MQTT? Explicación del protocolo MQTT. Recuperado el 6 de octubre de 2025, de <https://aws.amazon.com/es/what-is/mqtt>
11. Erickson, J. (2024, abril 4). ¿Qué es JSON? Oracle América Latina. Recuperado el 14 de noviembre de 2025, de <https://www.oracle.com/latam/database/what-is-json>
12. Last Minute Engineers. (s.f.). Empezando con ESP32-CAM: Guía para principiantes. Recuperado el 19 de octubre de 2025, de <https://lastminuteengineers.com/getting-started-with-ESP32-CAM>
13. UNIT Electronics. (s.f.). Cómo utilizar el lector de huella dactilar AS608. Recuperado el 22 de noviembre de 2025, de <https://blog.uelectronics.com/tarjetas-desarrollo/como-utilizar-el-lector-de-huella-dactilar-AS608>
14. MDN Web Docs. (s.f.). Generalidades del protocolo HTTP. Recuperado el 30 de octubre de 2025, de <https://developer.mozilla.org/es/docs/Web/HTTP/Guides/Overview>
15. Paessler GmbH. (s.f.). ¿Qué es MQTT? - IT Explained. Recuperado el 2 de noviembre de 2025, de <https://www.paessler.com/es/it-explained/mqtt>
16. Docker Inc. (s.f.). ¿Qué es un contenedor? Recuperado el 21 de octubre de 2025, de <https://www.docker.com/resources/what-container>
17. Sunny Valley Cyber Security Inc. (2025, febrero 3). Definición, tipos y ejemplos de servidor de base de datos. Zenarmor. Recuperado el 10 de noviembre de 2025, de <https://www.zenarmor.com/docs/network-basics/what-is-database-server>
18. GeoVictoria. (s.f.). Dispositivos GeoVictoria. Recuperado el 12 de octubre de 2025, de <https://info.geovictoria.com/es-co/dispositivos>
19. ZKTeco Latinoamérica. (s.f.). Tiempo y asistencia. Recuperado el 5 de noviembre de 2025, de <https://zktecolatinoamerica.com/categoria-producto/tiempo-y-asistencia>
20. Talana. (s.f.). Software de asistencia de personal en Chile. Recuperado el 7 de noviembre de 2025, de <https://web.talana.com/software-de-asistencia-de-personal>

21. Buk. (s.f.). Control de asistencia: Gestiona tiempos y alertas. Recuperado el 26 de octubre de 2025, de <https://www.buk.cl/productos/administracion/control-de-asistencia>
22. Defontana. (s.f.). Software y sistema ERP. Recuperado el 18 de noviembre de 2025, de <https://www.defontana.com/cl>
23. Softland Inversiones S.L. (s.f.). Softland Chile: Sistema ERP, contable y de administración. Recuperado el 9 de octubre de 2025, de <https://www.softland.cl>
24. Drumond, C. (s.f.). Scrum: Qué es, cómo funciona y cómo empezar. Atlassian. Recuperado el 3 de noviembre de 2025, de <https://www.atlassian.com/es/agile/scrum>
25. Teach-ICT. (s.f.). Prototipado evolutivo. Recuperado el 22 de octubre de 2025, de https://www.teach-ict.com/as_a2_ict_new/ocr/A2_G063/331_systems_cycle/prototyping_R
26. GeeksforGeeks. (2025, julio 11). Modelo de procesos incrementales - Ingeniería de software. Recuperado el 20 de noviembre de 2025, de <https://www.geeksforgeeks.org/software-engineering/software-engineering-incremental-process-model>
27. Pressman, R. S., & Maxim, B. R. (2021). Ingeniería de software: Un enfoque práctico (9.^a ed.). McGraw-Hill Interamericana.
28. IBM Corporation. (2025). ¿Qué son las pruebas de integración? IBM Think. <https://www.ibm.com/es-es/think/topics/integration-testing>
29. International Organization for Standardization. (2018). ISO 9241-11: Ergonomics of human-system interaction - Part 11: Usability: Definitions and concepts.
30. Nielsen, J. (1993). Usability engineering. Morgan Kaufmann.
31. Brooke, J. (1996). SUS: A “quick and dirty” usability scale. En P. W. Jordan et al. (Eds.), Usability evaluation in industry (pp. 189-194). Taylor & Francis.
32. Bangor, A., Kortum, P. T., & Miller, J. T. (2008). An empirical evaluation of the System Usability Scale. *International Journal of Human-Computer Interaction*, 24(6), 574-594.
33. Microsoft. (2025, julio 10). Introducción a Internet de las cosas (IoT) de Azure. Microsoft Learn. <https://learn.microsoft.com/es-es/azure/iot/iot-introduction>
34. Atlassian. (s.f.). Reliability vs. availability: Key metrics for system performance. Recuperado el 11 de noviembre de 2025, de <https://www.atlassian.com/incident-management/kpis/reliability-vs-availability>

35. Perry, W. E. (2006). Effective methods for software testing (3.^a ed.). Wiley.
36. Kaspersky Lab. (2025). ¿Qué es la biometría? Definición y tipos. <https://latam.kaspersky.com/center/definitions/biometrics>
37. Espressif Systems. (2025). SPIFFS - SPI Flash File System. <https://docs.espressif.com/projects/idf/en/stable/esp32/api-reference/storage/spiffs.html>
38. Amazon Inc. (2025). What is a RESTful API? <https://aws.amazon.com/what-is/restful-api>
39. Red Hat Inc. (2025). ¿Qué son los microservicios? <https://www.redhat.com/es/topics/microserv>
40. EMQX Technologies. (2025). The ultimate guide to MQTT brokers. <https://www.emqx.com/en/ultimate-guide-to-mqtt-broker-comparison>

Anexo A: Preguntas complementarias de usabilidad

Nota: Las siguientes preguntas complementarias no forman parte del cuestionario SUS estándar (Brooke, 1996). Se aplicaron a cada grupo de usuarios por separado con el objetivo de obtener información contextual adicional sobre aspectos específicos del prototipo. Sus resultados se analizan de forma descriptiva y no intervienen en el cálculo de la puntuación SUS.

A.1 Preguntas complementarias para usuarios no técnicos

Tareas (Sí / No)

- ¿Lograste configurar la red WiFi del dispositivo sin necesitar asistencia?
- ¿Lograste configurar el endpoint del backend desde la interfaz embebida (sin editar archivos de configuración) sin necesitar asistencia?
- ¿Pudiste encontrar y visualizar los registros de asistencia en el panel web sin ayuda?
- ¿Pudiste identificar la imagen asociada a una marcación específica sin ayuda?

Percepción (escala 1 a 5)

- Me resultó fácil encontrar la opción de configuración de WiFi en la interfaz embebida.
- Tuve que pensar bastante para entender la estructura del panel web al buscar registros de asistencia.
- El uso de los filtros por fecha, usuario o tipo de autenticación me pareció intuitivo.

- Me costó saber, a partir de la información mostrada, si el dispositivo estaba funcionando online u offline.
- La información mostrada en la tabla de registros fue clara.
- Encontré difícil leer la información de la tabla de registros (tamaño, contraste, orden visual).
- Me siento capaz de usar la interfaz completa (embebida + panel web) sin instrucción adicional.
- Sentiría que necesito ayuda de alguien con más conocimientos técnicos para usar esta interfaz regularmente.

Preguntas abiertas

- ¿En algún momento sentiste confusión respecto a qué botón o sección debías utilizar para continuar una tarea? ¿Cuál?
- ¿Qué cambiarías o mejorarías de la interfaz embebida o del panel web?

A.2 Preguntas complementarias para usuarios técnicos

Tareas (Sí / No)

- ¿Pudiste consumir los endpoints del prototipo (Postman, cURL o aplicación propia) sin complicaciones?
- ¿Lograste modificar el endpoint configurado en el dispositivo para redirigir los datos hacia tu propio servidor sin asistencia?
- ¿Lograste integrar o reutilizar las imágenes enviadas por la ESP32-CAM en un procesamiento posterior?

Percepción (escala 1 a 5)

- La documentación técnica del backend contenía la información necesaria para comenzar a integrar la API sin asistencia externa.
- Tuve que recurrir a prueba y error, más que a la documentación, para entender cómo funcionaban los endpoints.
- La estructura JSON entregada por el dispositivo es clara y fácil de mapear a otros sistemas.

- Me costó identificar los campos esenciales del registro (timestamp, usuario, métodos biométricos, estado de sincronización).
- Los mensajes de error entregados por el backend fueron comprensibles y útiles para depurar problemas de integración.
- Encontré inconsistencias entre los distintos endpoints del backend (nombres, métodos HTTP o estructura de respuesta).
- Creo que esta API podría integrarse a un sistema ERP real sin necesidad de modificaciones mayores.
- Necesitaría hacer cambios significativos en mi propio sistema o flujo de trabajo para poder integrar esta API.

Preguntas abiertas

- ¿Tuviste dificultades técnicas significativas durante la integración? Descríbelas.
- ¿Qué cambiarías o mejorarías de la API o del backend?